

Spark Python

Septembre 2026



Objectifs

- Comprendre Spark, son architecture et ses composants.
- Comprendre les traitements de données Spark (transformations et actions).
- Utiliser Spark SQL pour lire, explorer, analyser, manipuler et écrire des données tabulaires.
- Utiliser Spark (Structured Streaming) pour le traitement de flux en temps réel.
- Utiliser Spark (SparkML et udf) pour l'apprentissage automatique (entraînement et inférence).
- Découvrir les bases de la manipulation de graphes avec Spark (Graphframes).

Programme



- 01 **Introduction à Spark**
- 02 **Fonctionnalités essentielles de Spark**
- 03 **Spark SQL**
- 04 **Streaming avec Spark**
- 05 **Machine Learning avec Spark**
- 06 **Graphes avec Spark**

Avant-propos

Ce support est pensé à la fois pour supporter la présentation mais également pour être utilisé comme un premier document de référence après la formation.

Certains slides ne seront pas détaillés lors de la formation (en particulier ceux qui présentent une grande quantité de texte), mais vous pourrez vous y reporter pour plus de détails si vous la retravaillez.

01

Introduction à Spark

Introduction à Spark

Quantité de données produites dans le monde



A votre avis, combien d'emails** sont envoyés dans le monde
en l'espace d'**une minute** ?**

De **SMS ?**



Introduction à Spark

Quantité de données produites dans le monde

Que se passe-t-il sur Internet en 1 minute ? (2021)



Crédit : L. Lewis

Introduction à Spark

Quantité de données produites dans le monde

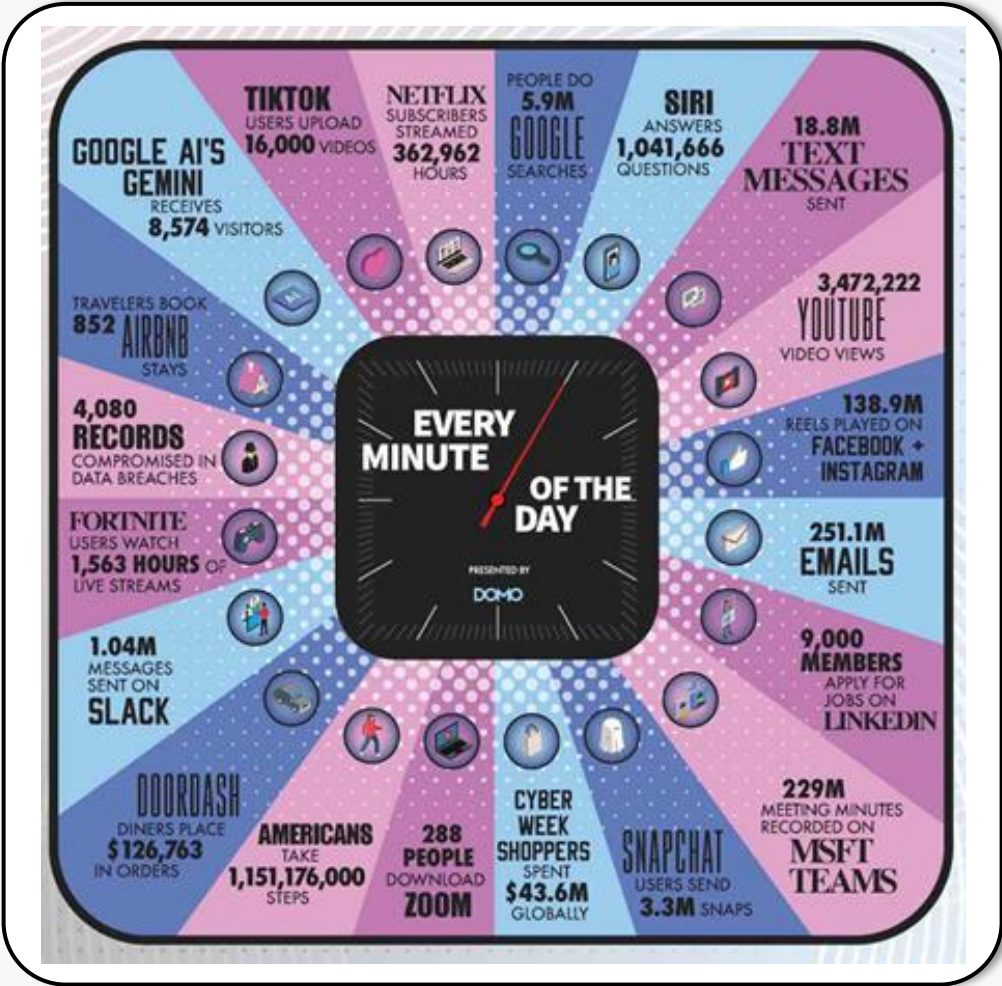
Que se passe-t-il sur Internet en 1 minute ?

Crédit : L. Lewis, Domo

2021



2024



Introduction à Spark

Ordres de grandeur

Unité	Abb.	Taille en octets	Exemple
1 octet		1	4 octets $\simeq$ Un caractère au format utf-8
1 kilooctet	ko	10^3	20 ko $\simeq$ Mots prononcés par une personne, par jour
1 mégaoctet	Mo	10^6	3 Mo $\simeq$ Texte d'une édition quotidienne du New York Times
1 gigaoctet	Go	10^9	1 Go $\simeq$ Mots prononcés par une personne durant une vie
1 téraoctet	To	10^{12}	593 To $\simeq$ Mots prononcés par l'ensemble de la population mondiale, par jour
1 pétaoctet	Po	10^{15}	4 Po $\simeq$ Données générées par Facebook, par jour
1 exaoctet	Eo	10^{18}	605 Eo $\simeq$ Données générées par le télescope SKA, par jour
1 zétaoctet	Zo	10^{21}	40 Zo $\simeq$ Données générées par le web en 2020, par an

Introduction à Spark

Une définition classique du Big Data

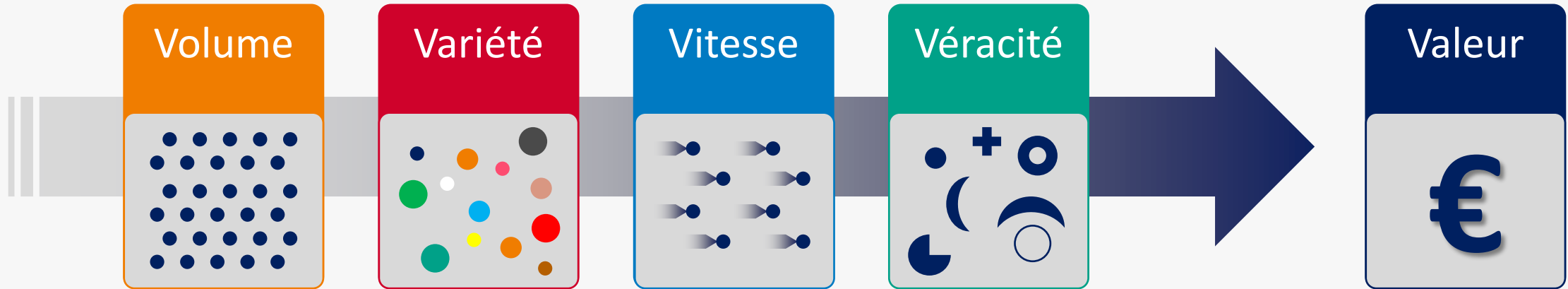


Les "big data" sont des informations de grand volume, de grande vitesse et/ou de grande variété qui exigent des formes rentables et innovantes de traitement de l'information permettant d'améliorer la compréhension, la prise de décision et l'automatisation des processus.

<https://www.gartner.com/en/information-technology/glossary/big-data>

Introduction à Spark

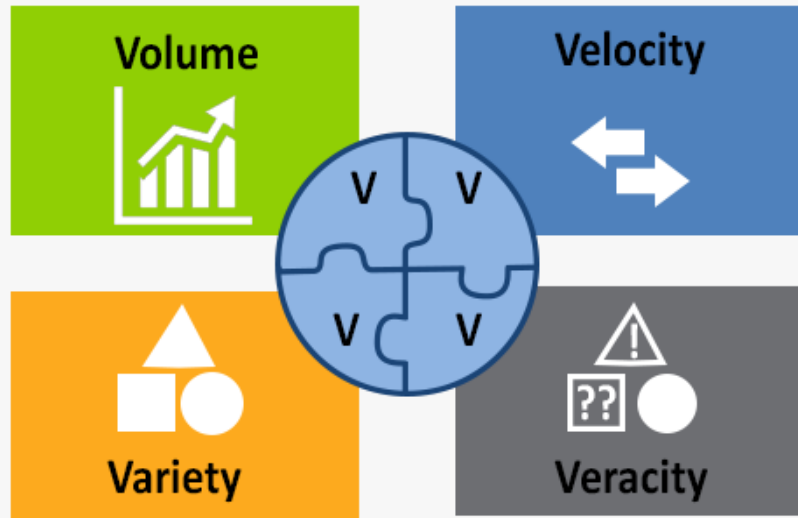
Les 3/4/5 V



- **Volume** : Croissance continue de la quantité de données à exploiter, souvent en téraoctets voire en pétaoctets.
- **Variété** : Traitement des données sous forme structurée et non structurée, devant faire l'objet d'une analyse dans leur ensemble (bases de données relationnelles, textes, données de capteurs, sons, vidéos...).
- **Vélocité** : Utilisation des données en temps réel dans de multiples applications (détection de fraude...).
- **Véracité** : Gestion de la fiabilité et de la véracité des données prédictives.

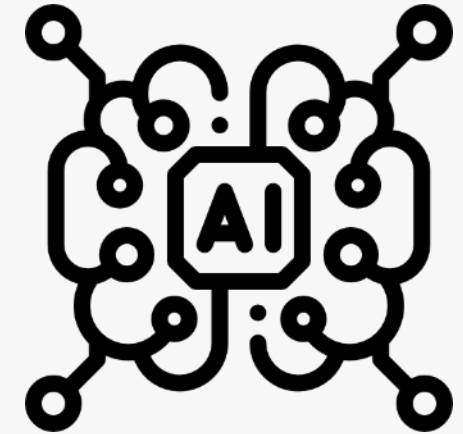
Introduction à Spark

Inscription dans la révolution numérique



Adoption par secteurs d'activité

	Commerce	33%
	Transports	30%
	Services Financiers	30%
	Distribution	29%
	Santé	25%
	Manufacturing	24%
	Services	24%
	Utilities	22%
	Télécoms	22%
	Secteur Public	17%
	Education	16%



La capacité à recueillir, stocker, gérer et analyser de grandes quantités de données est devenue essentielle pour les entreprises et les organisations qui souhaitent rester compétitives et innovantes. Le Big Data alimente des avancées technologiques majeures, contribuant à des découvertes scientifiques, à l'amélioration des produits et services, et à la transformation des modèles commerciaux. En ce sens, le Big Data est plus qu'une mode, il est une réalité fondamentale de l'ère numérique.

Introduction à Spark

Spark dans tout cela

- Le développement du Web et d'Internet a engendré une explosion de la quantité de données informatiques produites.
- Cette croissance est si considérable que les ordres de grandeur associés sont difficiles à appréhender.
- La problématique du Big Data se pose souvent quand on parle de passer à l'échelle ("scaler") une activité, en lien avec ce développement.
- Le stockage, le traitement et l'analyse de ces données massives nécessitent une approche spécifique.
- Cette approche implique la parallélisation du stockage et des traitements sur plusieurs machines distinctes.
- MapReduce, Hadoop MapReduce et Spark sont des moteurs de traitement adaptés aux volumes de données massifs, qui sont basés sur une approche de traitement distribué.



Introduction à Spark

Projet SETI@home



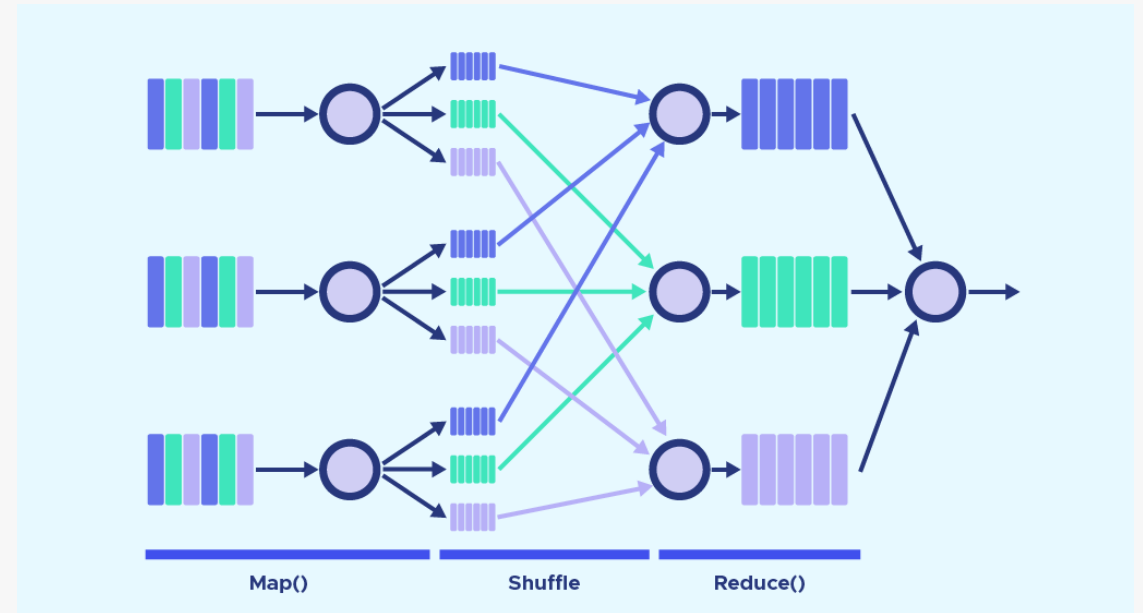
- Initiative de calcul distribué dans le domaine de la recherche en astronomie, plus spécifiquement dans la recherche de signaux extraterrestres.
 - Vise à analyser les signaux radio provenant de l'espace à la recherche de preuves d'une intelligence extraterrestre.
 - Utilise les données recueillies par le radiotélescope d'Arecibo et d'autres radiotélescopes à travers le monde.
- ➡ Ces données sont découpées en petits morceaux appelés "unités de travail".
 - ➡ Les unités de travail sont ensuite distribuées à un grand nombre d'ordinateurs personnels via Internet.
 - ➡ Les ordinateurs personnels analysent ces données lorsqu'ils sont inactifs, en utilisant une application logicielle fournie par le projet SETI@home.
 - ➡ Une fois l'analyse terminée, les résultats sont renvoyés au projet SETI@home pour être examinés et interprétés par les chercheurs.

Introduction à Spark

De MapReduce à Spark, premières descriptions

MapReduce :

- MapReduce est un modèle de programmation et un framework de traitement de données largement utilisé pour le traitement parallèle de gros volumes de données sur des clusters de serveurs.
- Il divise les tâches de traitement de données en deux étapes principales : map et reduce.
- L'étape de map consiste à traiter et à transformer les données en paires clé-valeur.
- L'étape de reduce agrège et traite les données résultantes de l'étape de map.
- MapReduce est efficace pour le traitement de données structurées et semi-structurées, mais peut être relativement lent pour les traitements itératifs et interactifs.



Introduction à Spark

De MapReduce à Spark, premières descriptions

Hadoop MapReduce :

- Hadoop MapReduce est une implémentation du modèle MapReduce développée par la fondation Apache Hadoop.
- Il fournit une infrastructure de calcul distribué pour le traitement de données sur des clusters de serveurs.
- Hadoop MapReduce est conçu pour fonctionner sur de grands ensembles de données stockés dans le système de fichiers distribué Hadoop HDFS (Hadoop Distributed File System).
- Il est extensible, tolérant aux pannes et capable de traiter des données de manière scalable.



Introduction à Spark

De MapReduce à Spark, premières descriptions

Spark :

- Spark est un framework de traitement de données en mémoire conçu pour le traitement rapide et efficace de gros volumes de données.
- Contrairement à MapReduce, Spark conserve les données en mémoire chaque fois que cela est possible, ce qui réduit considérablement les temps d'accès aux données.
- Spark prend en charge un large éventail de charges de travail, y compris le traitement de flux de données, le traitement de graphes, l'apprentissage automatique et le traitement interactif.
- Il fournit des APIs faciles à utiliser pour le développement d'applications de traitement de données en Python, Scala, Java et R.



Introduction à Spark

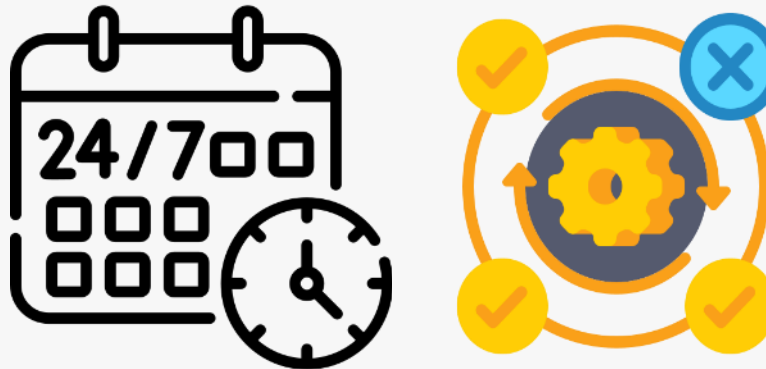
Systèmes distribués : notions importantes



Evolutivité (« scalabilité »)



Latence



Disponibilité & tolérance aux pannes

Introduction à Spark

Evolutivité

- Capacité d'un système à gérer efficacement une **charge de travail croissante** en ajoutant des ressources matérielles ou en élargissant son infrastructure sans compromettre ses **performances**.

- **Enjeux :**

- Répondre à la croissance des besoins**

- L'évolutivité vise à permettre au système de répondre à la demande croissante des utilisateurs et à s'adapter à l'augmentation du volume de données ou de trafic.

- Maintenir la performance**

- L'évolutivité doit permettre au système de maintenir des performances acceptables même lorsque la charge de travail augmente, en garantissant des temps de réponse rapides et une disponibilité élevée.

- Optimiser l'utilisation des ressources**

- L'évolutivité cherche à utiliser efficacement les ressources matérielles disponibles, en répartissant équitablement la charge de travail sur les différentes unités de traitement.

Introduction à Spark

Evolutivité

- **Caractéristiques de l'évolutivité dans les systèmes distribués :**

- Scalabilité et performance linéaire**

Idéalement, un système évolutif devrait présenter une amélioration linéaire de ses performances lorsque des ressources supplémentaires sont ajoutées. Cela signifie que le système devrait devenir proportionnellement plus rapide avec l'ajout de ressources.

- Flexibilité géographique**

L'évolutivité doit permettre la mise en place de centres de données supplémentaires dans différentes régions géographiques afin de réduire la latence et d'améliorer la disponibilité pour les utilisateurs répartis sur différentes zones géographiques.

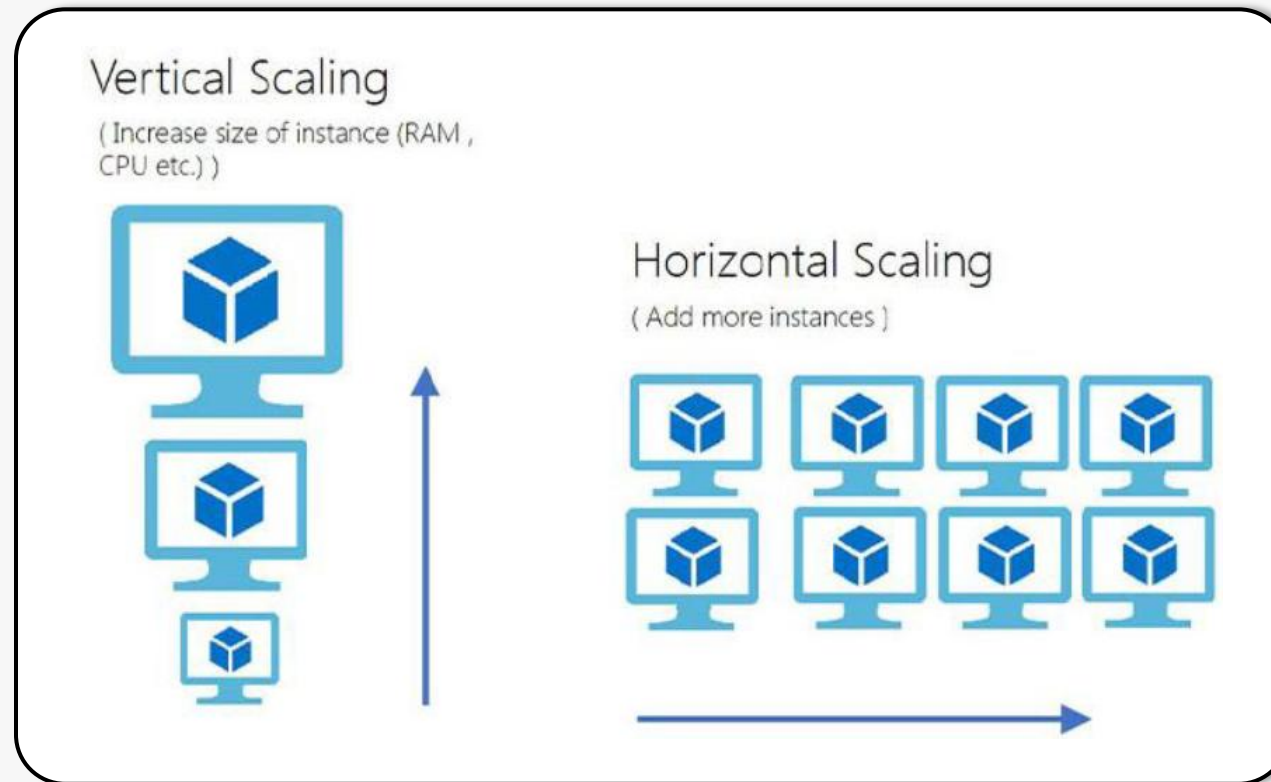
- Faible coût administratif**

L'évolutivité devrait être réalisée de manière à minimiser les coûts administratifs associés à la gestion et à la maintenance du système distribué dans son ensemble.

Introduction à Spark

Scaling vertical et horizontal

L'évolutivité peut être réalisée à travers la mise à niveau des ressources matérielles d'une seule unité (scaling vertical) ou en ajoutant des unités de traitement supplémentaires (scaling horizontal).



Introduction à Spark

Scaling vertical et horizontal

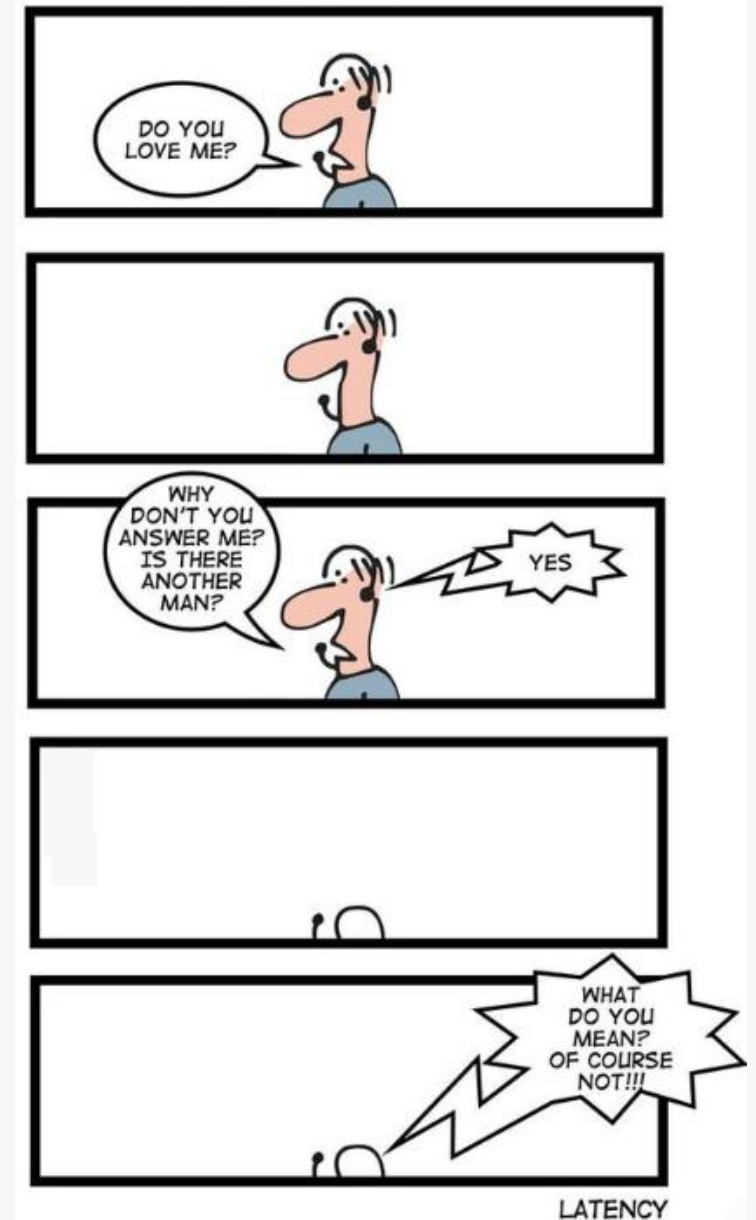
Caractéristique	Scaling vertical (« up »)	Scaling horizontal (« out »)
Évolutivité	Extension de la capacité d'une seule machine	Extension de la capacité par ajout de machines
Coût matériel	Élevé	Relativement plus bas
Limites	Peut atteindre des limites en termes de performances	Moins de limites en termes de performances
Exemples	Ajout de mémoire, augmentation de la puissance CPU	Ajout de serveurs, clusters
Flexibilité	Moins flexible en termes d'ajustement dynamique	Plus flexible, permet une croissance plus dynamique

Introduction à Spark

Latence

Représente le délai ou le temps écoulé entre l'envoi d'une requête et la réception de la réponse associée.

- ➡ Il s'agit d'un aspect critique de la performance des systèmes distribués, car elle influence directement l'expérience utilisateur et la réactivité globale du système.
- ➡ Exemple : application de commerce électronique qui doit enregistrer des informations sur les produits et récupérer ces informations ultérieurement pour afficher les détails des produits aux utilisateurs. Quand l'écriture devient-elle disponible à la lecture ?



Crédit : Geek & Poke

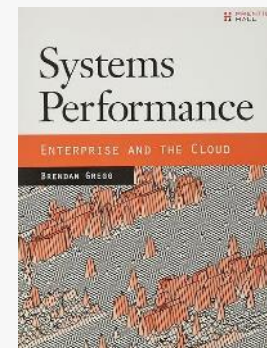
Introduction à Spark

Latence



Table 2.2 Example Time Scale of System Latencies

Event	Latency	Scaled
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access (DRAM, from CPU)	120 ns	6 min
Solid-state disk I/O (flash memory)	50–150 µs	2–6 days
Rotational disk I/O	1–10 ms	1–12 months
Internet: San Francisco to New York	40 ms	4 years
Internet: San Francisco to United Kingdom	81 ms	8 years
Internet: San Francisco to Australia	183 ms	19 years
TCP packet retransmit	1–3 s	105–317 years
OS virtualization system reboot	4 s	423 years
SCSI command time-out	30 s	3 millennia
Hardware (HW) virtualization system reboot	40 s	4 millennia
Physical system reboot	5 m	32 millennia



[Brendan Gregg, 2013](#)

Introduction à Spark

Disponibilité

- Capacité d'un système à rester opérationnel et accessible aux utilisateurs (même en cas de défaillance d'une partie du système, lié à la tolérance aux pannes, c.f. plus tard).
- La disponibilité mesure la proportion de temps pendant lequel un système est en état de fonctionnement et prêt à répondre aux demandes des utilisateurs.

	Par jour	Par mois	Par an
99 %	14,5 min	7,5 h	3,6 j
99,9 %	1,5 min	43,9 min	9 h
99,99 %	8 s	4,5 min	53 min
99,999 %	0,4 s	26 s	5 min

Introduction à Spark

Tolérance aux pannes

« La plupart des systèmes tombent un jour en panne »

- Capacité d'un système à maintenir un comportement défini malgré la survenue d'une panne.
- Un système fiable demeure opérationnel même en cas de dysfonctionnement de certains de ses composants.



Introduction à Spark

Paradigmes d'architecture dans les systèmes distribués

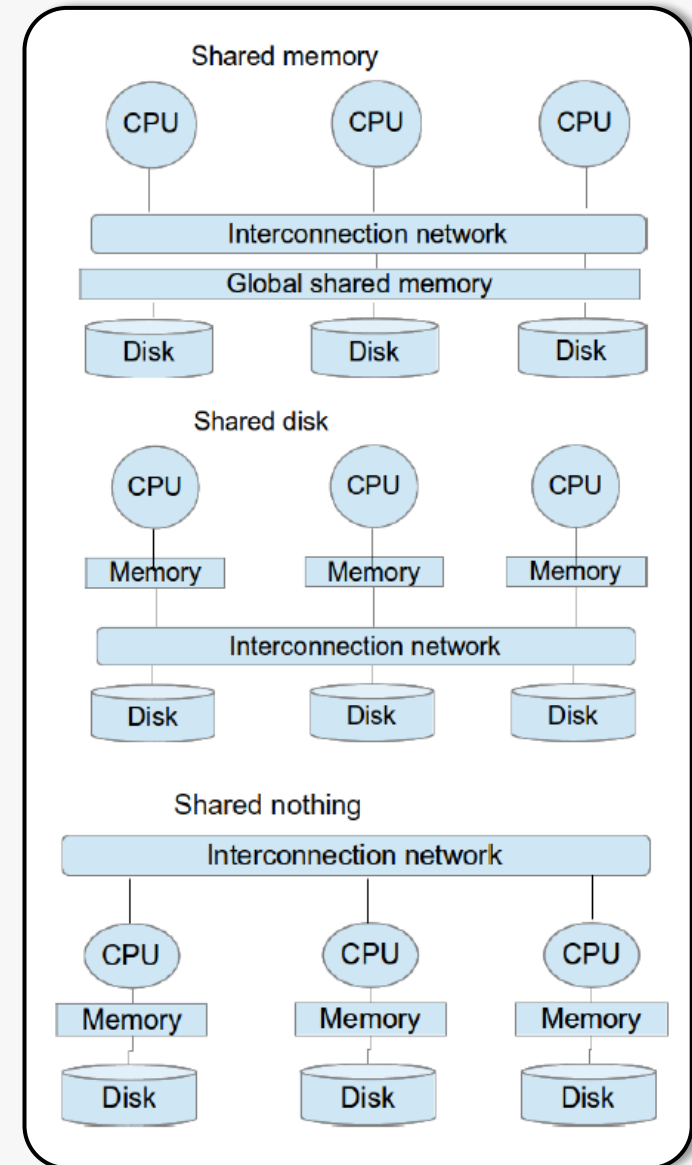
- **Shared-memory** Plusieurs nœuds ou processus partagent l'accès à un pool commun de mémoire.

Cela permet une communication facile et un partage de données entre les processus.
Cependant, cela peut également entraîner des problèmes tels que la cohérence des données et des limitations d'évolutivité.
- **Shared-disk** Plusieurs nœuds ont accès à un système de stockage partagé (disque) où les données sont stockées. Cela permet à plusieurs nœuds de lire et d'écrire sur le même magasin de données simultanément.

Couramment utilisé dans les systèmes de fichiers et les systèmes de bases de données distribués.

Offre une haute disponibilité et des capacités de partage de données, mais peut souffrir de goulets d'étranglement de performance dus à la contention pour les ressources disque.

Coût intéressant pour un cluster de petite à moyenne taille (comprenant généralement jusqu'à une dizaine de machines).



Introduction à Spark

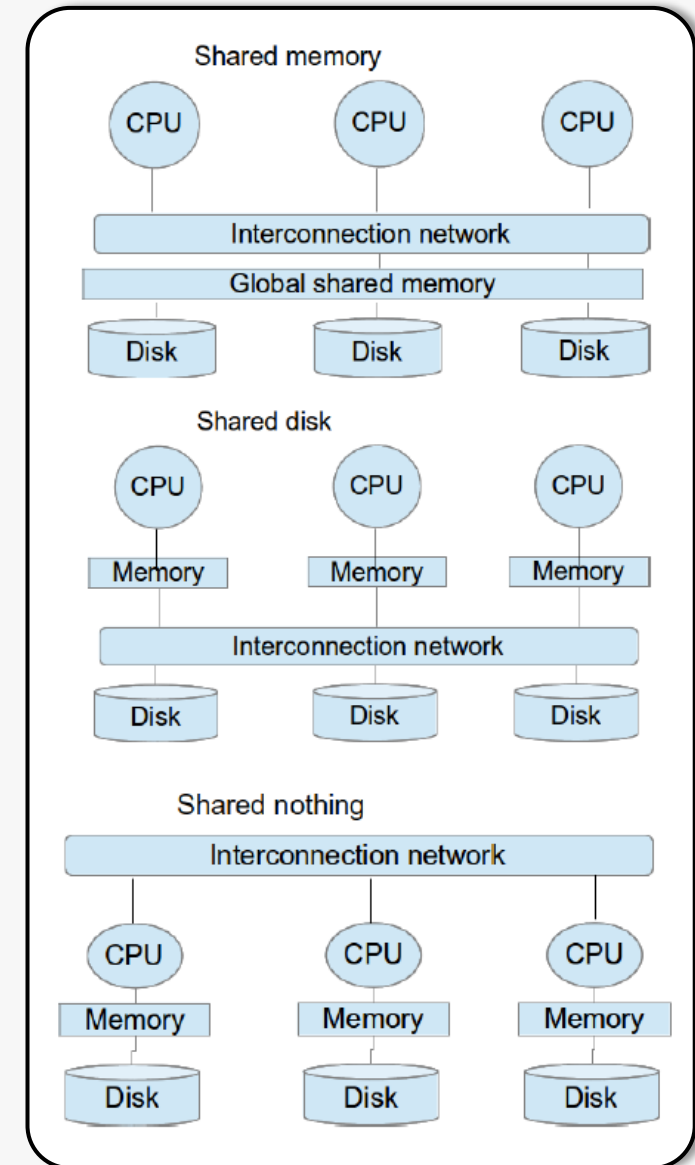
Paradigmes d'architecture dans les systèmes distribués

- **Shared-nothing** Chaque nœud du système fonctionne de manière indépendante et ne partage pas de mémoire ou de stockage disque avec les autres nœuds.

Les nœuds communiquent en se passant des messages via un réseau.

Cette architecture offre une évolutivité et une tolérance aux pannes car la défaillance d'un nœud n'affecte pas les autres.

MapReduce et Hadoop sont des exemples de systèmes shared-nothing.



Introduction à Spark

Eclosion

- L'idée centrale derrière MapReduce était le traitement parallèle de données sur un grand nombre de machines tout en abstrayant la complexité associée à la gestion de ces traitements distribués
- MapReduce a inspiré (entre autres) la création des moteurs Apache Hadoop, et par la suite, Apache Spark.
- Il peut être intéressant de comprendre l'évolution de l'écosystème de traitement des mégadonnées de Google, de l'innovation interne de Google à l'émergence d'un écosystème global de traitement des données open source dynamique et en évolution constante.



Introduction à Spark

GFS

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung
Google*

ABSTRACT

We have designed and implemented the Google File System, a scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients.

While sharing many of the same goals as previous distributed file systems, our design has been driven by observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system assumptions. This has led us to reexamine traditional choices and explore radically different design points.

The file system has successfully met our storage needs. It is widely deployed within Google as the storage platform for the generation and processing of data used by our service as well as research and development efforts that require large data sets. The largest cluster to date provides hundreds of terabytes of storage across thousands of disks on over a thousand machines, and it is concurrently accessed by hundreds of clients.

In this paper, we present file system interface extensions

1. INTRODUCTION

We have designed and implemented the Google File System (GFS) to meet the rapidly growing demands of Google's data processing needs. GFS shares many of the same goals as previous distributed file systems such as performance, scalability, reliability, and availability. However, its design has been driven by key observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system design assumptions. We have reexamined traditional choices and explored radically different points in the design space.

First, component failures are the norm rather than the exception. The file system consists of hundreds or even thousands of storage machines built from inexpensive commodity parts and is accessed by a comparable number of client machines. The quantity and quality of the components virtually guarantee that some are not functional at any given time and some will not recover from their current failures. We have seen problems caused by application bugs, operating system bugs, human errors, and the failures of disks, memory, connectors, networking, and power sup-

<https://static.googleusercontent.com/media/research.google.com/fr//archive/gfs-sosp2003.pdf>

Introduction à Spark

MapReduce

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the program's execution across a set of machines, handling machine failures, and managing the required inter-machine communication. This allows programmers without any experience with parallel and distributed systems to easily utilize the resources of a large distributed system.

given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is inspired by the *map* and *reduce* primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a *map* operation to each logical "record" in our input in order to compute a set of intermediate key/value pairs, and then

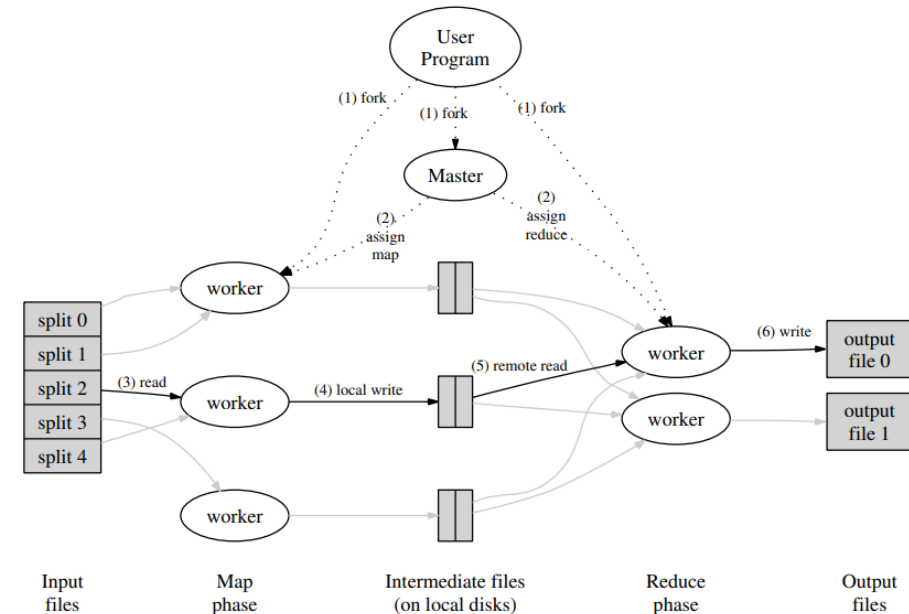


Figure 1: Execution overview

<https://static.googleusercontent.com/media/research.google.com/fr//archive/mapreduce-osdi04.pdf>

Introduction à Spark

Sawzall

Interpreting the Data: Parallel Analysis with Sawzall

Rob Pike, Sean Dorward, Robert Griesemer, Sean Quinlan
Google, Inc.

Abstract

Very large data sets often have a flat but regular structure and span multiple disks and machines. Examples include telephone call records, network logs, and web document repositories. These large data sets are not amenable to study using traditional database techniques, if only because they can be too large to fit in a single relational database. On the other hand, many of the analyses done on them can be expressed using simple, easily distributed computations: filtering, aggregation, extraction of statistics, and so on.

We present a system for automating such analyses. A filtering phase, in which a query is expressed using a new procedural programming language, emits data to an aggregation phase. Both phases are distributed over hundreds or even thousands of computers. The results are then collated and saved to a file. The design—including the separation into two phases, the form of the programming language, and the properties of the aggregators—exploits the parallelism inherent in having data and computation distributed across many machines.

1 Introduction

Many data sets are too large, too dynamic, or just too unwieldy to be housed productively in a relational database. One common scenario is a set of many plain files—sometimes amounting to

<https://static.googleusercontent.com/media/research.google.com/fr//archive/sawzall-sciprog.pdf>

Introduction à Spark

Chubby

The Chubby lock service for loosely-coupled distributed systems

Mike Burrows, *Google Inc.*

Abstract

We describe our experiences with the Chubby lock service, which is intended to provide coarse-grained locking as well as reliable (though low-volume) storage for a loosely-coupled distributed system. Chubby provides an interface much like a distributed file system with advisory locks, but the design emphasis is on availability and reliability, as opposed to high performance. Many instances of the service have been used for over a year, with several of them each handling a few tens of thousands of clients concurrently. The paper describes the initial design and expected use, compares it with actual use, and explains how the design had to be modified to accommodate the differences.

1 Introduction

This paper describes a *lock service* called Chubby. It is intended for use within a loosely-coupled distributed sys-

example, the Google File System [7] uses a Chubby lock to appoint a GFS master server, and Bigtable [3] uses Chubby in several ways: to elect a master, to allow the master to discover the servers it controls, and to permit clients to find the master. In addition, both GFS and Bigtable use Chubby as a well-known and available location to store a small amount of meta-data; in effect they use Chubby as the root of their distributed data structures. Some services use locks to partition work (at a coarse grain) between several servers.

Before Chubby was deployed, most distributed systems at Google used *ad hoc* methods for primary election (when work could be duplicated without harm), or required operator intervention (when correctness was essential). In the former case, Chubby allowed a small saving in computing effort. In the latter case, it achieved a significant improvement in availability in systems that no longer required human intervention on failure.

Readers familiar with distributed computing will recognize the election of a primary among peers as an in-

<https://static.googleusercontent.com/media/research.google.com/fr//archive/chubby-osdi06.pdf>

Introduction à Spark

BigTable

Bigtable: A Distributed Storage System for Structured Data

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach
Mike Burrows, Tushar Chandra, Andrew Fikes, Robert E. Gruber

{fay,jeff,sanjay,wilsonh,kerr,m3b,tushar,fikes,gruber}@google.com

Google, Inc.

Abstract

Bigtable is a distributed storage system for managing structured data that is designed to scale to a very large size: petabytes of data across thousands of commodity servers. Many projects at Google store data in Bigtable, including web indexing, Google Earth, and Google Finance. These applications place very different demands on Bigtable, both in terms of data size (from URLs to web pages to satellite imagery) and latency requirements (from backend bulk processing to real-time data serving). Despite these varied demands, Bigtable has successfully provided a flexible, high-performance solution for all of these Google products. In this paper we describe the simple data model provided by Bigtable, which gives clients dynamic control over data layout and format, and we describe the design and implementation of Bigtable.

1 Introduction

Over the last two and a half years we have designed.

achieved scalability and high performance, but Bigtable provides a different interface than such systems. Bigtable does not support a full relational data model; instead, it provides clients with a simple data model that supports dynamic control over data layout and format, and allows clients to reason about the locality properties of the data represented in the underlying storage. Data is indexed using row and column names that can be arbitrary strings. Bigtable also treats data as uninterpreted strings, although clients often serialize various forms of structured and semi-structured data into these strings. Clients can control the locality of their data through careful choices in their schemas. Finally, Bigtable schema parameters let clients dynamically control whether to serve data out of memory or from disk.

Section 2 describes the data model in more detail, and Section 3 provides an overview of the client API. Section 4 briefly describes the underlying Google infrastructure on which Bigtable depends. Section 5 describes the fundamentals of the Bigtable implementation, and Sec-

<https://static.googleusercontent.com/media/research.google.com/fr//archive/bigtable-osdi06.pdf>

Introduction à Spark

Hadoop

- Désormais nous avons à disposition les principes du modèle de programmation MapReduce.
- Pour passer à l'échelle les traitements sur de gros volumes de données. il faut pour cela qu'il soit associé à une infrastructure logicielle dédiée qui permette d'exécuter le schéma MapReduce de manière massivement distribuée sur un cluster de machines tout en prenant à sa charge les enjeux du calcul distribué :
 - ➡ l'optimisation des transferts disques et réseau en limitant les déplacements de données (data locality),
 - ➡ la scalabilité pour permettre d'adapter la puissance au besoin (scalability),
 - ➡ la tolérance aux pannes (embracing failure).

Introduction à Spark

Hadoop

- En 2002, Doug Cutting et Mike Cafarella, deux ingénieurs, décident de s'attaquer au passage à l'échelle de Lucene, le moteur de recherche open source. L'objectif était de le rendre capable d'indexer et de rechercher dans des collections de la taille du Web. C'est le projet Nutch.
- Pour cela, ils s'inspirent de deux articles de recherche publiés par les Google Labs que nous avons vus : GFS et MapReduce.
- L'architecture de Nutch, qui repose donc sur un système de fichiers distribué et sur MapReduce, est relativement générique et donnera lieu au projet Hadoop, initié en 2006.
- Il rejoint la fondation Apache en 2008.
- La version stable actuelle est la version 2.7.



Introduction à Spark

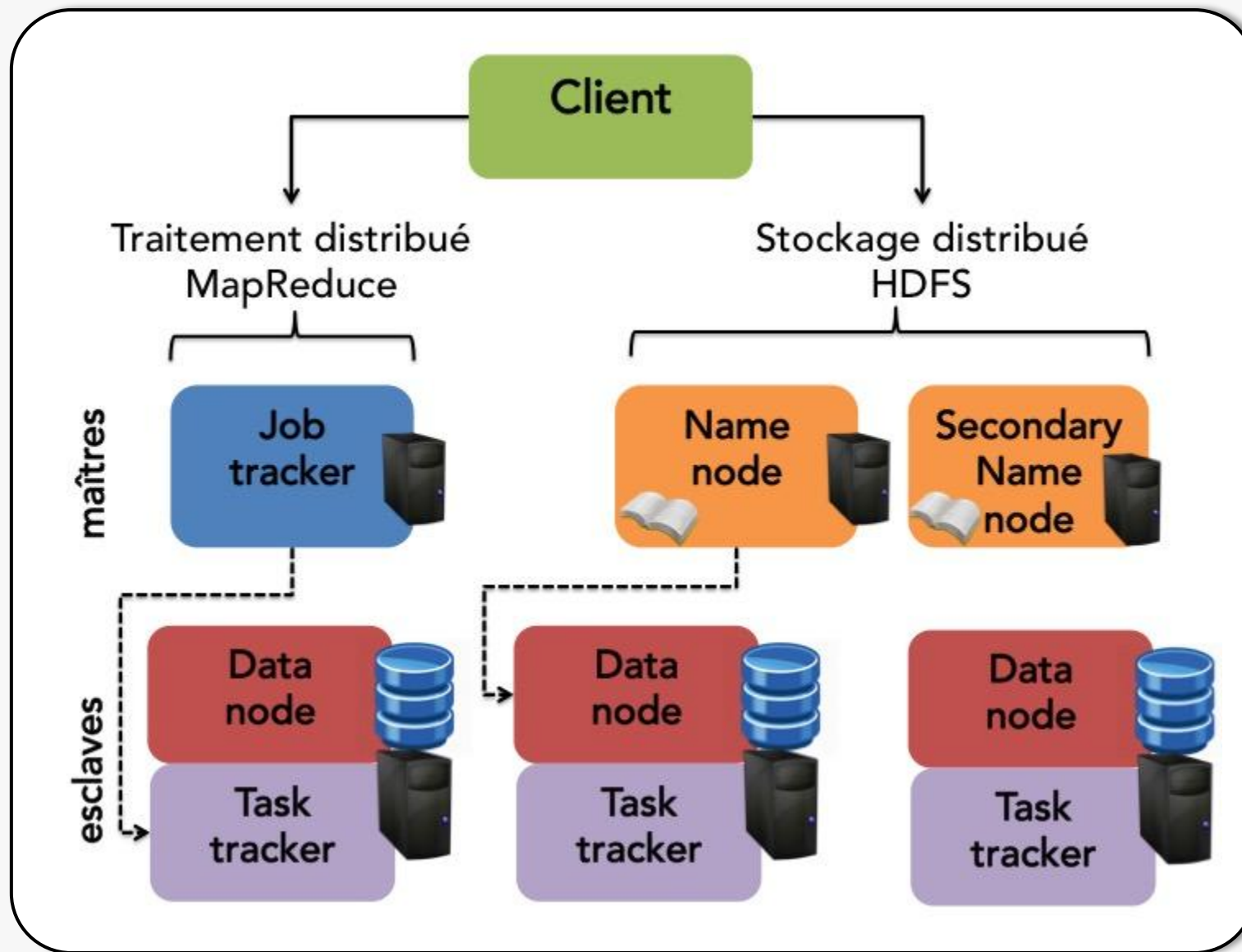
Hadoop

Le socle technique d'Hadoop est composé :

- De toute l'architecture support nécessaire pour l'orchestration de MapReduce, c'est-à-dire :
 - ➡ l'ordonnancement des traitements,
 - ➡ la localisation des fichiers,
 - ➡ la distribution de l'exécution.
- D'un système de fichiers **HDFS** qui est :
 - ➡ Distribué : les données sont réparties sur les machines du cluster.
 - ➡ Répliqué : en cas de panne, aucune donnée n'est perdue.
 - ➡ Optimisé pour la colocalisation des données et des traitements.

Introduction à Spark

Hadoop



Introduction à Spark

HDFS

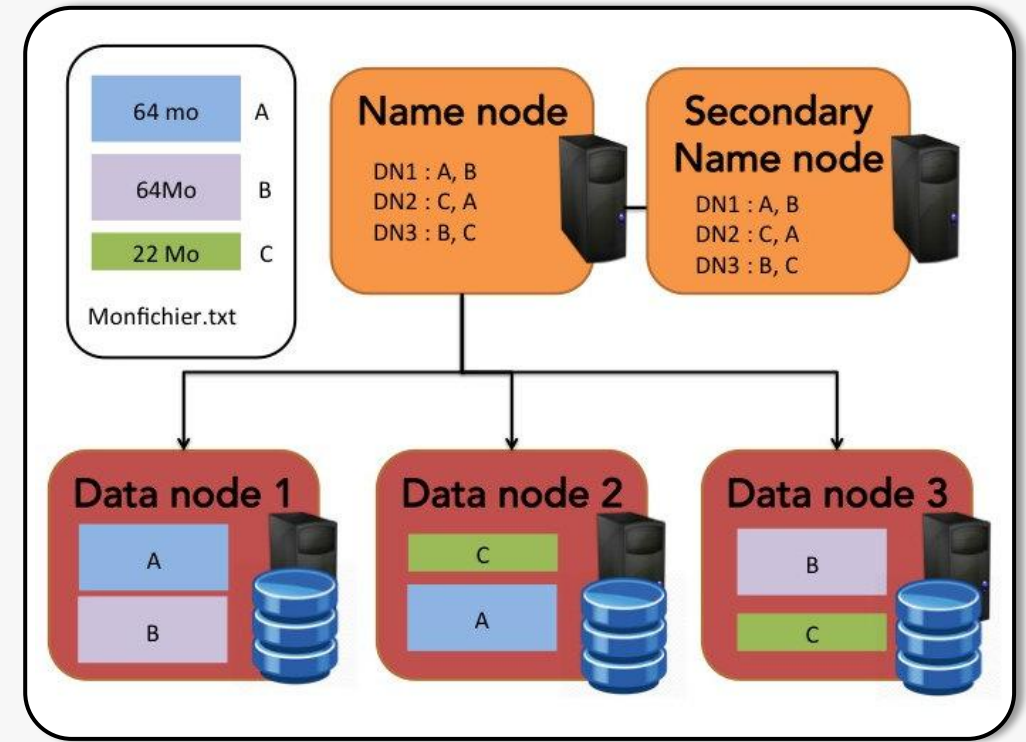
- HDFS (*Hadoop Distributed File System*) : système de fichiers distribué et couche native de stockage et d'accès à des données d'Hadoop.
- Il a été conçu pour stocker des fichiers de très grande taille et, comme son nom l'indique, dans un cadre distribué.
 - ➡ les fichiers sont physiquement **découpés en blocs d'octets de grande taille** (par défaut 64 Mo) pour optimiser les temps de transfert et d'accès ;
 - ➡ ces blocs sont ensuite **répartis** sur plusieurs machines, permettant ainsi de traiter un même fichier en parallèle. Cela permet aussi de ne pas être limité par la capacité de stockage d'une seule machine pour au contraire tirer parti de tout l'espace disponible du cluster de machines ;
 - ➡ pour garantir une tolérance aux pannes, les blocs de chaque fichier sont **repliqués**, de manière intelligente, sur plusieurs machines.

Introduction à Spark

HDFS

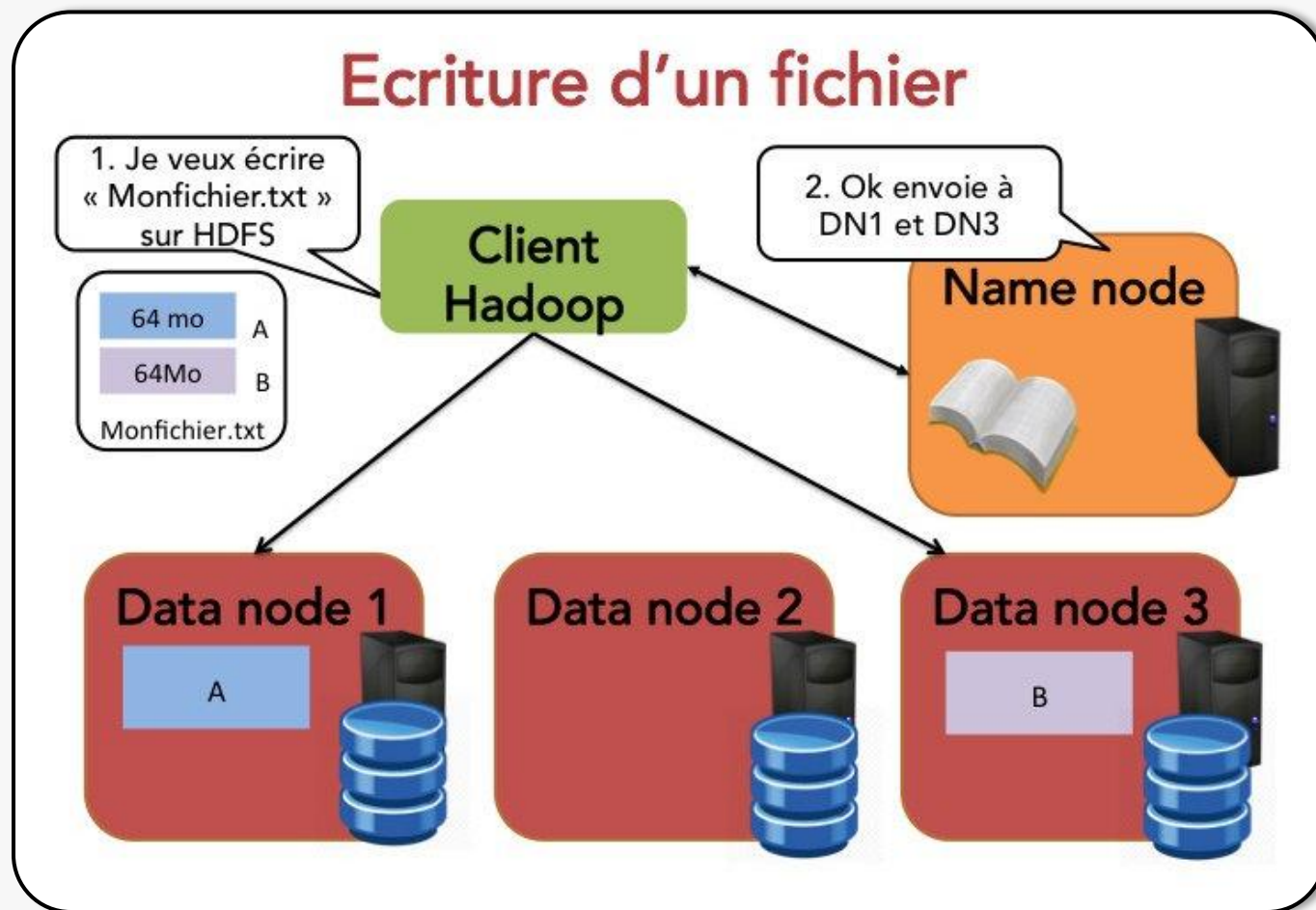
Dans Hadoop, l'architecture de stockage est une architecture maître-esclave.

- Le nœud maître appelé name node contient et stocke tous les noms et blocs des fichiers ainsi que leur localisation dans le cluster. On peut donc le voir comme un gros annuaire.
- Une autre machine, appelée secondary name node sert de name node de secours en cas de défaillance du nœud maître et il a donc pour rôle de faire des sauvegardes régulières de l'annuaire.
- Les autres nœuds, les esclaves, sont les nœuds de stockage en tant que tels. Ce sont les data nodes qui ont pour rôle la gestion des opérations de stockage locales (création, suppression et réplication de blocs) sur instruction du name node.



Introduction à Spark

HDFS

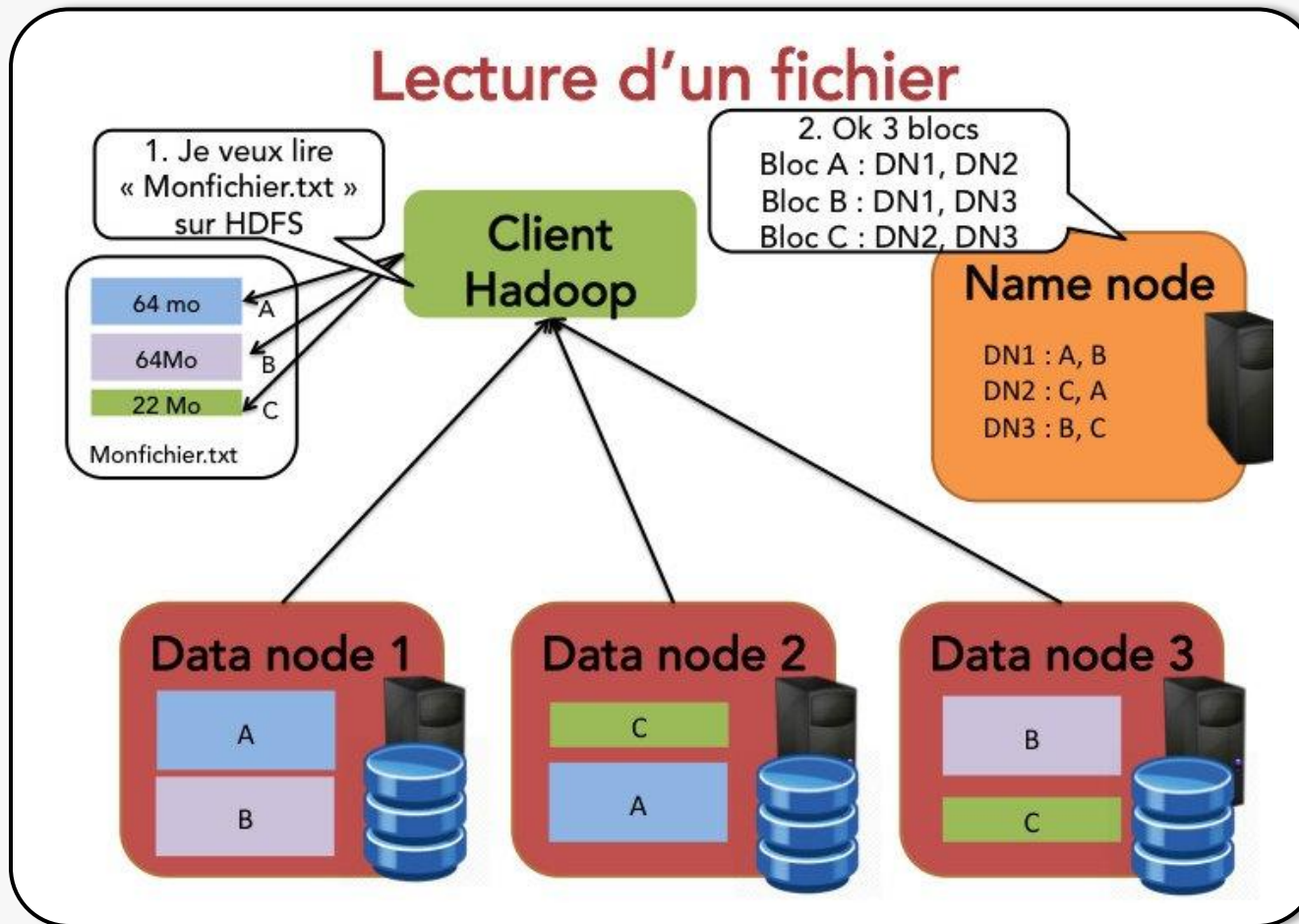


Si on souhaite écrire un fichier dans HDFS, on utilise un client Hadoop :

1. Le client indique au name node qu'il souhaite écrire un bloc.
2. Le name node indique le data node à contacter.
3. Le client envoie le bloc au data node.
4. Les data nodes répliquent les blocs entre eux.
5. Le cycle se répète pour le bloc suivant.

Introduction à Spark

HDFS



Si on souhaite lire un fichier dans HDFS :

1. Le client indique au name node qu'il souhaite lire un fichier.
2. Le name node indique sa taille ainsi que les différents data nodes contenant les blocs.
3. Le client récupère chacun des blocs sur l'un des data nodes.
4. Si le data node est indisponible, le client en contacte un autre.

Introduction à Spark

Hadoop MapReduce

- ➡ Comment ordonnancer les traitements ?
- ➡ Comment distribuer l'exécution sur les différents nœuds du cluster ?
- ➡ Comment connaître l'emplacement des fichiers à traiter ?

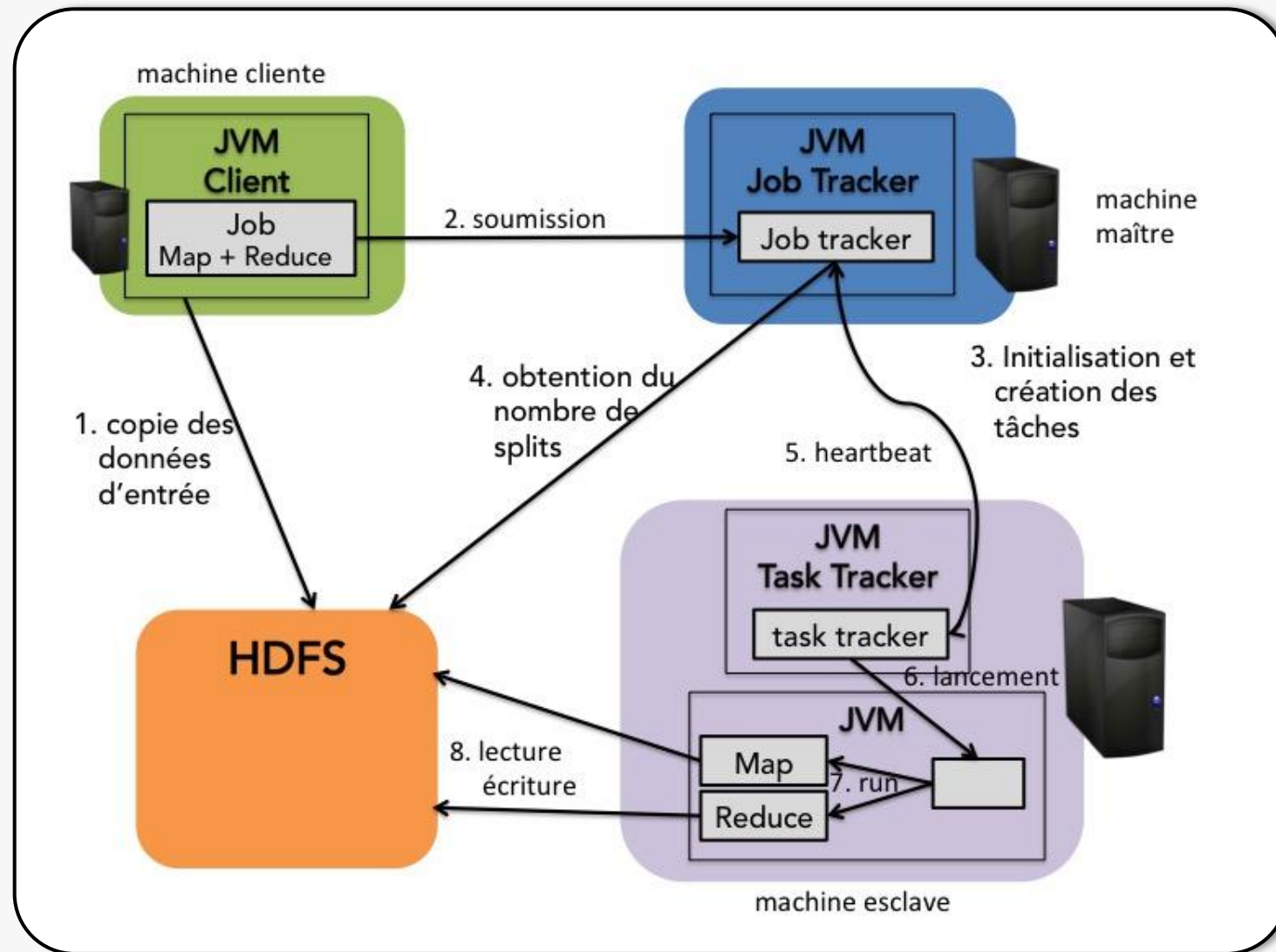
Hadoop peut s'occuper de tout cela, à nouveau avec une architecture de type **maître-esclave** :

Le **job tracker** est un processus maître qui va se charger de l'ordonnancement des traitements et de la gestion de l'ensemble des ressources du système. Il reçoit (du client) la ou les tâches MapReduce à exécuter (un .jar Java) ainsi que les données d'entrée et le répertoire où stocker les données de sorties. Il est pour cela en communication avec le **name node** d'HDFS. Le job tracker est en charge de planifier l'exécution des tâches et de les distribuer sur des **task trackers**. Comme il sait où sont situés les blocs de données, il peut optimiser la colocalisation traitements/données.

Un **task tracker** est une unité de calcul du cluster. Il assure, en lançant une nouvelle machine virtuelle java (JVM), l'exécution et le suivi des tâches MAP ou REDUCE s'exécutant sur son nœud et qu'il reçoit du **job tracker**. Il dispose d'un nombre limité de slots d'exécution et donc un nombre limité de tâches MAP, REDUCE ou SHUFFLE pouvant s'exécuter simultanément sur le nœud. Il est aussi en communication constante avec le **job tracker** pour l'informer de l'état d'avancement des tâches (*heartbeat call*). En cas de défaillance, le **job tracker**, informé ou sans nouvelle du task tracker, doit pouvoir ordonner la réexécution de la tâche.

Introduction à Spark

Hadoop MapReduce



Introduction à Spark

Hadoop 2 et Yarn

Nous avons vu que l'algorithme MapReduce permet d'implémenter de nombreux types de traitement en vue de leur parallélisation.

➡ Tous les problèmes rentrent-ils dans le moule MapReduce ?

Introduction à Spark

Hadoop 2 et Yarn

Nous avons vu que l'algorithme MapReduce permet d'implémenter de nombreux types de traitement en vue de leur parallélisation.

➡ Tous les problèmes rentrent-ils dans le moule MapReduce ?

Non pas forcément et très souvent, si c'est le cas, cela demande beaucoup d'efforts de transformer un algorithme en MapReduce.

Introduction à Spark

Hadoop 2 et Yarn

Nous avons vu que l'algorithme MapReduce permet d'implémenter de nombreux types de traitement en vue de leur parallélisation.

➡ Tous les problèmes rentrent-ils dans le moule MapReduce ?

Non pas forcément et très souvent, si c'est le cas, cela demande beaucoup d'efforts de transformer un algorithme en MapReduce.

Pour traiter des problèmes complexes, les deux étapes MAP et REDUCE ne suffisent pas, il est très souvent nécessaire d'enchaîner des séquences de MapReduce ce qui est très coûteux car cela nécessite de démarrer un job MapReduce à chaque fois.

Introduction à Spark

Hadoop 2 et Yarn

Nous avons vu que l'algorithme MapReduce permet d'implémenter de nombreux types de traitement en vue de leur parallélisation.

➡ Tous les problèmes rentrent-ils dans le moule MapReduce ?

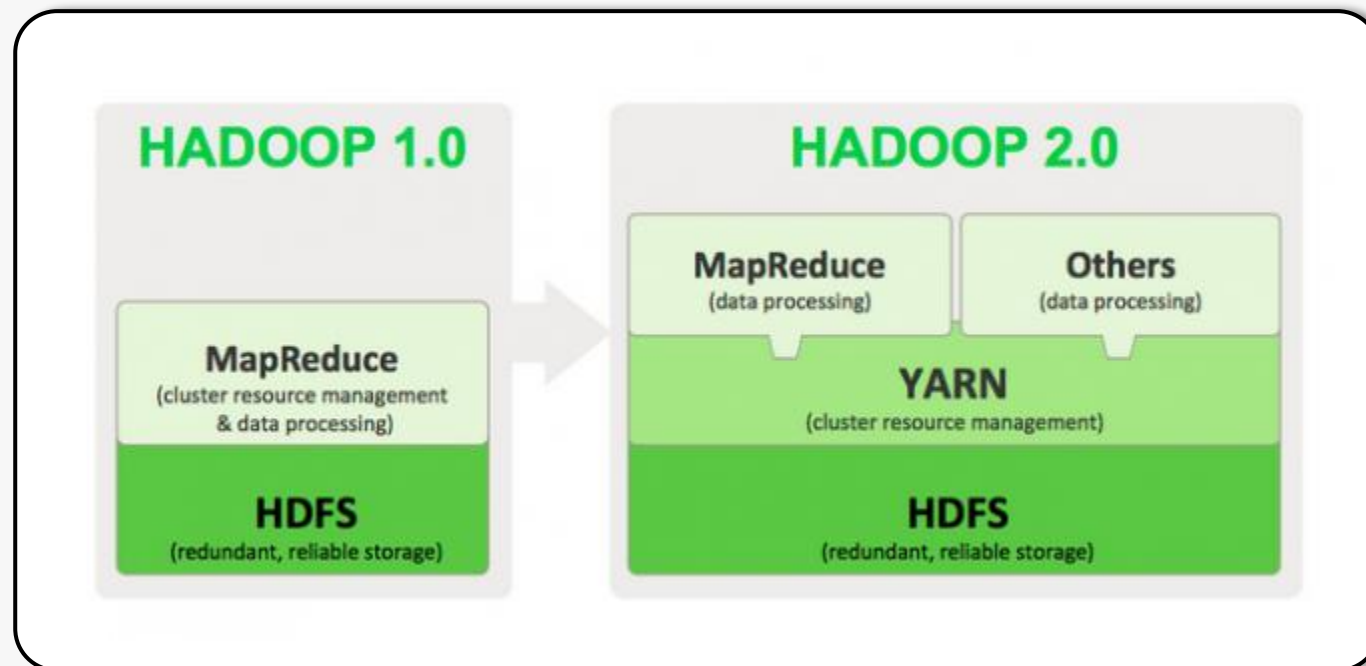
Non pas forcément et très souvent, si c'est le cas, cela demande beaucoup d'efforts de transformer un algorithme en MapReduce.

Pour traiter des problèmes complexes, les deux étapes MAP et REDUCE ne suffisent pas, il est très souvent nécessaire d'enchaîner des séquences de MapReduce ce qui est très coûteux car cela nécessite de démarrer un job MapReduce à chaque fois.

Le job tracker a une double responsabilité : gérer les ressources du cluster et ordonnancer les jobs. C'est un point critique en cas de défaillance.

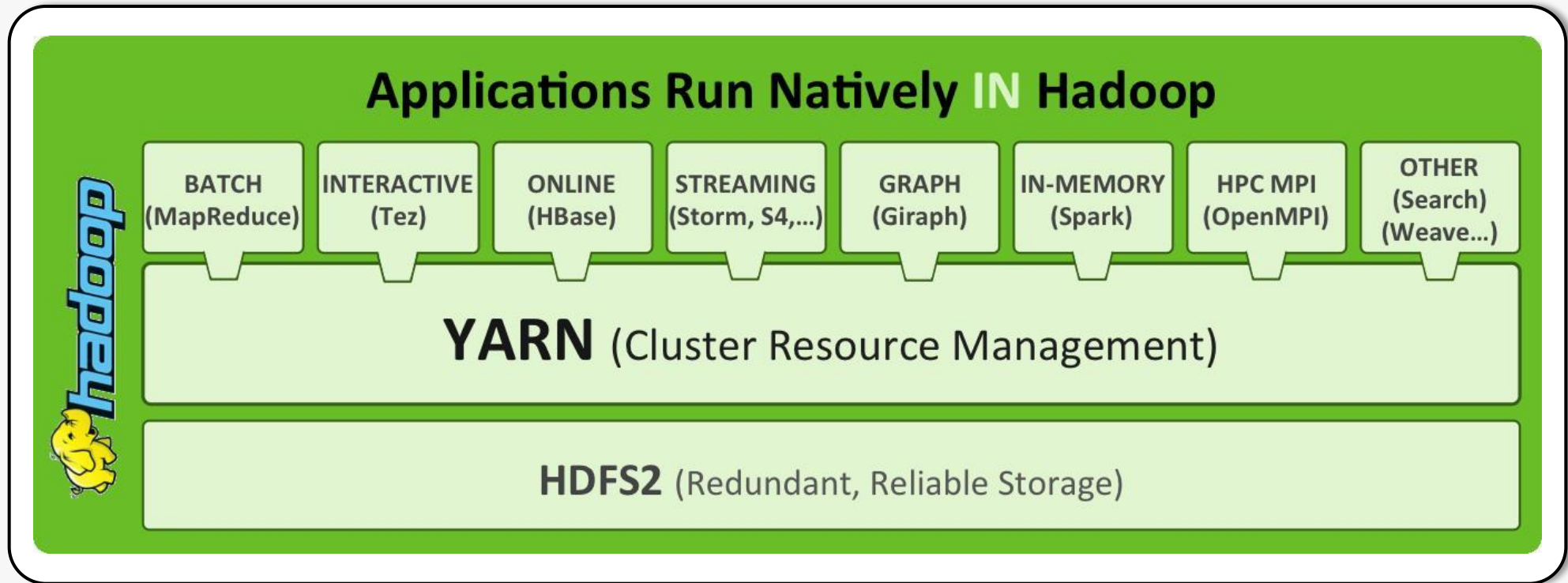
Introduction à Spark

Hadoop 2 et Yarn



Introduction à Spark

Hadoop 2 et Yarn



Introduction à Spark

Spark, le retour

Spark :

- Spark est un framework de traitement de données en mémoire conçu pour le traitement rapide et efficace de gros volumes de données.
- Contrairement à MapReduce, Spark conserve les données en mémoire chaque fois que cela est possible, ce qui réduit considérablement les temps d'accès aux données.
- Spark prend en charge un large éventail de charges de travail, y compris le traitement de flux de données, le traitement de graphiques, l'apprentissage automatique et le traitement interactif.
- Il fournit des APIs faciles à utiliser pour le développement d'applications de traitement de données en Python, Scala, Java et R.



Question 1

Volumes de données

La BNF (Bibliothèque Nationale de France) contient (à la louche) 30 millions de livres. En admettant que, en moyenne, chaque livre contienne 300 pages et que chaque page contienne 2000 caractères, combien de données non compressées cela représente-t-il ?

- ☐ Moins de 1 Go
- ☐ Entre 1 Go et 1 To
- ☐ Entre 1 To et 1 Po
- ☐ Entre 1 Po et 1 Eo

Question 2

Passage à l'échelle

Quand on ajoute des machines à un cluster pour augmenter sa capacité, il s'agit d'un passage à l'échelle :

- ☐ Horizontal
- ☐ Transversal
- ☐ Vertical

Question 3

Hadoop

Hadoop est :

- ☐ un framework théorique de calcul particulièrement adapté aux calculs distribués
- ☐ un écosystème open source destiné au stockage et au traitement distribués de données massives,

Introduction à Spark

Définition des composants

Driver

Processus principal de l'application Spark. Il contient la fonction **main** du programme et est responsable de la création du contexte Spark, de la création des RDDs, de la planification des opérations, et de la coordination des différentes étapes de l'exécution.

Executor

Processus qui s'exécute sur les nœuds du cluster et est responsable de l'exécution réelle des tâches (tasks) de l'application Spark. Chaque application Spark dispose de ses propres executors qui sont lancés sur les nœuds du cluster par le driver.

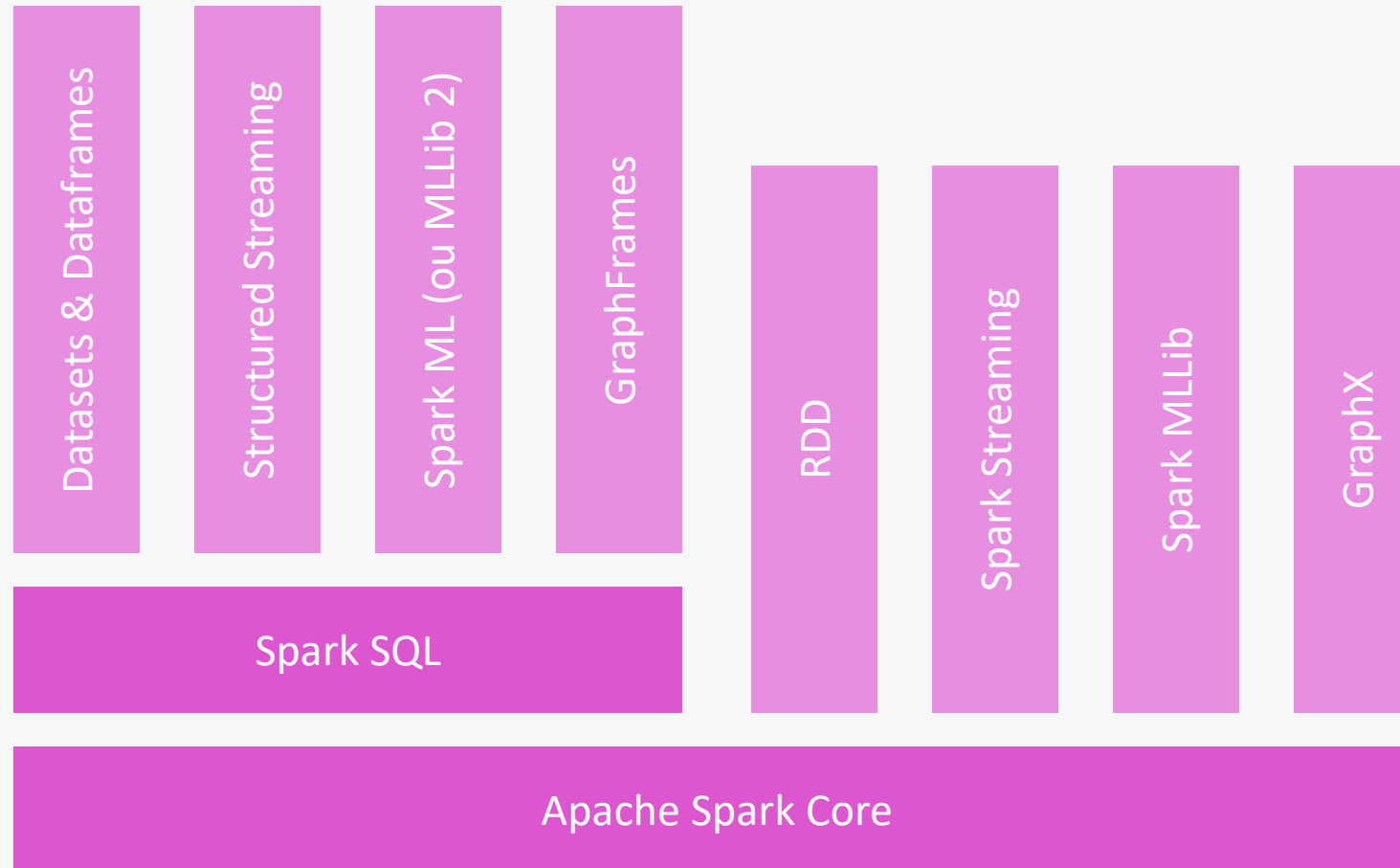
Introduction à Spark

Définition des composants

- Job** Unité logique de travail que le driver envoie aux executors pour être exécutée. Généralement créé lorsqu'une action est appelée sur un RDD, ce qui déclenche une séquence d'opérations (transformations) sur les données, qui sont ensuite regroupées en plusieurs stages et exécutées par les executors.
- Stage** Unité d'opérations qui peut être exécutée en parallèle sur les données. Les stages sont divisés en fonction des opérations de transformation et des partitions des données. Un job peut contenir plusieurs stages, et chaque stage peut contenir plusieurs tasks qui sont exécutées en parallèle sur les Executors.
- Task** La plus petite unité d'exécution dans Spark. Chaque task est responsable du traitement d'une partition de données spécifique. Les tasks sont exécutées sur les executors et effectuent les calculs nécessaires pour transformer ou traiter les données selon les opérations spécifiées par l'application Spark.

Introduction à Spark

Les principaux composants



Introduction à Spark

Setup

En pratique, quel langage de programmation faut-il installer ?



Inclus dans la distribution Spark → pas besoin d'installer Scala séparément pour utiliser spark-shell.



Toujours nécessaire (même en cas d'utilisation de PySpark, car Spark tourne sur la JVM).



Nécessaire pour, et uniquement uniquement pour, utiliser PySpark ou exécuter des jobs Python.

Introduction à Spark

Setup

Il existe différentes façons d'installer Spark (voire même de manipuler Spark sans l'installer).

➡ Installer Spark "à la main"

1. Télécharger l'archive officielle depuis spark.apache.org (<https://archive.apache.org/dist/spark/>).
2. Installer Java.
3. Configurer éventuellement pour accéder à HDFS ou Hive.
4. Décompresser Spark et définir les variables d'environnement (SPARK_HOME, PATH).

Avantages :

- Contrôle total sur la version de Spark et la configuration.
- Possibilité d'utiliser Spark en mode local ou cluster (standalone ou YARN).

Introduction à Spark

Setup

Il existe différentes façons d'installer Spark (voire même de manipuler Spark sans l'installer).

➡ Environnements cloud en ligne

- Utiliser des services managés comme :
 - Databricks (notebooks collaboratifs, Spark optimisé) -> il s'agit d'une méthode possible pour notre formation.
 - Google Dataproc (Spark sur GCP).
 - Amazon EMR (Spark sur AWS).
 - Azure Synapse / HDInsight (Spark sur Azure).

Avantages :

- Pas d'installation locale nécessaire.
- Gestion automatique du cluster et scalabilité selon les besoins.
- Intégration facile avec le stockage cloud (S3, GCS, ADLS).

Introduction à Spark

Setup : avec un environnement en ligne Databricks

1. Se rendre sur le site <https://community.cloud.databricks.com/>
2. Suivre les étapes pour créer un compte gratuit ou se connecter à un compte existant.
3. Il est alors possible de téléverser des données via + New > Add or upload data.
4. Il est possible de créer un nouveau notebook via + New > Notebook puis d'en importer un via le Menu File > Import.

Introduction à Spark

Setup

Il existe différentes façons d'installer Spark (voire même de manipuler Spark sans l'installer).

➡ Utiliser Spark via Docker

- Docker Hub propose des images officielles ou communautaires de Spark.
- Commande typique : `docker run -it --rm -p 8888:8888 bitnami/spark:latest`
- Permet de lancer Spark avec toutes les dépendances déjà configurées.

Avantages :

- Contrôle total sur la version de Spark et la configuration.
- Possibilité d'utiliser Spark en mode local ou cluster (standalone ou YARN).

Introduction à Spark

Setup : avec Docker

Il s'agit d'une méthode possible pour cette formation.

1. Récupérer les fichiers du dépôt Github de la formation.
2. Lancer un terminal.
3. Se déplacer dans le dossier contenant le fichier `docker-compose.yaml`
4. Lancer la commande `docker compose up`

Introduction à Spark

Setup

Dans tous les cas, deux formats sont généralement pris en charge :



Scripts Python (.py), pour un usage classique ou en production



Notebooks Jupyter (.ipynb), pour l'expérimentation interactive et la visualisation

Introduction à Spark

Soumettre des jobs à Spark

➡ spark-submit

```
spark-submit --master <master-url> --deploy-mode <deploy-mode> --executor-memory <memory> your_script.py
```

➡ Spark Shell

Lancer le shell interactif pour interagir avec Spark.
Cela est utile pour l'exploration de données et le prototypage rapide.

```
spark-shell
```

➡ Notebooks

Introduction à Spark

Soumettre des jobs à Spark (Python)

➡ **spark-submit**

```
spark-submit --master <master-url> --deploy-mode <deploy-mode> --executor-memory <memory> your_script.py
```

Python

➡ **PySpark Shell** Lancer le shell interactif PySpark pour interagir avec Spark en utilisant Python. Cela est utile pour l'exploration de données et le prototypage rapide.

```
pyspark
```

➡ **Notebooks Jupyter** Installer le package pyspark et configurer un kernel Jupyter pour PySpark.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Nom de votre choix").getOrCreate()
# Votre code Spark à suivre
```

Introduction à Spark

Etude de code



MapReduce sans Spark

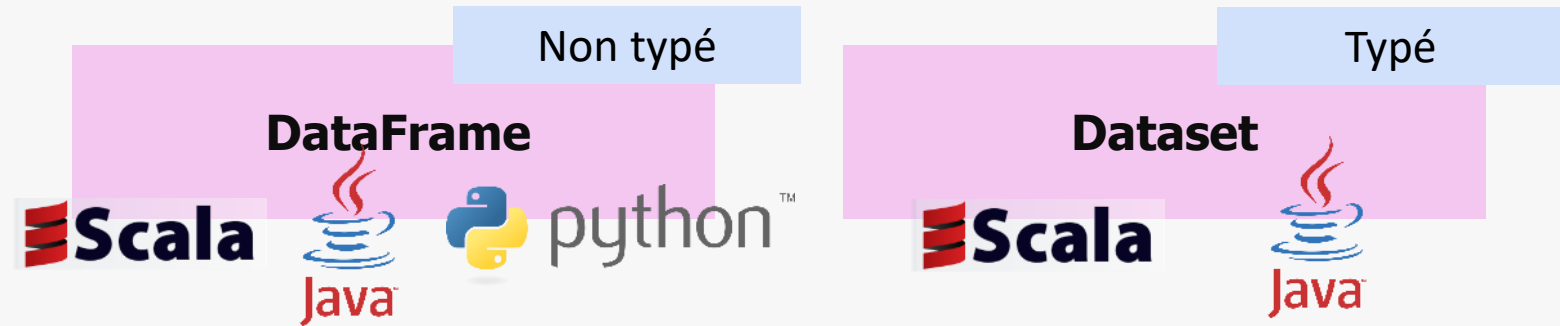
02

Fonctionnalités essentielles de Spark

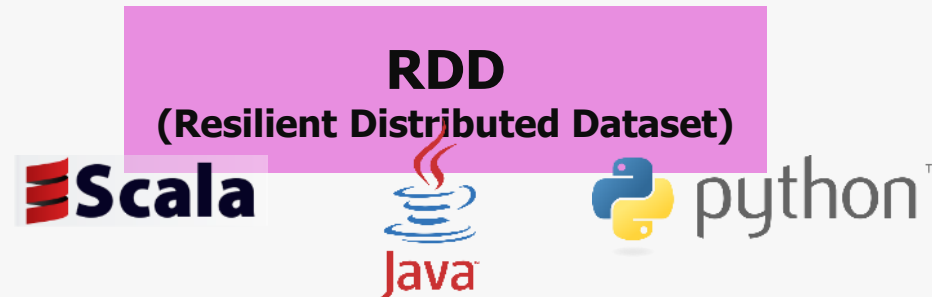
Fonctionnalités essentielles de Spark

Abstractions de Spark

Haut niveau



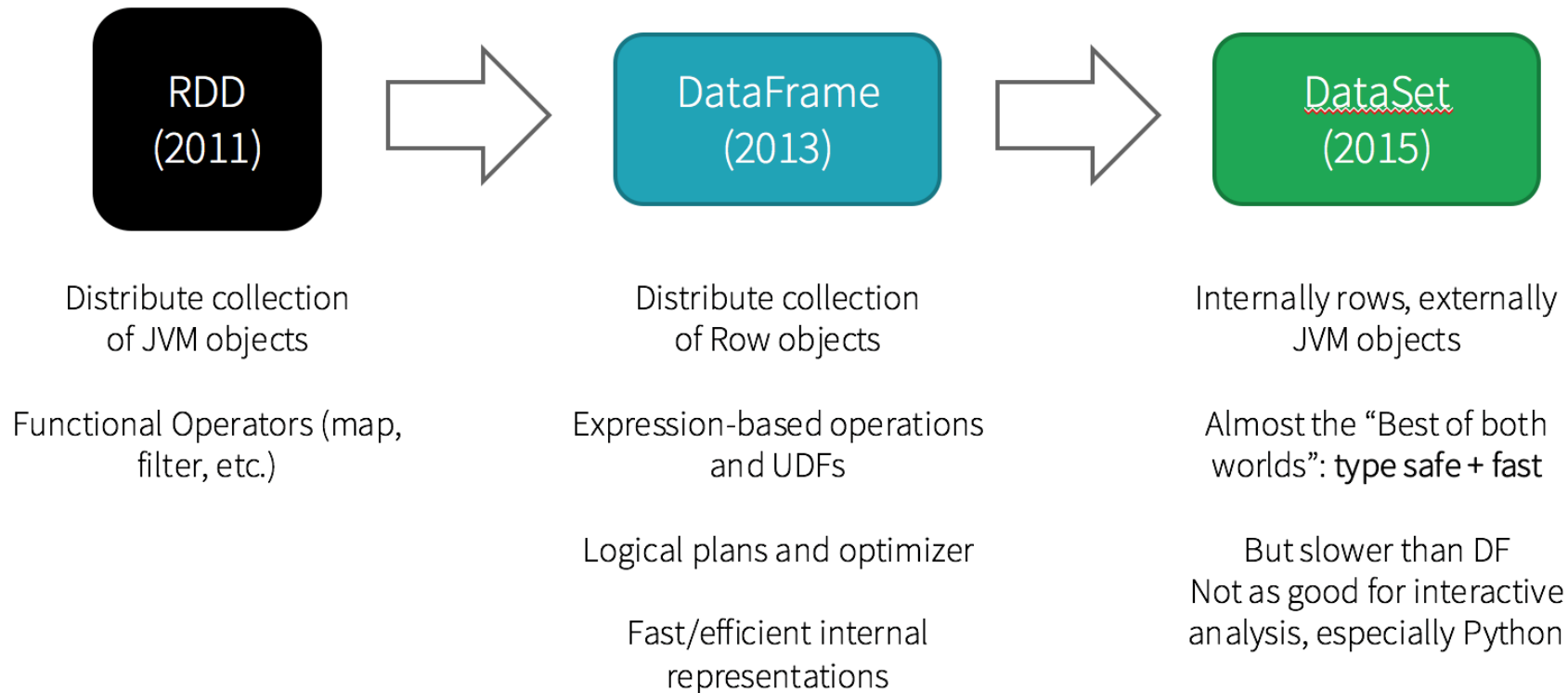
Bas niveau



Fonctionnalités essentielles de Spark

Abstractions de Spark

History of Spark APIs



Source : Databricks

Fonctionnalités essentielles de Spark

RDDs

- RDD : Resilient Distributed Dataset.
Abstraction de base dans Spark. Abstraction de mémoire distribuée.
Collection immuable et distribuée d'éléments qui peuvent être traités en parallèle.
- Résilients car tolérants aux pannes : en cas de défaillance d'un nœud, les données peuvent être reconstruites à partir des opérations de transformation appliquées à partir de l'ensemble d'origine.
- Les RDD peuvent être créés à partir de fichiers stockés dans HDFS (Hadoop Distributed File System), de données en mémoire ou de tout autre stockage pris en charge par Spark.
- Les RDD peuvent être transformés à l'aide d'opérations telles que map, filter et reduce, et peuvent être actionnés pour déclencher des calculs.

Fonctionnalités essentielles de Spark

RDDs

Dans Spark, les RDD (Resilient Distributed Datasets) peuvent être de deux types principaux :

- ➡ génériques,
- ➡ key-value.

RDD générique Ensemble de données réparties sur plusieurs nœuds du cluster Spark.
Chaque élément dans un RDD générique est un enregistrement indépendant sans structure spécifique.
Les opérations sur les RDD génériques sont appliquées à chaque élément individuel du RDD.
Ces RDD sont utilisés pour des opérations simples où la structure des données n'est pas importante.

```
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

Fonctionnalités essentielles de Spark

RDDs

RDD key-value Ensemble de paires clé-valeur réparties sur plusieurs nœuds du cluster Spark. Chaque élément dans un RDD key-value est une paire (clé, valeur). Les opérations sur les RDD key-value sont souvent effectuées en fonction des clés. Ces RDD sont utilisés pour des opérations qui impliquent des agrégations par clé, des jointures et d'autres opérations de traitement par clé.

```
rdd = sc.parallelize([(1, 'a'), (2, 'b'), (3, 'c')])
```

Les RDD key-value offrent des transformations et des actions spécifiques adaptées à ce type de données, telles que `reduceByKey`, `groupByKey`, `sortByKey`, `join`, etc.



Les RDD génériques sont utilisés pour traiter des enregistrements indépendants, tandis que les RDD key-value sont utilisés pour le traitement par clé et les opérations qui impliquent des relations clé-valeur.

Fonctionnalités essentielles de Spark

Quand utiliser des RDD ?

- ➡ Vous avez besoin de fonctionnalités spécifiques uniquement disponibles dans les RDDs, distinctes des DataFrames et des Datasets.
 - ➡ Vous souhaitez maintenir un code préexistant écrit avec des RDDs.
 - ➡ Vous devez manipuler des variables partagées.
-
- Dans ces situations, l'utilisation des RDDs est justifiée.
 - Cependant, pour la plupart des cas d'utilisation, il est recommandé d'opter pour des abstractions de plus haut niveau telles que les DataFrames ou les Datasets.
 - Tout code Spark est finalement compilé en RDD, mais ces abstractions offrent une meilleure expressivité et des optimisations intégrées pour de nombreuses tâches courantes.

Fonctionnalités essentielles de Spark

Quelques caractéristiques supplémentaires des RDD

- Liste de partitions

Les RDD sont divisés en partitions, qui sont des morceaux de données stockées sur différents nœuds du cluster. Les partitions sont la base du parallélisme dans Spark, car les opérations peuvent être exécutées sur différentes partitions en même temps.

- Fonction pour calculer chaque partition (en parallèle)

Chaque partition d'un RDD est traitée de manière indépendante, ce qui signifie que la fonction de transformation ou d'action est appliquée à chaque partition en parallèle.

- Liste de dépendances sur d'autres RDD

Un RDD peut être dérivé d'un ou plusieurs autres RDDs. Cela permet de construire des pipelines de traitement de données complexes dans Spark.

- En option, un Partitioner pour les RDD clé-valeur

Pour les RDD clé-valeur, un Partitioner peut être spécifié pour contrôler comment les paires clé-valeur sont distribuées sur les partitions. Cela peut être utile pour optimiser les opérations de jointure et de regroupement.

Fonctionnalités essentielles de Spark

Tolérance aux pannes des RDD

Tolérance aux pannes des RDD = élément crucial pour garantir la fiabilité des traitements de données distribués.

- Lorsqu'une panne survient, les séquences de transformations peuvent être recalculées en cas de perte de données.
- Les RDD conservent les informations de lignée nécessaires pour reconstruire les partitions perdues, ce qui permet de ne recalculer que les partitions affectées.
- Pour renforcer la résilience du système, il peut être parfois utile de pointer certains RDD vers un stockage stable, ce qui réduit le temps nécessaire pour reconstruire les partitions perdues, surtout pour les RDD impliqués dans des chaînes de transformations complexes. Cette fonctionnalité, unique aux RDD, est similaire à la mise en cache, mais elle stocke les données sur le disque plutôt que dans la mémoire.
- En utilisant `sc.setCheckpointDir()` pour définir un répertoire de vérification et `anRDD.checkpoint()` pour marquer un RDD pour la vérification, Spark est capable de gérer efficacement les pannes en réexécutant les étapes à partir du début avec une reconstitution judicieuse des données.

Fonctionnalités essentielles de Spark

Spark Context

- Spark Context : le point d'entrée pour créer des RDD.
- Plus précisément, le SparkContext représente la connexion à un cluster Spark et peut être utilisé pour créer des RDD, appliquer des transformations et des actions sur les données, configurer des paramètres d'application, et gérer les ressources du cluster.
- En Python, accès au contexte Spark via la variable `sc` (natif sur les clouders, convention dans les scripts que vous développez).

```
from pyspark import SparkContext

sc = SparkContext("local", "Exemple RDD")

data = [1, 2, 3, 4, 5]
first_rdd = sc.parallelize(data)
even_rdd = first_rdd.filter(lambda x: x % 2 == 0)
resultat = even_rdd.collect()
print("Nombres pairs : ", resultat)
sc.stop()
```

Fonctionnalités essentielles de Spark

Transformations

- Les transformations sont des opérations sur les RDD qui **créent un nouveau RDD à partir d'un RDD existant**.
- Les transformations sont **paresseuses** (lazy), ce qui signifie qu'elles ne sont pas évaluées immédiatement. Au lieu de cela, elles construisent un **plan de calcul**.
- Exemples de transformations : **map, filter, flatMap, reduceByKey, join, sortBy...**
- Les transformations sont généralement utilisées pour transformer les données en appliquant des opérations de filtrage, de projection, d'agrégation...



Fonctionnalités essentielles de Spark

Transformations sur un RDD

Function name	Purpose	Example	Result when applied to {1, 2, 3, 3}
map()	Apply a function to each element in the RDD and return an RDD of the result.	<code>rdd.map(lambda x: x + 1)</code>	{2, 3, 4, 4}
flatMap()	Apply a function to each element in the RDD and return an RDD of the contents of the iterators returned. Often used to extract words.	<code>rdd.flatMap(lambda x: range(x, 4))</code>	{1, 2, 3, 2, 3, 3, 3}
filter()	Return an RDD consisting of elements that pass the condition passed to filter().	<code>rdd.filter(lambda x: x != 1)</code>	{2, 3, 3}
distinct()	Remove duplicates.	<code>rdd.distinct()</code>	{1, 2, 3}
sample(withReplacement, fraction, [seed])	Sample an RDD, with or without replacement.	<code>rdd.sample(False, 0.5)</code>	Non-deterministic

Adapté de : <https://kks32-courses.gitbook.io/data-analytics/spark/rdd/common-rdd-operations>

Fonctionnalités essentielles de Spark

Transformations sur deux RDD

Function name	Purpose	Example	Result when applied to {1, 2, 3} and {3, 4, 5}
union()	Produce an RDD containing elements from both RDDs.	rdd.union(other)	{1, 2, 3, 3, 4, 5}
intersection()	Produce an RDD containing only elements found in both RDDs.	rdd.intersection(other)	{3}
subtract()	Remove the contents of one RDD (e.g., remove training data).	rdd.subtract(other)	{1, 2}
cartesian()	Cartesian product with the other RDD.	rdd.cartesian(other)	{{(1, 3), (1, 4), ... (3,5)}

Adapté de : <https://kks32-courses.gitbook.io/data-analytics/spark/rdd/common-rdd-operations>

Fonctionnalités essentielles de Spark

Actions

- Les actions sont des opérations sur les RDD qui déclenchent l'évaluation des transformations et déclenchent le traitement des données.
- Contrairement aux transformations, les actions déclenchent des calculs et des opérations réels sur les données.
- Exemples d'actions : `collect`, `take`, `reduce`, `count`, `saveAsTextFile`...
- Les actions sont généralement utilisées pour obtenir des résultats à partir des données, telles que la collecte des données sur le driver, le comptage des éléments, la sauvegarde des résultats dans un fichier...



Fonctionnalités essentielles de Spark

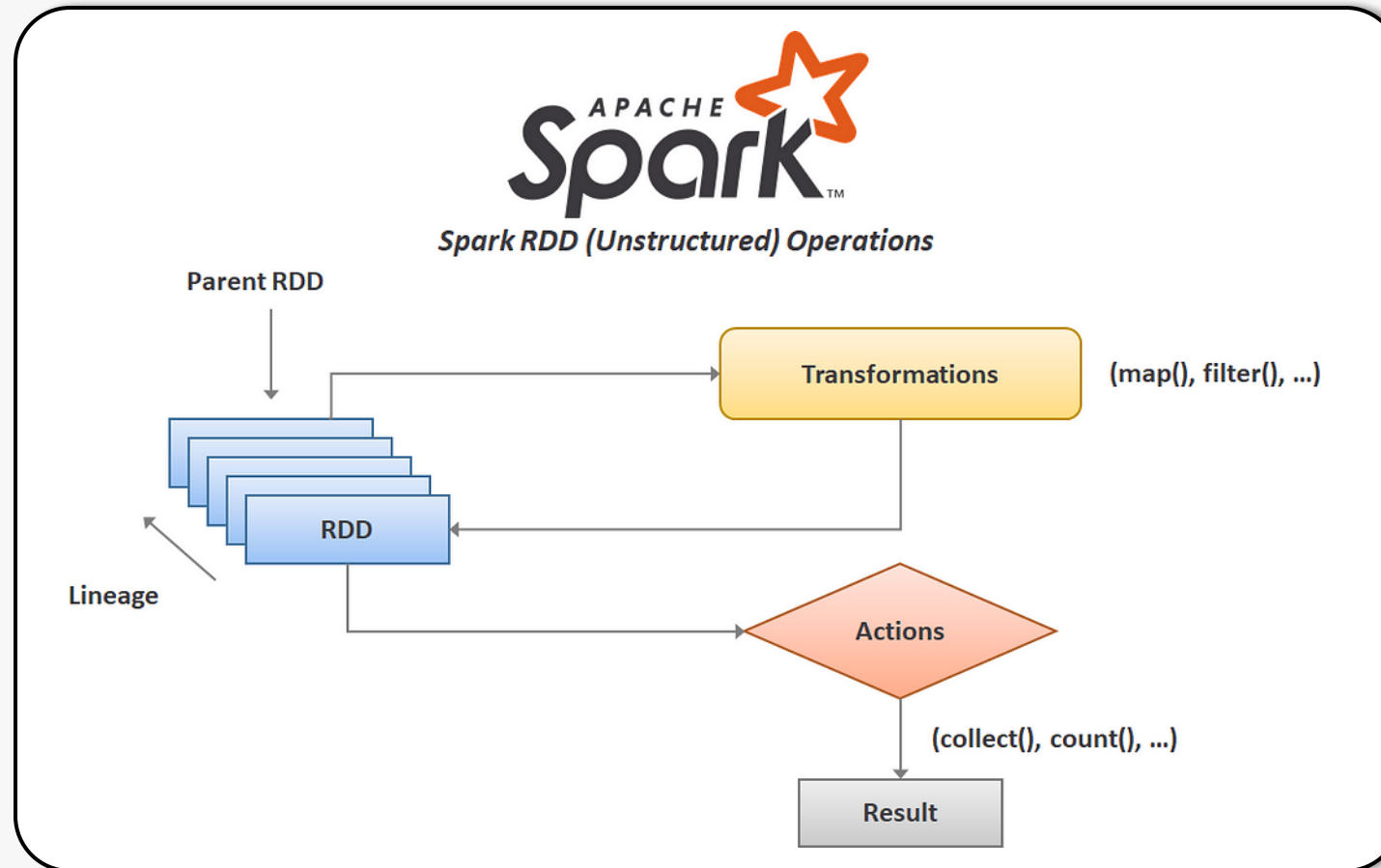
Actions sur un RDD

Function name	Purpose	Example	Result when applied to {1, 2, 3, 3}
collect()	Return all elements of RDD.	rdd.collect()	{1, 2, 3, 3}
count()	Return the number of all elements of RDD.	rdd.count()	4
countByValue()	Number of times each element occurs in the RDD.	rdd.countByValue()	{{(1, 1), (2, 1), (3, 2)}}
take(num)	Return num elements from the RDD.	rdd.take(2)	{1, 2}
top(num)	Return the top num elements from the RDD.	rdd.top(2)	{3, 3}
takeSample(withReplacement, num, [seed])	Return num elements at random.	rdd.takeSample(False, 1)	Non-deterministic
reduce(func)	Combine the elements of the RDD together in parallel (e.g., sum).	rdd.reduce(add)	9
fold(zero)(func)	Same as reduce() but with the provided zero value.	rdd.fold(0)(add)	9
aggregate((0, 0), seqOp, combOp)	Similar to reduce() but used to return a different type.	rdd.aggregate((0, 0)) ((x, y) => (x._1 + y, x._2 + 1), (x, y) (9, 4) => (x._1 + y._1, x._2 + y._2))	
foreach(func)	Apply the provided function to each element of the RDD.	def f(x): print(x) rdd.foreach(f)	Nothing

Adapté de : <https://kks32-courses.gitbook.io/data-analytics/spark/rdd/common-rdd-operations>

Fonctionnalités essentielles de Spark

RDD, transformations et actions



Source : <https://medium.com/analytics-vidhya/spark-rdd-low-level-api-basics-using-pyspark-a9a322b58f6i>

Fonctionnalités essentielles de Spark

Mise en cache

- La mise en cache est le processus par lequel les RDD sont stockés **en mémoire** pour une réutilisation ultérieure.
- Lorsqu'un RDD est mis en cache, les données sont stockées dans la mémoire RAM des nœuds du cluster Spark. Cela permet d'accélérer les calculs en évitant de re-calculer les RDD intermédiaires à chaque action.
- Pour cela, on utilise la méthode **cache()**.



```
rdd = sc.parallelize([("Alice", 1), ("Bob", 2), ("Alice", 3), ("Bob", 4)])  
# Appliquer une opération sur le RDD (par exemple, un groupBy)  
rdd_cached = rdd.cache()  
# Effectuer un calcul sur le RDD mis en cache, par exemple :  
rdd_count = rdd_cached.reduceByKey(lambda a, b: a + b)  
print(rdd_count.collect())
```

Fonctionnalités essentielles de Spark

Persistance

- La persistance est similaire à la mise en cache, mais elle offre plus de flexibilité en permettant de **spécifier le niveau de stockage des données**.
- Spark offre différents niveaux de stockage pour la persistance des RDD, tels que la mémoire, le disque ou une combinaison des deux.
- Pour cela, on utilise la méthode **persist()**.



```
lines = sc.textFile("chemin/vers/fichier.txt")
words = lines.flatMap(lambda line: line.split(" "))
persisted_words = words.persist()
word_counts = persisted_words.map(
    lambda word: (word, 1)
).reduceByKey(
    lambda x, y: x + y
)
word_counts.collect()
```

Fonctionnalités essentielles de Spark

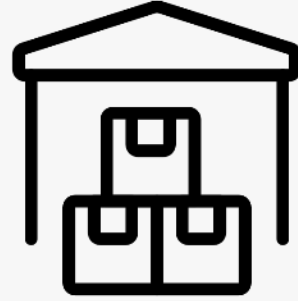
Niveaux de stockage

Storage Level	Description
MEMORY_ONLY	It stores the RDD as deserialized Java objects in the JVM. This is the default level. If the RDD doesn't fit in memory, some partitions will not be cached and recomputed each time they're needed.
MEMORY_AND_DISK	It stores the RDD as deserialized Java objects in the JVM. If the RDD doesn't fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
MEMORY_ONLY_SER (Java and Scala)	It stores RDD as serialized Java objects (i.e. one-byte array per partition). This is generally more space-efficient than deserialized objects.
MEMORY_AND_DISK_SER (Java and Scala)	It is similar to MEMORY_ONLY_SER, but spill partitions that don't fit in memory to disk instead of recomputing them.
DISK_ONLY	It stores the RDD partitions only on disk.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.	It is the same as the levels above, but replicate each partition on two cluster nodes.
OFF_HEAP (experimental)	It is similar to MEMORY_ONLY_SER, but store the data in off-heap memory. The off-heap memory must be enabled.

Source : <https://www.javatpoint.com/apache-spark-rdd-persistence>

Fonctionnalités essentielles de Spark

Niveaux de stockage



```
from pyspark import SparkContext
from pyspark import StorageLevel
sc = SparkContext("local", "Exemple de niveaux de stockage")
lines = sc.textFile("chemin/vers/fichier.txt")
lines.persist(StorageLevel.MEMORY_ONLY)
lines.count()
sc.stop()
```

Fonctionnalités essentielles de Spark

Caractéristiques impactant la pertinence de l'utilisation de `cache()`/`persist()`

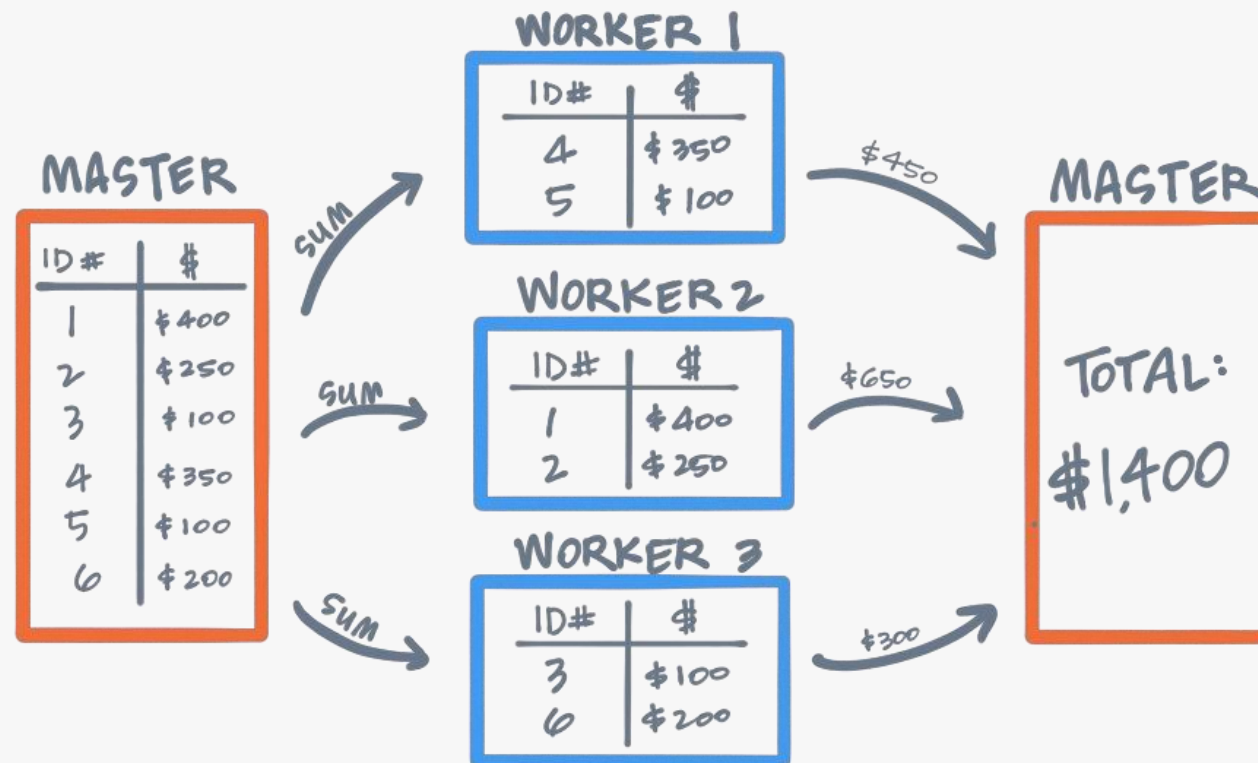
- ➡ Fréquence d'utilisation de l'abstraction
- ➡ Coût de la transformation sur l'abstraction
- ➡ Volume de l'abstraction



Fonctionnalités essentielles de Spark

Les partitions : unités de base pour la répartition des données et le parallélisme

Partition : Subdivision d'un ensemble de données (RDD ou DataFrame) dans Spark. Chaque partition contient un sous-ensemble des données, est stockée sur un seul nœud du cluster, et peut être traitée indépendamment par une tâche dédiée. L'ensemble des partitions est distribué entre les différents nœuds pour permettre le traitement parallèle.



Fonctionnalités essentielles de Spark

Les partitions : unités de base pour la répartition des données et le parallélisme

- ➡ **Taille et parallélisme** : le nombre et la taille des partitions dépendent de la volumétrie des données, du nombre de nœuds disponibles dans le cluster et de la configuration de Spark. Le choix d'une taille de partition adaptée est essentiel pour optimiser les performances : trop grandes, elles ralentissent le traitement ; trop petites, elles génèrent un surcoût de gestion. Le nombre de partitions dans un RDD ou un DataFrame détermine directement le niveau de parallélisme.
- ➡ Lors de la création d'un RDD ou d'un DataFrame (par exemple à partir de fichiers CSV ou de bases de données), Spark tente en général de répartir les données de façon équilibrée entre les partitions.
- ➡ Il peut être nécessaire d'ajuster explicitement le nombre de partitions pour optimiser le traitement parallèle, notamment à l'aide des transformations `repartition()` et `coalesce()`.

Une gestion efficace des partitions a un impact direct sur les performances : une répartition équilibrée et une taille de partition adaptée permettent de maximiser l'utilisation des ressources du cluster et d'accélérer l'exécution des traitements.

Fonctionnalités essentielles de Spark

Les partitions : unités de base pour la répartition des données et le parallélisme

```
data = [("Jean", 25), ("Anne", 30), ("Robert", 35), ("Julie", 40)]
rdd = sc.parallelize(data)

print("Nombre de partitions actuelles :", rdd.getNumPartitions())

rdd_repartition = rdd.repartition(2)
print("Nombre de partitions après repartition() :", rdd_repartition.getNumPartitions())

rdd_coalesce = rdd.coalesce(1)
print("Nombre de partitions après coalesce() :", rdd_coalesce.getNumPartitions())
```

Fonctionnalités essentielles de Spark

Partitionnement par plage ou par hachage

Partitionnement par hachage (hash partitioning)

Les enregistrements sont répartis entre les partitions en fonction de la valeur de hachage de leur clé. Chaque enregistrement se voit attribuer une clé de hachage dérivée de la clé spécifiée dans la fonction de partitionnement, et tous les enregistrements ayant la même clé de hachage sont regroupés dans la même partition.

Cette méthode est efficace pour répartir uniformément les données.

Partitionnement par plage (range partitioning)

Les enregistrements sont répartis entre les partitions selon des plages de valeurs d'une clé donnée. Les données sont d'abord triées selon la clé de partitionnement, puis distribuées dans les partitions en fonction des intervalles définis.

Cette méthode est particulièrement utile lorsque l'on souhaite préserver l'ordre des données au sein des partitions.

Exemple d'utilisation : pour des données temporelles ou des ensembles déjà triés selon une clé spécifique.

Fonctionnalités essentielles de Spark

Partitionnement par plage ou par hachage

Supposons avoir un RDD ou DataFrame contenant des logs de serveur avec la structure suivante :

timestamp	user_id	action
2025-11-12 08:15	101	login
2025-11-12 08:17	102	logout
2025-11-12 09:01	103	login

On souhaite effectuer des agrégations par jour ou par heure.

Avec un partitionnement par hachage, les données sont réparties aléatoirement dans les partitions. Chaque partition contiendra des logs de différents jours ou heures. Pour calculer les agrégations par heure, Spark devra répartir les données entre les partitions via un shuffle, ce qui est coûteux.

Avec un partitionnement par plage sur timestamp, on définit des plages de temps pour chaque partition :

Partition 1 : 2025-11-12 00:00 → 2025-11-12 06:00

Partition 2 : 2025-11-12 06:01 → 2025-11-12 12:00

Partition 3 : 2025-11-12 12:01 → 2025-11-12 18:00

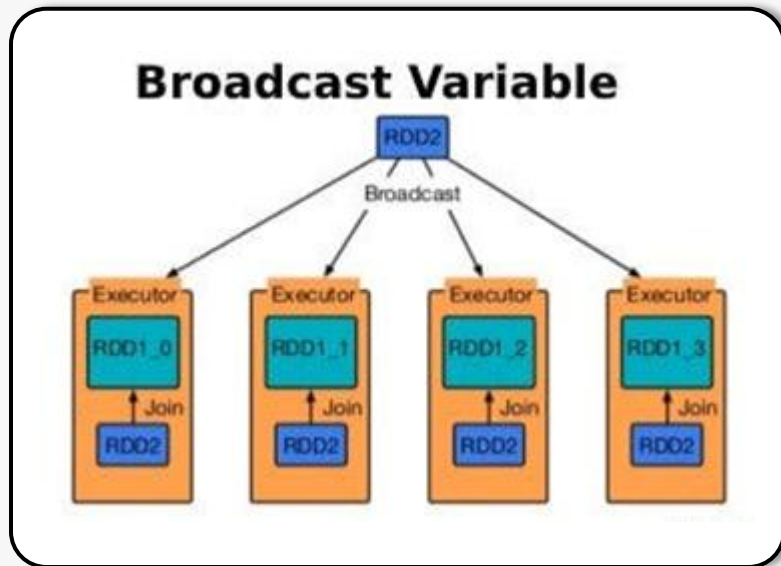
Partition 4 : 2025-11-12 18:01 → 2025-11-12 23:59

Les données restent triées par timestamp dans chaque partition. Les calculs par plage de temps (par exemple par heure) peuvent se faire localement dans chaque partition, sans shuffle supplémentaire.

Fonctionnalités essentielles de Spark

Variables partagées : variables diffusées et accumulateurs

- Les variables partagées (shared variables) sont des fonctionnalités de Spark qui permettent de distribuer efficacement les données aux tâches exécutées sur différents nœuds du cluster.
- Accumulateur** (accumulator) : Pour agréger des données provenant de toutes les tâches dans un résultat partagé.
- Variable diffusée** (broadcast variable) : Pour diffuser une abstraction à tous les nœuds de travail et l'utiliser sans avoir à la renvoyer à travers le cluster.



Accumulators

Accumulator	Value
counter	48

Tasks

Index	ID	Attempt	Status	Locality Level	Executor ID / Host	Launch Time	Duration	GC Time	Accumulators	Errors
0	0	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms			
1	1	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 1	
2	2	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 2	
3	3	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 3	
4	4	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 4	
5	5	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 5	
6	6	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 6	
7	7	0	SUCCESS	PROCESS_LOCAL	driver / localhost	2019/04/21 10:16:41	17 ms		counter: 7	

Fonctionnalités essentielles de Spark

Variables diffusées

- Permet au programmeur de garder une variable en lecture seule en cache sur chaque machine plutôt que d'en envoyer une copie avec les tâches.
- La variable n'est pas envoyée aux nœuds plus d'une fois.
- Spark utilise un algorithme de diffusion pour réduire les coûts de communication.

```
broadcast_var = sc.broadcast([1, 2, 3, 4, 5])
```

Fonctionnalités essentielles de Spark

Accumulateurs

- C'est une variable partagée aux workers et qui autorise l'écriture.
-
-

A votre avis, quelles propriétés doivent être vérifiées par l'opération qu'on cherche à appliquer à notre accumulateur ?

Fonctionnalités essentielles de Spark

Accumulateurs

- C'est une variable partagée aux workers et qui autorise l'écriture.
- Ces variables ne peuvent être ajoutées que par une opération **associative** et **commutative**.
- Les mises à jour de l'accumulateur sont effectuées uniquement par les ???.

Compléter les ???

Fonctionnalités essentielles de Spark

Accumulateurs

- C'est une variable partagée aux workers et qui autorise l'écriture.
- Ces variables ne peuvent être ajoutées que par une opération **associative** et **commutative**.
- Les mises à jour de l'accumulateur sont effectuées uniquement par les **actions**.

```
accum = sc.accumulator(0)
rdd.foreach(lambda x: accum.add(x))
```

Fonctionnalités essentielles de Spark

Note sur l'utilisation des variables partagées dans les versions récentes de Spark



NEW

À partir de Spark 2.2, Spark peut optimiser l'utilisation des variables diffusées de manière transparente dans de nombreux cas. Même s'il est toujours possible d'utiliser `sc.broadcast()` pour diffuser des variables, c'est beaucoup moins fréquent qu'avant !

Fonctionnalités essentielles de Spark

Pyspark



The screenshot shows the Apache Spark PySpark Overview page. The header includes the Apache Spark logo, navigation links (Overview, Getting Started, User Guides, API Reference, Development, Migration Guides), and a version dropdown set to 3.5.1. A search bar is on the left. The main content area is titled 'PySpark Overview' and includes the date 'Feb 24, 2024' and version 'master'. It lists useful links: Live Notebook, GitHub, Issues, Examples, and Community. The text describes PySpark as the Python API for Apache Spark, enabling real-time, large-scale data processing in a distributed environment. It also mentions that PySpark combines Python's learnability with Spark's power for data processing and analysis. At the bottom, four blue boxes link to Spark SQL and DataFrames, Pandas API on Spark, Structured Streaming, and Machine Learning (MLlib).

APACHE
Spark

Overview Getting Started User Guides API Reference Development Migration Guides 3.5.1

Search the docs ...

PySpark Overview

Date: Feb 24, 2024 **Version:** master

Useful links: [Live Notebook](#) | [GitHub](#) | [Issues](#) | [Examples](#) | [Community](#)

PySpark is the Python API for Apache Spark. It enables you to perform real-time, large-scale data processing in a distributed environment using Python. It also provides a PySpark shell for interactively analyzing your data.

PySpark combines Python's learnability and ease of use with the power of Apache Spark to enable processing and analysis of data at any size for everyone familiar with Python.

PySpark supports all of Spark's features such as Spark SQL, DataFrames, Structured Streaming, Machine Learning (MLlib) and Spark Core.

Spark SQL and DataFrames Pandas API on Spark Structured Streaming Machine Learning MLlib

<https://spark.apache.org/docs/latest/api/python/index.html>

Fonctionnalités essentielles de Spark

Etude de code



Manipulations de RDD
Lecture et écriture de données
Transformations et actions

Question 4

Abstractions de Spark

Quelle abstraction de Spark est utilisée pour représenter un ensemble de données distribué, immuable et partitionné ?

- ☐ RDD
- ☐ DataFrame
- ☐ Dataset
- ☐ SparkContext

Question 5

Librairie Python pour Spark

Quel module Python est généralement utilisé pour interagir avec Spark ?

- ☐ PySpark
- ☐ SparkPy
- ☐ SparkTools
- ☐ PySparkle

Question 6

spark-submit

Quelle option est utilisée pour spécifier l'URL du maître dans la commande spark-submit ?

- ☐ --cluster-url
- ☐ --master-url
- ☐ --url
- ☐ --master

02 bis

Quelques compléments

Quelques compléments

Le repartitionnement d'un RDD est-il une modification ?

- Non, on ne peut pas modifier un RDD, même pour uniquement le repartitionner. On crée un nouveau RDD.

Quelques compléments

Comment `RDD.take(n)` est-il distribué en pratique ?

- Spark interroge les partitions jusqu'à atteindre `n`. Par exemple : `take(100)` avec 3 partitions de 80 -> les 2 premières partitions sont interrogées, Spark récupère 160 points de données et en affiche 100. Le nombre de partitions nécessaire pour satisfaire `n` est estimé par Spark à partir du nombre d'éléments de la première partition. Si besoin, il est mis à jour.
- <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.take.html>

03

Spark SQL

Spark SQL

Spark SQL



- Spark SQL : module Spark dédié au traitement de données structurées.
- Contrairement à l'API de base Spark RDD, les interfaces fournies par Spark SQL fournissent à Spark plus d'informations sur la structure à la fois des données et des calculs effectués.
- En interne, Spark SQL utilise ces informations supplémentaires pour effectuer des optimisations supplémentaires.
- Il existe plusieurs façons d'interagir avec Spark SQL (SQL et API DataFrame), mais, lors du calcul d'un résultat, le même moteur d'exécution est utilisé, indépendamment de l'API/langage utilisé pour exprimer le calcul.

Spark SQL

Requêtes SQL

- Spark SQL permet d'exécuter des requêtes SQL de manière distribuée.
- Il peut lire des données issues de multiples sources, comme Parquet, JSON, ORC, Avro ou encore des tables Delta Lake.
- Lorsqu'une requête SQL est lancée depuis un autre langage de programmation, les résultats sont renvoyés sous forme de Dataset ou de DataFrame.
- L'interaction avec l'interface SQL peut se faire depuis la ligne de commande, ou via des connecteurs standards tels que JDBC et ODBC.



Spark SQL

Datasets et DataFrames

Dataset

Collection de données distribuée.

Introduite avec Spark 1.6, cette interface combine les avantages des RDD (typage fort, utilisation de fonctions lambda) avec les optimisations du moteur d'exécution de Spark SQL.

Un Dataset peut être construit à partir d'objets JVM et manipulé au moyen de transformations fonctionnelles comme `map`, `flatMap` ou `filter`.

L'API Dataset est disponible en Scala et Java, mais pas en Python ni en R. Toutefois, en raison de leur nature dynamique, ces langages bénéficient déjà de fonctionnalités proches : par exemple, accéder directement aux champs d'une ligne par leur nom (`row.columnName`).

Spark SQL

Datasets et DataFrames

DataFrame

Collection distribuée organisée en colonnes nommées.

Conceptuellement équivalent à une table dans une base de données relationnelle ou à un data frame en R/Python, mais il bénéficie d'optimisations supplémentaires grâce au moteur d'exécution de Spark SQL.

Peut être construit à partir de différentes sources : fichiers de données structurées (ex. Parquet, JSON, CSV), bases de données externes accessibles via JDBC, ou encore à partir de RDD existants.

L'API DataFrame est disponible en Scala, Java, Python et R. En pratique, un DataFrame correspond à un `Dataset[Row]`. Dans l'API Scala, DataFrame n'est qu'un alias de type pour `Dataset[Row]`, tandis que dans l'API Java, il est manipulé directement sous la forme `Dataset<Row>`.

Etant donné que cette formation est orientée vers une utilisation de Spark avec Python, nous nous focaliserons sur les DataFrames.

Spark SQL

Fonctions PySpark classiques pour la préparation des données (1/2)

Catégorie	Fonction/méthode	Description
Chargement	<code>spark.read.csv()</code> <code>spark.read.format()</code> <code>spark.read.jdbc()</code> <code>spark.readStream()</code>	Charge les données à partir d'un fichier CSV. Charge les données à partir de différents formats de fichiers (Parquet, JSON...). Charge les données à partir d'une base de données JDBC. Charge les données en streaming à partir de différentes sources.
Exploration	<code>DataFrame.printSchema()</code> <code>DataFrame.show()</code> <code>DataFrame.describe()</code>	Affiche le schéma des données. Affiche les premières lignes du DataFrame. Affiche des statistiques descriptives pour les colonnes numériques.
Nettoyage	<code>DataFrame.dropna()</code> <code>DataFrame.fillna()</code> <code>DataFrame.dropDuplicates()</code>	Supprime les lignes contenant des valeurs manquantes. Remplace les valeurs manquantes par une valeur spécifiée. Supprime les lignes en double.
Transformation	<code>DataFrame.select()</code> <code>DataFrame.filter()</code> <code>DataFrame.groupBy()</code> <code>DataFrame.join()</code> <code>DataFrame.withColumn()</code>	Sélectionne un sous-ensemble de colonnes du DataFrame. Filtre les lignes du DataFrame en fonction d'une condition. Regroupe les données en fonction des valeurs d'une ou plusieurs colonne(s). Effectue une jointure entre deux DataFrames. Ajoute une nouvelle colonne à partir d'une transformation sur les valeurs existantes d'une colonne (similaire à <code>.map()</code> sur les RDD).

Spark SQL

Fonctions PySpark classiques pour la préparation des données (2/2)

Catégorie	Fonction/méthode	Description
Feature engineering		Ajout de colonnes avec des expressions SQL ou des fonctions UDF Utilisation de fonctions intégrées comme concat(), substring(), split()...
Partitionnement	DataFrame.repartition() DataFrame.coalesce()	Réorganise les partitions du DataFrame. Réduit le nombre de partitions du DataFrame.
Enregistrement	DataFrame.write.format() DataFrame.write.saveAsTable() DataFrame.writeStream()	Ecrit les données dans différents formats de fichiers. Enregistre le DataFrame en tant que table dans le catalogue Spark SQL. Ecrit les données en streaming vers une destination.

Spark SQL

Spark Session (et Spark Context le retour)

Spark Context (sc)

Point d'entrée principal dans les anciennes versions de Spark (avant 2.0).
Représente la connexion au cluster Spark et gère les ressources de calcul.
Responsable de la création des RDDs (Resilient Distributed Datasets), la principale abstraction de données dans Spark.
Opérations sur les RDDs effectuées via ce Spark Context.

Spark Session (spark)



Point d'entrée principal dans Spark depuis Spark 2.0.
Regroupe les fonctionnalités de SparkContext, SQLContext et HiveContext dans une seule interface.
Fournit des fonctionnalités avancées pour travailler avec des DataFrames et des Datasets structurés.
Fournit des fonctionnalités pour interagir avec Spark SQL, comme l'exécution de requêtes SQL sur des DataFrames.
Facilite la création et la gestion de DataFrames, ainsi que l'exécution de tâches dans un environnement Spark.

Spark SQL

Spark Session : point de départ

Le point d'entrée dans toutes les fonctionnalités de Spark est la classe `SparkSession`.

```
from pyspark.sql import SparkSession
spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
```

Spark SQL

Créer des DataFrames

Avec une `SparkSession`, une application peut créer des `DataFrames` à partir d'un `RDD` existant, de tables externes ou de diverses sources de données prises en charge par Spark.

Par exemple, le code suivant crée un `DataFrame` basé sur le contenu d'un fichier JSON :

```
# spark is an existing SparkSession
df = spark.read.json("examples/src/main/resources/people.json")
# Displays the content of the DataFrame to stdout
df.show()
# +-----+-----+
# |  age|  name|
# +-----+-----+
# |null|Michael|
# | 30|  Andy|
# | 19| Justin|
# +-----+-----+
```

Spark SQL

Opérations sur les DataFrames (Datasets non typés)

- Les DataFrames fournissent un langage spécifique au domaine pour la manipulation de données structurées en Scala, Java, Python et R (parfois appelé DSL pour Domain-Specific Language).
- Rappel : Les DataFrames sont simplement des Datasets de lignes dans l'API Scala et Java. Ces opérations sont également appelées "transformations non typées" par opposition aux "transformations typées" associées aux Datasets Scala/Java.
- En Python, il est possible d'accéder aux colonnes d'un DataFrame soit par attribut (`df.age`) soit par indexation (`df['age']`). Bien que le premier soit pratique pour l'exploration interactive des données, il est fortement recommandé d'utiliser le second, qui est évolutif et ne créera pas de conflit lorsque les noms de colonnes seront également des attributs de la classe DataFrame.

Spark SQL

Opérations sur les DataFrames (Datasets non typés)

```
# Print the schema in a tree format
df.printSchema()
# root
# |-- age: long (nullable = true)
# |-- name: string (nullable = true)

# Select only the "name" column
df.select("name").show()
# +-----+
# | name|
# +-----+
# |Michael|
# | Andy|
# | Justin|
# +-----+

# Select everybody, but increment the age by 1
df.select(df['name'], df['age'] + 1).show()
# +-----+-----+
# | name|(age + 1)|
# +-----+-----+
# |Michael| null|
# | Andy| 31|
# | Justin| 20|
# +-----+-----+
```

```
# Select people older than 21
df.filter(df['age'] > 21).show()
# +---+---+
# |age|name|
# +---+---+
# | 30|Andy|
# +---+---+

# Count people by age
df.groupBy("age").count().show()
# +---+---+
# | age|count|
# +---+---+
# | 19| 1|
# | null| 1|
# | 30| 1|
# +---+---+
```

Spark SQL

Jointures

Left Join :

```
result = df1.join(df2, on='key', how='left')
```

Right Join :

```
result = df1.join(df2, on='key', how='right')
```

Inner Join :

```
result = df1.join(df2, on='key', how='inner')
```

Outer Join :

```
result = df1.join(df2, on='key', how='outer')
```

Spark SQL

Vues temporaires globales

Les vues temporaires dans Spark SQL sont spécifiques à la session et disparaîtront si la session qui les crée se termine. Pour disposer d'une vue temporaire partagée entre toutes les sessions et conservée jusqu'à la fin de l'application Spark, il est possible de créer une vue temporaire globale. La vue temporaire globale est liée à une base de données préservée par le système, `global_temp`, et il faut utiliser le nom qualifié pour y faire référence (par exemple `SELECT * FROM global_temp.view1`).

```
# Register the DataFrame as a global temporary view
df.createGlobalTempView("people")

# Global temporary view is tied to a system preserved database
spark.sql("SELECT * FROM global_temp.people").show()
# +-----+-----+
# | age| name|
# +-----+-----+
# |null|Michael|
# | 30| Andy|
# | 19| Justin|
# +-----+-----+

# Global temporary view is cross-session
spark.newSession().sql("SELECT * FROM global_temp.people").show()
# +-----+-----+
# | age| name|
# +-----+-----+
# |null|Michael|
# | 30| Andy|
# | 19| Justin|
# +-----+-----+
```

Spark SQL

Interopérabilité avec les RDD

Spark SQL prend en charge deux méthodes différentes pour convertir des RDD existants en Dataframes.

- ➡ La première méthode utilise la « réflexion » (reflection) pour inférer le schéma d'un RDD qui contient des types d'objets spécifiques. Cette approche basée sur la réflexion conduit à un code plus concis et fonctionne bien lorsque le schéma est déjà connu lors de l'écriture de l'application Spark.
- ➡ La deuxième méthode pour créer des Dataframes consiste à utiliser une interface programmatique qui permet de construire un schéma, puis de l'appliquer à un RDD existant. Bien que cette méthode soit plus verbeuse, elle permet de construire des Dataframes lorsque les colonnes et leurs types ne sont pas connus avant l'exécution.

Spark SQL

Interopérabilité avec les RDD

```
from pyspark.sql import Row

sc = spark.sparkContext
# Load a text file and convert each line to a Row.
lines = sc.textFile("examples/src/main/resources/people.txt")
parts = lines.map(lambda l: l.split(","))
people = parts.map(lambda p: Row(name=p[0], age=int(p[1])))

# Infer the schema, and register the DataFrame as a table.
schemaPeople = spark.createDataFrame(people)
schemaPeople.createOrReplaceTempView("people")

# SQL can be run over DataFrames that have been registered as a table.
teenagers = spark.sql("SELECT name FROM people WHERE age >= 13 AND age <= 19")

# The results of SQL queries are Dataframe objects.
# rdd returns the content as an :class:`pyspark.RDD` of :class:`Row`.
teenNames = teenagers.rdd.map(lambda p: "Name: " + p.name).collect()
for name in teenNames:
    print(name)
# Name: Justin
```

Spark SQL

Interopérabilité avec les RDD

```
# Import data types
from pyspark.sql.types import StringType, StructType, StructField

sc = spark.sparkContext

# Load a text file and convert each line to a Row.
lines = sc.textFile("examples/src/main/resources/people.txt")
parts = lines.map(lambda l: l.split(","))
# Each line is converted to a tuple.
people = parts.map(lambda p: (p[0], p[1].strip()))

# The schema is encoded in a string.
schemaString = "name age"

fields = [StructField(field_name, StringType(), True) for field_name in schemaString.split()]
schema = StructType(fields)

# Apply the schema to the RDD.
schemaPeople = spark.createDataFrame(people, schema)

# Creates a temporary view using the DataFrame
schemaPeople.createOrReplaceTempView("people")

# SQL can be run over DataFrames that have been registered as a table.
results = spark.sql("SELECT name FROM people")

results.show()
# +-----+
# | name|
# +-----+
# |Michael|
# | Andy|
# | Justin|
# +-----+
```

Question 7

API de Spark

Quelle est la principale différence entre un DataFrame et un RDD dans Spark ?

- ☐ Un DataFrame est une abstraction de données structurées tandis qu'un RDD est une collection distribuée d'objets.
- ☐ Un DataFrame est une collection d'enregistrements tandis qu'un RDD est une collection d'objets arbitraires.
- ☐ Un DataFrame ne peut pas être utilisé avec Spark SQL.
- ☐ Un RDD est une abstraction plus récente que le DataFrame.

Question 8

Les méthodes des DataFrames de Spark

Comment peut-on créer un DataFrame à partir d'un fichier CSV dans Spark ?

- ☐ En utilisant la fonction `spark.createDataFrame()`.
- ☐ En utilisant la fonction `spark.load.csv()`.
- ☐ En utilisant la fonction `spark.read.csv()`.
- ☐ En convertissant un RDD en DataFrame.

Question 9

Les méthodes des DataFrames de Spark

Quelle opération est utilisée pour sélectionner des colonnes spécifiques dans un DataFrame ?

- ☐ `DataFrame.filter()`
- ☐ `DataFrame.select()`
- ☐ `DataFrame.groupBy()`
- ☐ `DataFrame.orderBy()`

Question 10

Les méthodes des DataFrames de Spark

Quelle opération est utilisée pour filtrer les lignes d'un DataFrame en fonction d'une condition donnée ?

- ☐ `DataFrame.select()`
- ☐ `DataFrame.groupBy()`
- ☐ `DataFrame.filter()`
- ☐ `DataFrame.sort()`

Question 11

Les méthodes des DataFrames de Spark

Comment peut-on renommer une colonne dans un DataFrame ?

- ☐ En utilisant la fonction `DataFrame.renameColumn()`.
- ☐ En utilisant la fonction `DataFrame.alterColumn()`.
- ☐ En utilisant la fonction `DataFrame.withColumnRenamed()`.
- ☐ En utilisant la fonction `DataFrame.updateColumn().DataFrame.filter()`

Fonctionnalités essentielles de Spark

TP



Introduction pratique Spark SQL
Chargement, exploration,
traitements sur les dataframes
(SQL et DSL)

Spark SQL

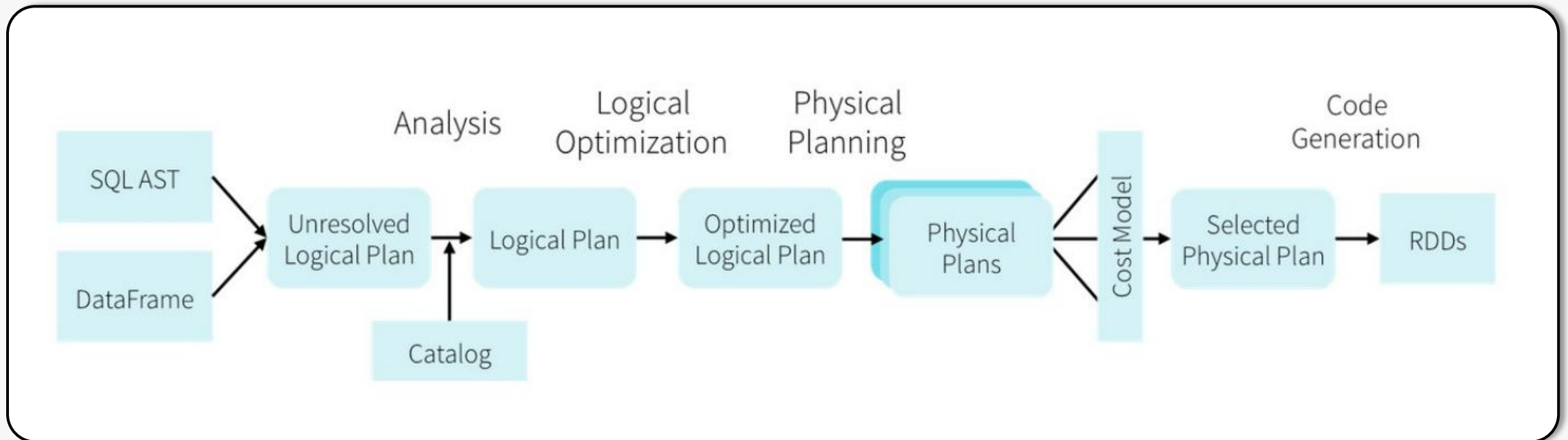
Retour sur les optimisations : fonctionnement de Catalyst

Nous avons mentionné le caractère lazy des transformations.

Cela permet d'effectuer des optimisations.

Catalyst : Optimizer dans Spark SQL.

Fonctionnement :



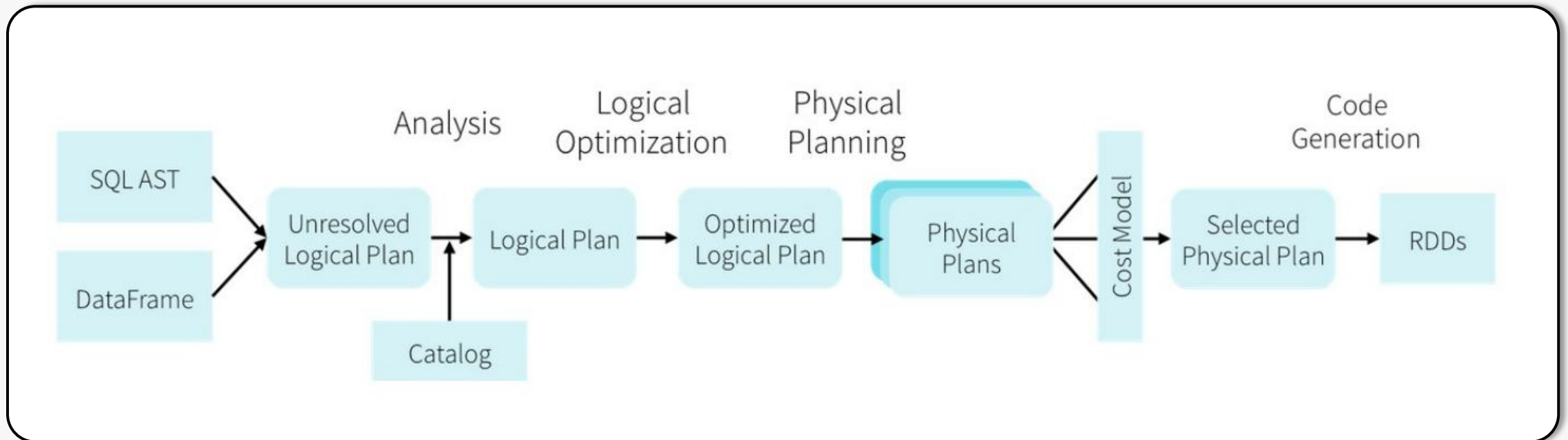
Spark SQL

Retour sur les optimisations : fonctionnement de Catalyst

Unresolved logical plan (plan logique non résolu)

Spark construit un plan logique, mais sans vérifier la validité des tables ou colonnes.

C'est une représentation abstraite de la requête sans aucune vérification sur les noms de colonnes, fonctions, ou relations entre tables.



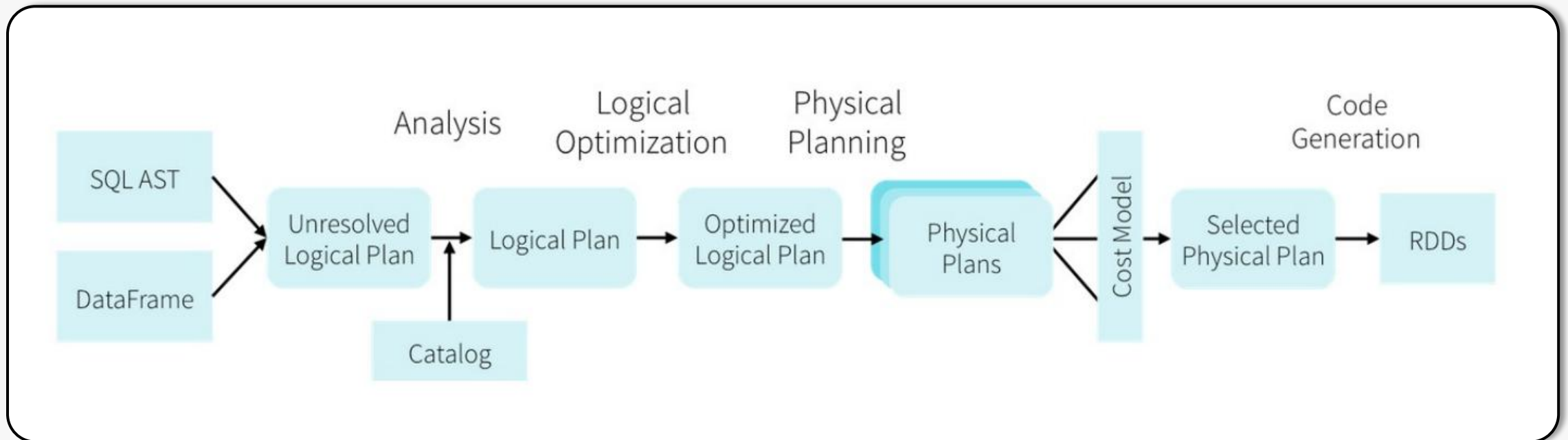
Spark SQL

Retour sur les optimisations : fonctionnement de Catalyst

Logical plan (plan logique résolu)

Spark valide le schéma et les relations.

Une fois que Spark a analysé la requête, il remplace les noms de tables et colonnes par des objets internes valides.



Spark SQL

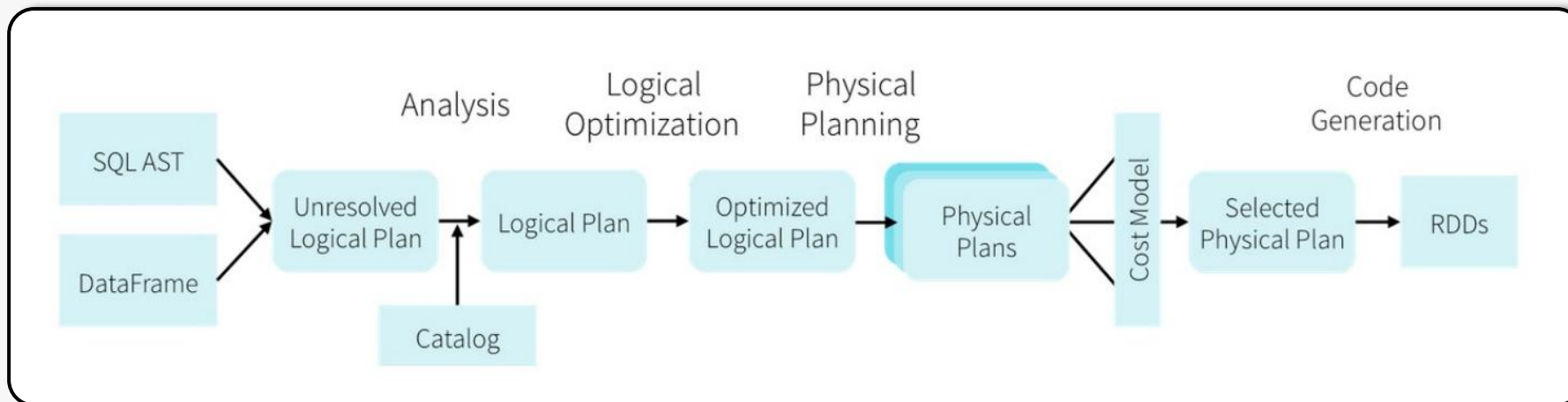
Retour sur les optimisations : fonctionnement de Catalyst

Optimized logical plan (plan logique optimisé)

Spark applique des transformations pour améliorer l'efficacité du plan logique.

Catalyst optimise la requête en appliquant plusieurs règles d'optimisation, comme :

- le « pruning » des colonnes inutiles (ne garder que celles nécessaires).
- le « pushdown » des filtres (exécuter les filtres au plus tôt pour limiter le volume de données).
- la fusion d'opérations (ex: fusionner deux filters successifs en un seul).



Spark SQL

Retour sur les optimisations : fonctionnement de Catalyst

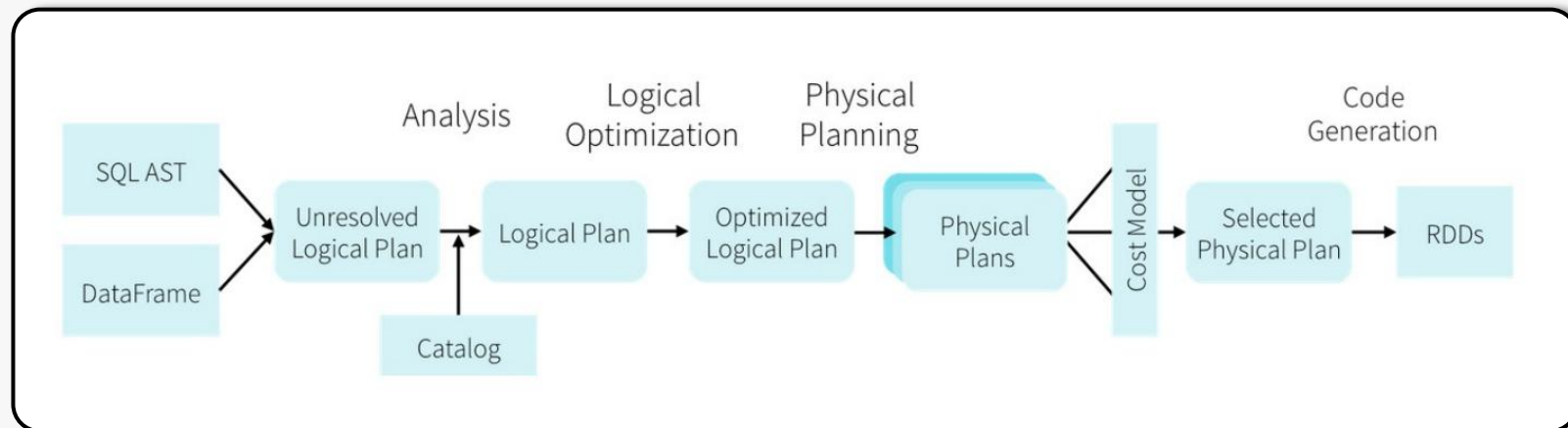
Physical plan(s)

Spark génère plusieurs stratégies d'exécution possibles.

Le plan logique optimisé est traduit en plusieurs plans physiques qui représentent les différentes stratégies d'exécution sur le cluster.

Par exemple, les stratégies possibles pour une jointure:

- Sort-Merge Join (pertinent si les données sont triées).
- Broadcast Hash Join (pertinent si une table est petite, elle est alors diffusée à tous les workers).
- Shuffle Hash Join.



Spark SQL

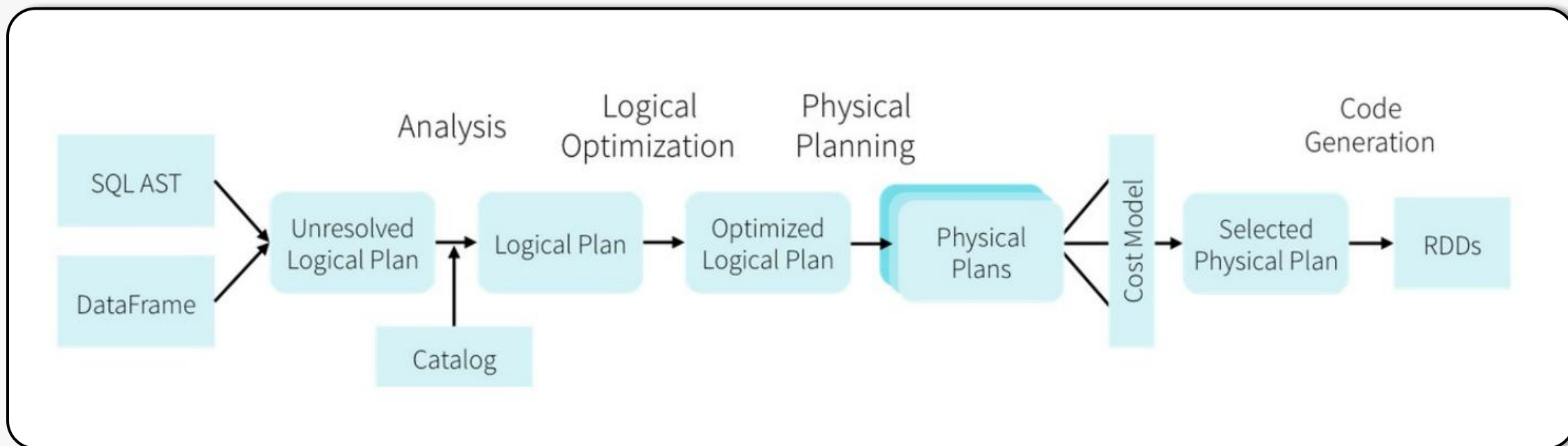
Retour sur les optimisations : fonctionnement de Catalyst

Selected physical plan

Catalyst choisit la meilleure stratégie et l'exécute.

Spark choisit le plan le plus efficace en fonction du coût estimé (temps, mémoire, ressources).

Une fois sélectionné, ce plan est converti en exécutions RDD et lancé sur le cluster.



Exploitation de Spark SQL

Les formats de données : binaires

- Représentation de données sous forme de séquences de bits (0 et 1), très utilisée pour stocker, traiter et transmettre des informations.
- Fichier binaire : formé d'une suite d'octets composés chacun de 8 bits. Chaque fichier est structuré selon un format spécifique permettant de l'interpréter.
- Les fichiers binaires sont généralement plus **compacts** que les fichiers textuels, car ils ne nécessitent pas de caractères de contrôle ou de délimiteurs. Plus efficaces en termes d'espace de stockage.
- D'autres avantages :
 - ➡ **Vitesse de lecture et d'écriture**
 - ➡ **Structure flexible**
 - ➡ **Interopérabilité**
 - ➡ **Sécurité**

Les formats de données : binaires

- Le format binaire le plus couramment utilisé dans l'écosystème Spark est le format **Parquet**.
- Parquet utilise des techniques de **compression** et d'encodage efficaces pour réduire la taille des données stockées : économie de l'espace de stockage dans les environnements distribués.
- Format de stockage **orienté colonnes** (stocke les données par colonne plutôt que par ligne) : permet une lecture sélective des colonnes lors de l'exécution de requêtes, ce qui peut considérablement améliorer les performances des requêtes analytiques.
- Parquet prend en charge les **schémas intégrés** (les informations sur le schéma des données sont stockées directement dans le fichier Parquet). Cela permet à Spark de lire efficacement les données sans avoir besoin de déduire le schéma lors de la lecture, ce qui réduit les coûts de traitement.
- Les fichiers Parquet peuvent être divisés en blocs de données qui peuvent être traités en **parallèle** sur différents nœuds d'un cluster Spark.
- Parquet est largement utilisé dans l'écosystème Hadoop et est **pris en charge par de nombreux autres outils et frameworks**, tels que Hive, Impala, Presto... Cela facilite l'intégration de Spark avec ces autres composants Big Data.

Exploitation de Spark SQL

Les formats de données : binaires

- On lit et écrit des données Parquet avec Spark en utilisant les méthodes `spark.read.parquet()` et `DataFrame.write.parquet()`.
- Il est possible de lire et écrire des données binaires d'autres formats. La plupart du temps, la syntaxe exige de spécifier le format approprié en utilisant la méthode `format()` et fournir le chemin d'accès au fichier ou au répertoire contenant les données.
- Par exemple, Avro est un format de données binaire avec un schéma. Les méthodes associées sont `spark.read.format("avro")` et `DataFrame.write.format("avro")`.
- ORC (Optimized Row Columnar) : autre format de stockage de données orienté colonnes, similaire à Parquet. Méthodes `spark.read.orc()` et `DataFrame.write.orc()`.
- SequenceFile : format de fichier binaire utilisé dans Hadoop pour stocker des paires clé-valeur séquentielles. Méthodes `spark.read.format("sequencefile")` et `DataFrame.write.format("sequencefile")`.
- BinaryFile : `spark.read.binaryFiles()`

Fonctionnalités essentielles de Spark

TP



**Nettoyage de données de
produits alimentaires avec Spark
SQL**

03 bis

Quelques compléments

Quelques compléments

Existe-t-il un moyen d'accéder aux métriques de complexité calculées par spark, en particulier par le modèle de coût de catalyst ?

- Non. On peut regarder `.explain()` voire les logs détaillés d'une part, et, a posteriori, les métriques d'exécution dans SparkUI d'autre part. Mais Spark n'expose pas d'API pour accéder aux « métriques de complexité » liées au modèle de coût.

Quelques compléments

Y a-t-il une différence entre les syntaxes `df["col_name"]`, `"col_name"` et `col("col_name")` ?

Syntaxe	+	-
<code>col("col_name")</code>	Meilleure pratique en particulier pour les transformations avancées, et pour le reste par souci d'homogénéité	Nécessite un import
<code>df["col_name"]</code>	Sélection assez simple	Peut poser problème dans des cas de noms complexes
<code>"col_name"</code>	Sélection très simple	Ne fonctionne pas si on souhaite effectuer des calculs

Néanmoins, il n'y a pas de différence de performance entre `df["col"]`, `"col"` et `col("col")` si la syntaxe est correcte et que l'opération est valide (Catalyst va transformer la requête en la même exécution optimisée). `col()` rend également les erreurs parfois plus claires.

Quelques compléments

Supprimer la variable Python avec `del` versus `.unpersist()`.

- Supprimer la variable Python ne supprime que la référence dans tous les cas. Utiliser `.unpersist()`.

Quelques compléments

Quels types de données peuvent être diffusés avec Spark ?

- Types de données Python simples (int, float, str, bool, list, tuple, dict) ainsi que tout objet sérialisable en Python : DataFrames Pandas, tableaux Numpy, RDDs Spark, objets Python personnalisés sérialisables, modèles de Machine Learning (scikit-learn, TensorFlow/Keras, XGBoost...), les objets qui implémentent `__getstate__` et `__setstate__` pour une sérialisation personnalisée.
- Les restrictions sont donc plutôt liées à la taille (l'objet diffusé doit être suffisamment petit pour tenir dans la mémoire de chaque nœud du cluster) et la mutabilité (la diffusion est destinée aux objets immutables ; bien qu'il soit techniquement possible de diffuser des objets mutables ce n'est généralement pas recommandé, car les modifications apportées à l'objet dans un nœud pourraient entraîner des incohérences. Cela veut dire qu'on va accepter de passer des listes et des dictionnaires lorsqu'on est confiant quant au fait qu'ils ne sont pas modifiés. Il est possible de convertir une liste (mutable) en RDD (non mutable).

Quelques compléments

Bien partitionner le Parquet en fonction des traitements aval.

- Optimisation de la lecture.
- Améliorer la localisation des données et réduire la communication entre les nœuds du cluster.

Quelques compléments

L'écriture dans des tables de bases de données est-elle distribuée ?

- Spark écrit en parallèle, mais chaque partition ouvre une connexion et chaque partition fait des INSERT. L'écriture est distribuée côté Spark mais la base peut devenir le goulot d'étranglement.
- PostgreSQL, MySQL : mononode. Donc : contention, verrous, voire saturation CPU / I/O.
- On revient sur notre remarque, pas si Big Data que ça par rapport aux formats fait pour gros volumes Big Data (Parquet / Delta) et donc souvent beaucoup plus lent.
- Mais des technos de bases distribuées : Snowflake, BigQuery, Redshift. Ingestion parallèle réelle optimisée pour Spark.

Quelques compléments

Job Spark sur Databricks Serverless

- <https://docs.databricks.com/aws/en/compute/serverless/limitations>

UI and logging

- The Spark UI is not available. Instead, use the query profile to view information about your Spark queries. See [Query profile](#).
- <https://www.google.com/search?q=databricks+spark+ui>
- Regarder en local (port 4040)

Quelques compléments

Quand faire de la lazy initialisation de modèle ?

- Lorsqu'il est difficile de prévoir la nécessité du modèle.

Exemple : pipeline de traitement de données où on applique différents modèles de ML à des sous-ensembles de données en fonction des résultats intermédiaires.

Remember: l'« initialisation » de dataframe est toujours lazy.

Quelques compléments

Si je fais un broadcast, Catalyst et/ou AQE peuvent-ils revenir dessus ?

- Pour le coup, la réponse n'est pas 100% claire et il faudrait faire quelques expérimentations supplémentaires.

AQE PEUT revenir sur un tel choix. A noter que le coût du broadcast est payé dès le départ, car le DataFrame est envoyé en mémoire sur tous les nœuds de travail. AQE n'interviendra qu'avant l'exécution, lorsqu'il analysera les caractéristiques réelles des données (par exemple, la taille des DataFrames et leur distribution).

Ce qui n'est pas très clair, c'est le poids du caractère « déclaration explicite » du broadcast dans le choix de Catalyst et AQE.

Une bonne façon d'intervenir sur le choix que font ces composants est de régler les seuils de taille pour le broadcasting, respectivement `spark.sql.autoBroadcastJoinThreshold` et `spark.sql.adaptive.shuffleTargetPostShuffleInputSize`.

Quelques compléments

Quelle est la différence entre YARN et Kubernetes ?

C'est vrai qu'ils partagent certains objectifs communs : gérer les ressources et orchestrer les tâches sur un cluster.

YARN est principalement utilisé dans l'écosystème Hadoop pour orchestrer les tâches de calcul de données massives (comme avec Spark, MapReduce, etc.). Il est particulièrement optimisé pour les applications de Big Data et les workflows de traitement de données distribuées.

Kubernetes est une plateforme d'orchestration de conteneurs (type Docker). Il est plus généraliste et peut être utilisé pour orchestrer n'importe quelle application conteneurisée, y compris des applications cloud-native, des microservices.

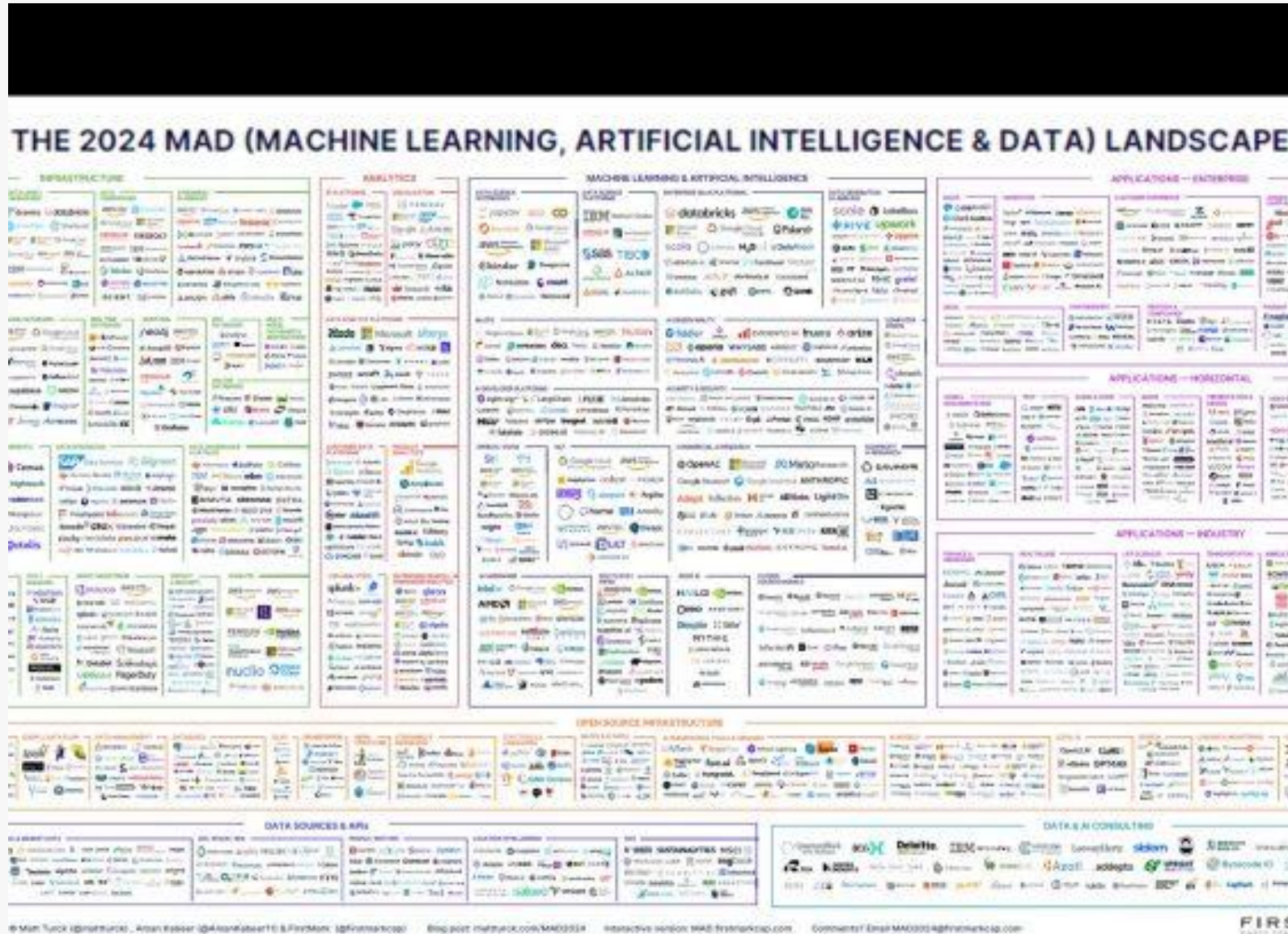
Kubernetes supporte l'exécution de Spark sur des conteneurs, mais si on utilise une stack Hadoop traditionnelle (MapReduce...), la transition vers Kubernetes va être plus complexe.

Plutôt que Kubernetes, l'alternative la plus connue à YARN est Mesos.

Quelques compléments

Big Data landscape

Quelques compléments



04

Analyser en temps réel avec Spark Streaming

Streaming

Batch processing et stream processing

Batch processing (traitement par lots)	Stream processing (traitement de flux)
Consiste à traiter un grand volume de données en une seule fois, par lots.	Consiste à traiter les données au fur et à mesure de leur arrivée, de manière continue.
Les données sont collectées, stockées et traitées en blocs à intervalles réguliers ou selon un calendrier défini.	Les données sont traitées au moment où elles sont produites, permettant une réaction en temps réel aux événements.
Convient aux tâches qui peuvent tolérer un certain délai de latence dans le traitement des données, telles que les analyses de données, les rapports générés à intervalles réguliers, etc.	Utilisé dans les cas où une analyse en temps réel des données est nécessaire, comme la surveillance des systèmes, la détection des fraudes, le traitement des événements en temps réel, etc.
 Apache Hadoop avec MapReduce Apache Spark avec les opérations sur les RDD Les travaux cron sur Unix/Linux Etc.	 Apache Kafka Streams Apache Flink Apache Storm Spark Structured Streaming Apache Samza Etc.

Streaming

Stream processing : applications

- Surveillance des transactions en temps réel pour la détection de fraude.
- Analyse en temps réel du trafic réseau pour la détection des pannes et l'optimisation des performances.
- Surveillance des flux de données pour la détection des attaques et des tentatives d'intrusion.
- Surveillance des signes vitaux en temps réel pour la télémédecine et les soins à distance.
- Surveillance des capteurs IoT pour la gestion des équipements industriels et la maintenance prédictive.
- Diffusion en continu de contenu multimédia en direct.
- Suivi des médias sociaux pour la surveillance de la réputation de la marque et l'analyse des tendances.
- ETL incrémental.
- Apprentissage automatique en ligne.

Streaming

Stream processing : challenges

- Latence et temps réel
- Gestion du volume des données
- Robustesse et fiabilité
- Traitement des données out-of-order
- Gestion des retardataires
- Garantie de livraison exactly-once
- Maintien d'une grande quantité d'états
- Gestion des déséquilibres de charge
- Joindre des données externes
- Mise à jour des cibles en sortie
- Écriture transactionnelle des données
- Mise à jour de la logique métier en temps réel
- Débogage et monitoring
- Synchronisation et cohérence
- Evolutivité et performance



Streaming

Batch processing et stream processing

Deux composants principaux :

➡ Système de messagerie

Sert de canal de communication entre les producteurs de données (sources) et les consommateurs de données (applications ou services).

Généralement équipé d'une capacité de stockage tampon (buffer) pour gérer les flux de données entrants et sortants.

Assure la transmission fiable et efficace des messages à travers le système de streaming.

➡ Moteur de gestion de flux

Stream Processing Engine

Cœur du système de streaming.

Responsable du traitement, de l'analyse et de la manipulation des flux de données en temps réel.

Exécute des opérations telles que la transformation, l'agrégation, le filtrage, la jointure et d'autres opérations analytiques sur les flux de données entrants.

Prend en charge la logique métier spécifique et les algorithmes de traitement définis par l'utilisateur pour répondre aux besoins applicatifs.

Les deux composants du système de streaming sont concernés par une **sémantique de livraison des messages**.

Streaming

Sémantiques de livraison

Livraison au plus une fois

At-most once

Un message est livré au consommateur au maximum une fois.

La plus simple à mettre en œuvre : privilégie la faible latence et le débit élevé.

Les messages peuvent être perdus, mais ils ne sont jamais renvoyés.

Livraison au moins une fois

At-least once

Un message est garanti d'être livré au consommateur au moins une fois, garantissant qu'aucun message n'est perdu.

Cela peut entraîner une livraison de messages en double (les messages peuvent être renvoyés plusieurs fois jusqu'à ce qu'ils soient accusés de réception par le consommateur).

Suppose généralement que les opérations de traitement soient idempotentes.

Livraison exactement une fois

Exactly once

Garantit que chaque message est livré au consommateur exactement une fois et une seule, sans perte ni duplication.

Plus complexe à atteindre, et implique généralement des mécanismes de coordination entre le producteur et le consommateur pour suivre la livraison des messages et gérer les échecs de manière efficace.

Streaming

Garantie de cohérence

Strong consistency

Chaque lecture après une écriture renvoie toujours la valeur la plus récente. Tous les nœuds du système voient les mêmes données au même moment, ce qui garantit une forte cohérence.

Cela peut entraîner des performances plus faibles et une plus grande latence, car chaque opération nécessite une coordination étroite entre les nœuds pour maintenir la cohérence.

Eventual consistency

Permet des écarts temporaires dans la cohérence des données.

Après une écriture, il peut y avoir un délai avant que toutes les lectures ne reflètent cette écriture.

Les systèmes basés sur la consistance éventuelle tolèrent les incohérences temporaires en échange de performances plus élevées et d'une latence réduite.

Les conflits potentiels sont résolus à terme à mesure que les mises à jour se propagent à travers le système.

Weak consistency

Entre la consistance forte et la consistance éventuelle.

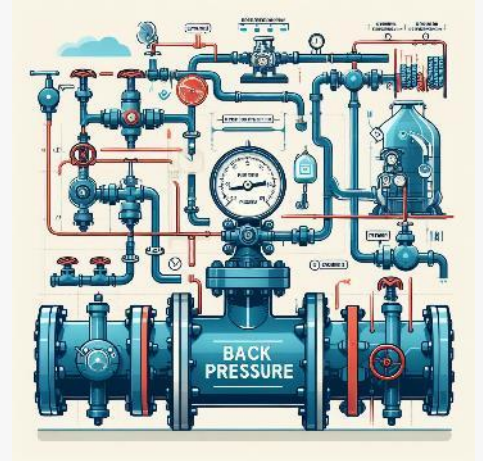
Garantit que les données sont cohérentes à l'intérieur d'une limite de temps spécifiée, mais ne garantit pas que toutes les parties du système voient les mêmes données au même moment.

Permet une certaine flexibilité dans la synchronisation des données, ce qui peut conduire à des performances optimisées tout en maintenant une certaine cohérence.

Streaming

Backpressure

- Backpressure (ou pression de retour) : capacité d'un système de streaming à gérer et de réguler le flux de données entrantes lorsqu'il est confronté à un débit de données supérieur à sa capacité de traitement.
- Lorsque le système est incapable de traiter les données entrantes à la même vitesse à laquelle elles sont reçues, cela peut entraîner des problèmes tels que des goulots d'étranglement, des retards dans le traitement et éventuellement des pannes.
- Le backpressure permet au système de contrôler le flux de données entrantes en ajustant la vitesse de traitement en fonction de sa capacité actuelle. Lorsque le système est surchargé, il peut ralentir ou même suspendre temporairement le traitement des données jusqu'à ce qu'il puisse rattraper son retard.
- Crucial pour assurer la stabilité et la fiabilité des applications de streaming, en particulier dans des environnements à forte charge de travail.



Streaming

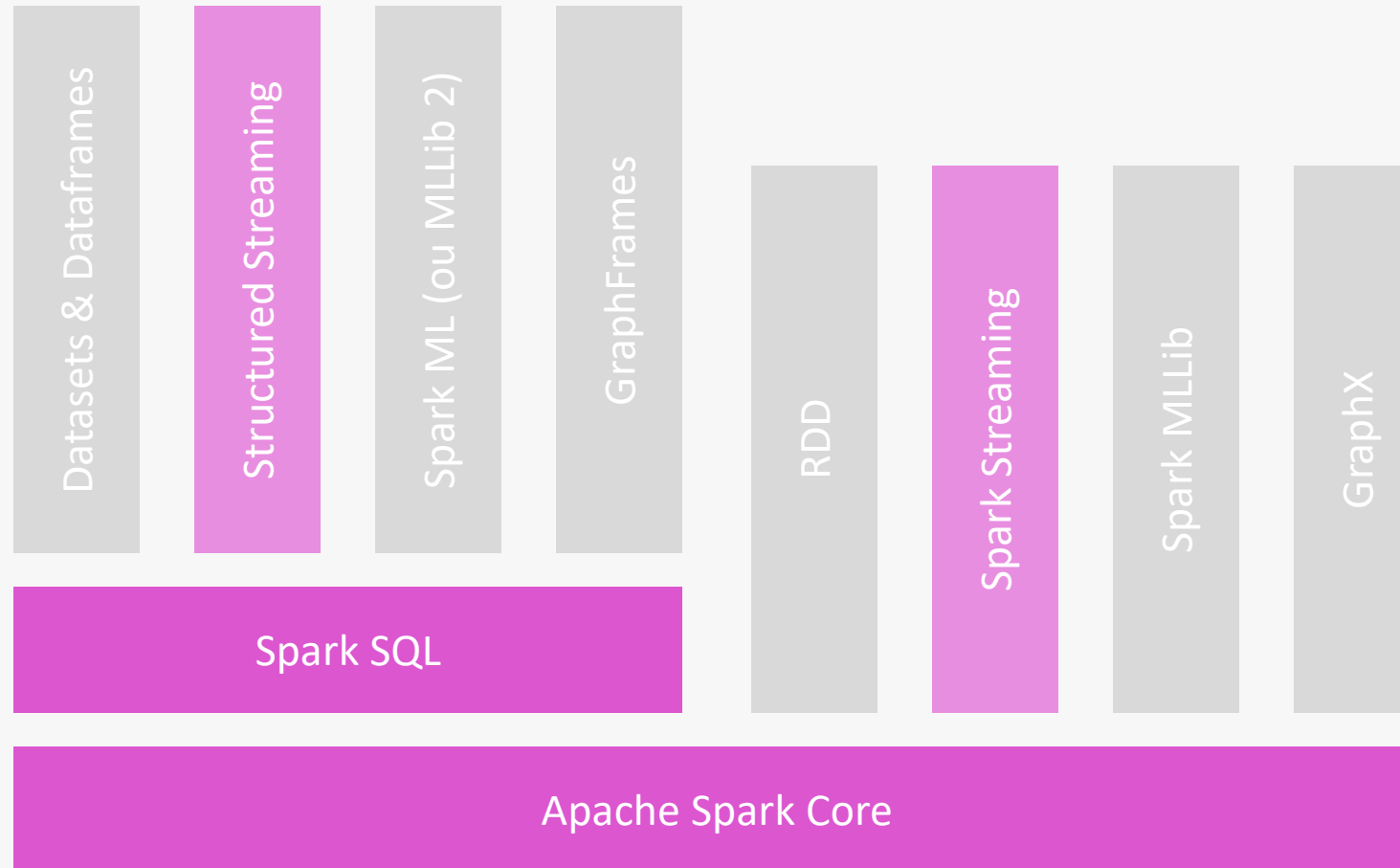
Backpressure

- Lorsque le débit de données entrantes dépasse la capacité de traitement d'un système de streaming, celui-ci utilise des **mémoires tampons** pour temporairement stocker les données en attente de traitement.
- Kafka est souvent utilisé pour gérer la transmission de flux de données entre les producteurs et les consommateurs dans les pipelines de streaming. Il offre des fonctionnalités de mise en mémoire tampon et de réplication des données, ce qui contribue à atténuer les effets de la backpressure en permettant aux consommateurs de traiter les données à leur propre rythme, tout en régulant le flux de données entrantes.



Analyser en temps réel avec Spark Streaming et Kafka

Les composants de streaming



Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming



Source : <https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming

Motivation de Spark Streaming (depuis Spark 0.7 en 2012) :

- Spark est d'abord un système orienté batch avec une extension pour la gestion de flux, utilisant le concept de micro-batches (au dessus de son moteur batch).
- Chaque micro-batch est un job Spark représenté comme un RDD, ce qui constitue les Discretized Streams (DStreams).
- Bien que livré avec des latences plus élevées que l'approche un enregistrement à la fois, Spark offre une meilleure récupération des pannes, notamment avec un traitement exactly-once.
- Spark vise à résoudre les problèmes de tolérance aux pannes et de retardataires.
- Pour la tolérance aux pannes, en général, deux solutions sont envisagées : la réplication et l'upstream backup.
- Cependant, aucune de ces approches ne traite efficacement les retardataires.

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming



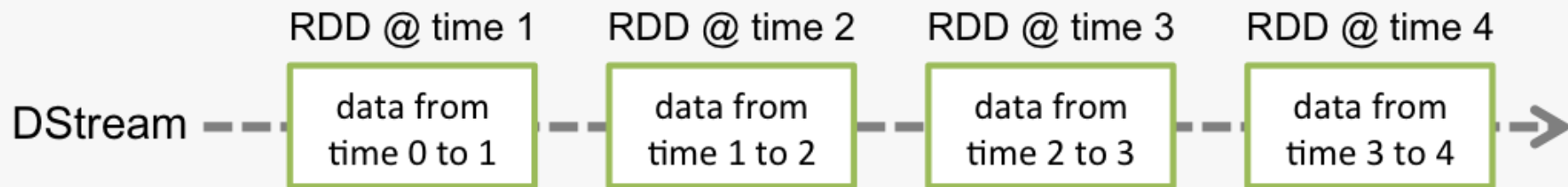
Source : <https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : DStreams

Dstream = flux discrétisé, abstraction de base fournie par Spark Streaming

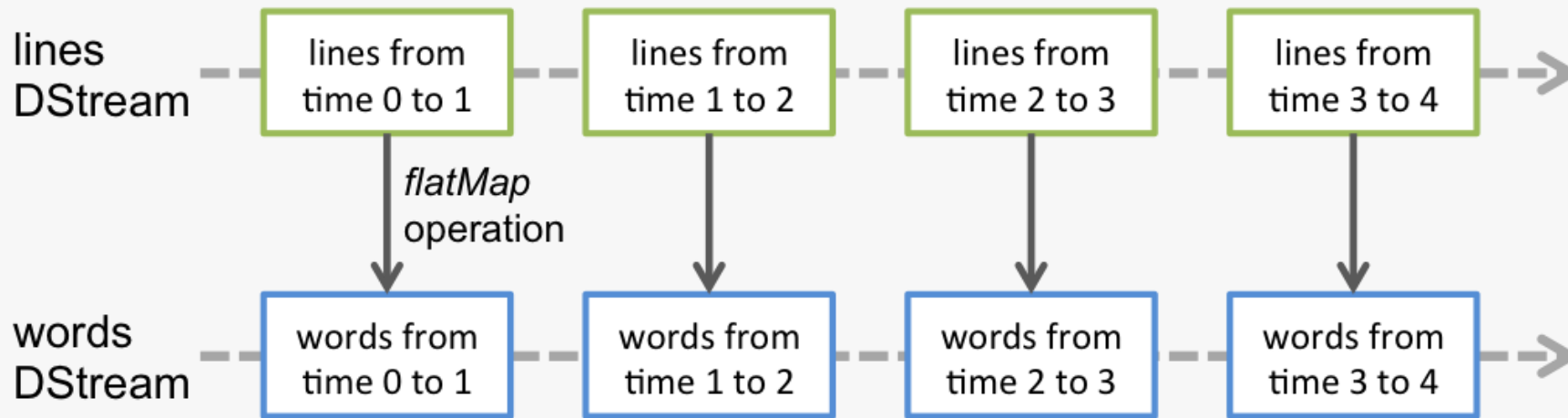
- Il représente un flux continu de données, soit le flux de données d'entrée reçu de la source, soit le flux de données traitées généré par la transformation du flux d'entrée.
- En interne, un DStream est représenté par une série continue de RDD.
- Chaque RDD dans un DStream contient des données d'un certain intervalle :



Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : DStreams

Toute opération appliquée à un DStream se traduit par des opérations sur les RDD sous-jacents. Par exemple, dans l'exemple précédent de conversion d'un flux de lignes en mots, l'opération `flatMap` est appliquée à chaque RDD dans le DStream des lignes pour générer les RDD du DStream de mots.



Ces transformations sous-jacentes de RDD sont calculées par le moteur Spark. Les opérations DStream masquent la plupart de ces détails et nous fournissent une API de plus haut niveau pour plus de commodité.

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : exemple du Word Count

Tout d'abord, on importe `StreamingContext`, qui est le point d'entrée principal pour toutes les fonctionnalités de streaming. Ensuite, on crée un `StreamingContext` local avec deux threads d'exécution et un intervalle de lot de 1 seconde.

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

# Create a Local StreamingContext with two working thread and batch interval of 1 second
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)
```

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : exemple du Word Count

À l'aide de ce contexte, nous pouvons créer un DStream qui représente des données en streaming à partir d'une source TCP, spécifiée par un nom d'hôte (par exemple localhost) et un port (par exemple 9999).

```
# Create a DStream that will connect to hostname:port, like localhost:9999  
lines = ssc.socketTextStream("localhost", 9999)
```

Ce DStream représente le flux de données qui sera reçu du serveur de données. Chaque enregistrement dans ce DStream est une ligne de texte.

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : exemple du Word Count

On veut diviser les lignes en mots (séparer selon les caractères espaces).

```
# Split each line into words  
words = lines.flatMap(lambda line: line.split(" "))
```

`flatMap` est une opération DStream one-to-many qui crée un nouveau DStream en générant plusieurs nouveaux enregistrements à partir de chaque enregistrement dans le DStream source. Dans ce cas, chaque ligne sera divisée en plusieurs mots et le flux de mots est représenté sous forme de DStream de mots.

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : exemple du Word Count

Ensuite, on compte ces mots.

```
# Count each word in each batch
pairs = words.map(lambda word: (word, 1))
wordCounts = pairs.reduceByKey(lambda x, y: x + y)

# Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.pprint()
```

Le DStream des mots est ensuite transformé (transformation one-to-one) en un DStream de paires (mot, 1), qui est ensuite réduit pour obtenir la fréquence des mots dans chaque lot de données. wordCounts.pprint() affiche des décomptes générés chaque seconde.

Analyser en temps réel avec Spark Streaming et Kafka

Spark Streaming : exemple du Word Count

Lorsque ces lignes sont exécutées, Spark Streaming configure uniquement le calcul qu'il effectuera lorsqu'il sera démarré, et aucun traitement réel n'a encore commencé. Pour démarrer le traitement après que toutes les transformations ont été configurées, il faut exécuter :

```
ssc.start() # Start the computation  
ssc.awaitTermination() # Wait for the computation to terminate
```

Analyser en temps réel avec Spark Streaming et Kafka

Structured Streaming

Structured Streaming = moteur de traitement de flux évolutif et tolérant aux pannes construit sur le moteur Spark SQL

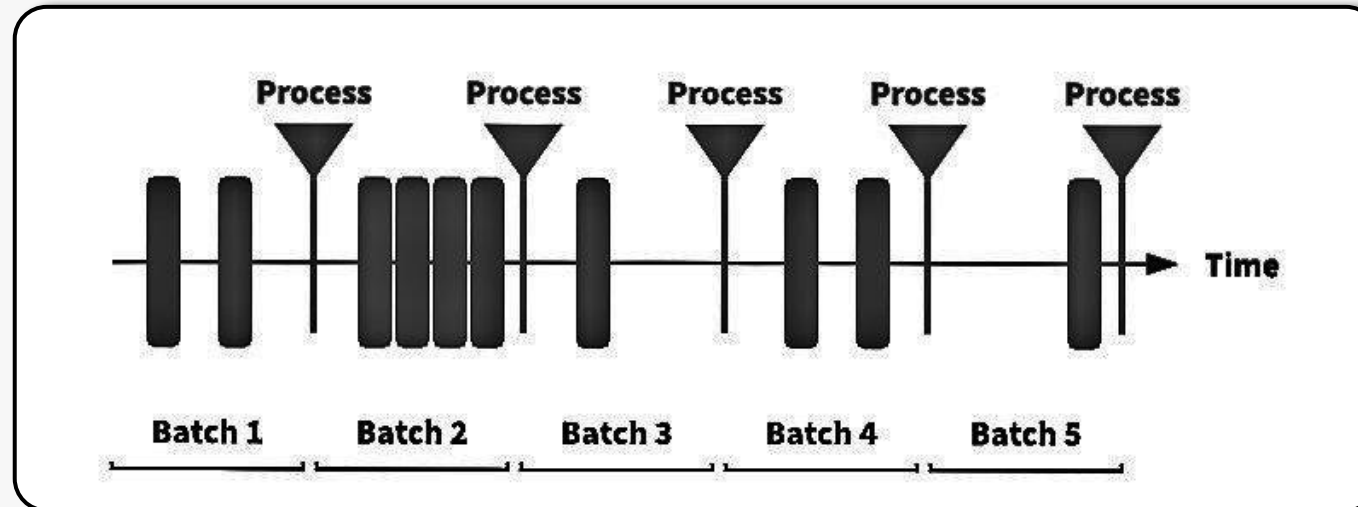
- Les calculs en streaming s'expriment de la même manière que ceux pour un lot de données statiques.
- Le moteur Spark SQL se charge de les exécuter de manière incrémentielle et continue et de mettre à jour le résultat final à mesure que les données de streaming continuent d'arriver.
- L'API s'utilise en Scala, Java, Python ou R pour exprimer des agrégations de streaming, des fenêtres de temps événementielles, des jointures de flux à lot...
- Le calcul est exécuté sur le même moteur Spark SQL optimisé.
- Le système garantit la tolérance aux pannes de bout en bout exactement une fois grâce à des checkpoints et à des logs de transactions.

Structured Streaming fournit ce traitement de flux sans que l'utilisateur ait à raisonner sur le streaming.

Analyser en temps réel avec Spark Streaming et Kafka

Structured Streaming

En interne, les requêtes de Structured Streaming sont traitées (par défaut) à l'aide d'un moteur de traitement par micro-lots, qui traite les flux de données comme une série de petits travaux par lot, ce qui permet d'obtenir des latences de bout en bout de l'ordre de la centaine de millisecondes et des garanties de tolérance aux pannes exactly-once.

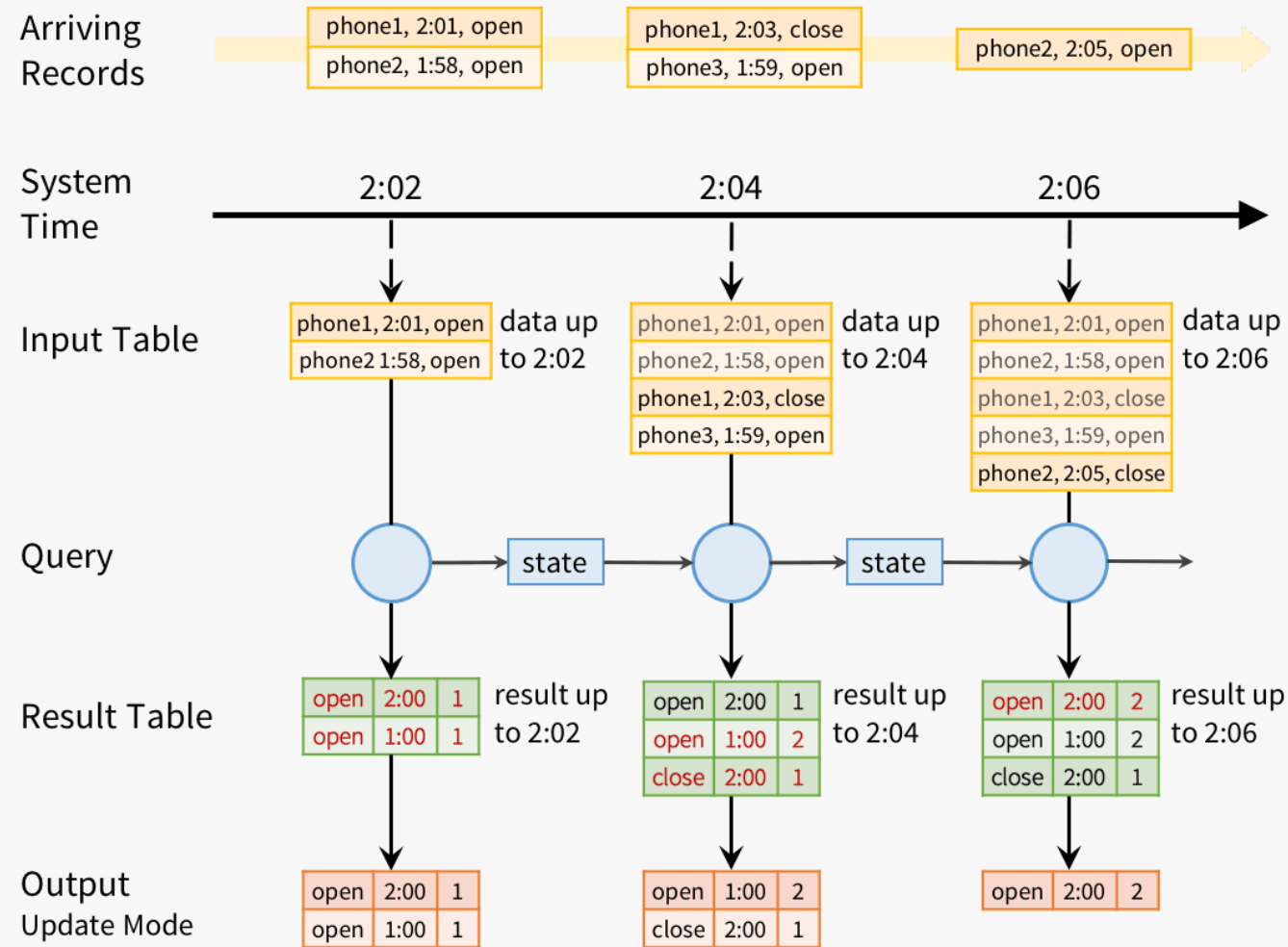


Source : https://www.researchgate.net/figure/Data-Stream-Processing-Flow-Micro-batching-Micro-batching-based-architecture-takes_fig2_336141201

Spark 2.3 a introduit un nouveau mode : Continuous Processing.

Analyser en temps réel avec Spark Streaming et Kafka

Structured Streaming – schéma récapitulatif



Source : <https://www.databricks.com/blog/2016/07/28/structured-streaming-in-apache-spark.html>

Analyser en temps réel avec Spark Streaming et Kafka

TP



Structured Streaming

Question 16

Streaming

Quel est désormais le principal composant utilisé pour le streaming avec Spark ?

- ☐ RDD
- ☐ MLlib
- ☐ Structured Streaming
- ☐ MapReduce
- ☐ Spark Streaming

Question 17

Streaming

Quel est le principal inconvénient de l'approche de micro-batch utilisée par Spark Streaming ?

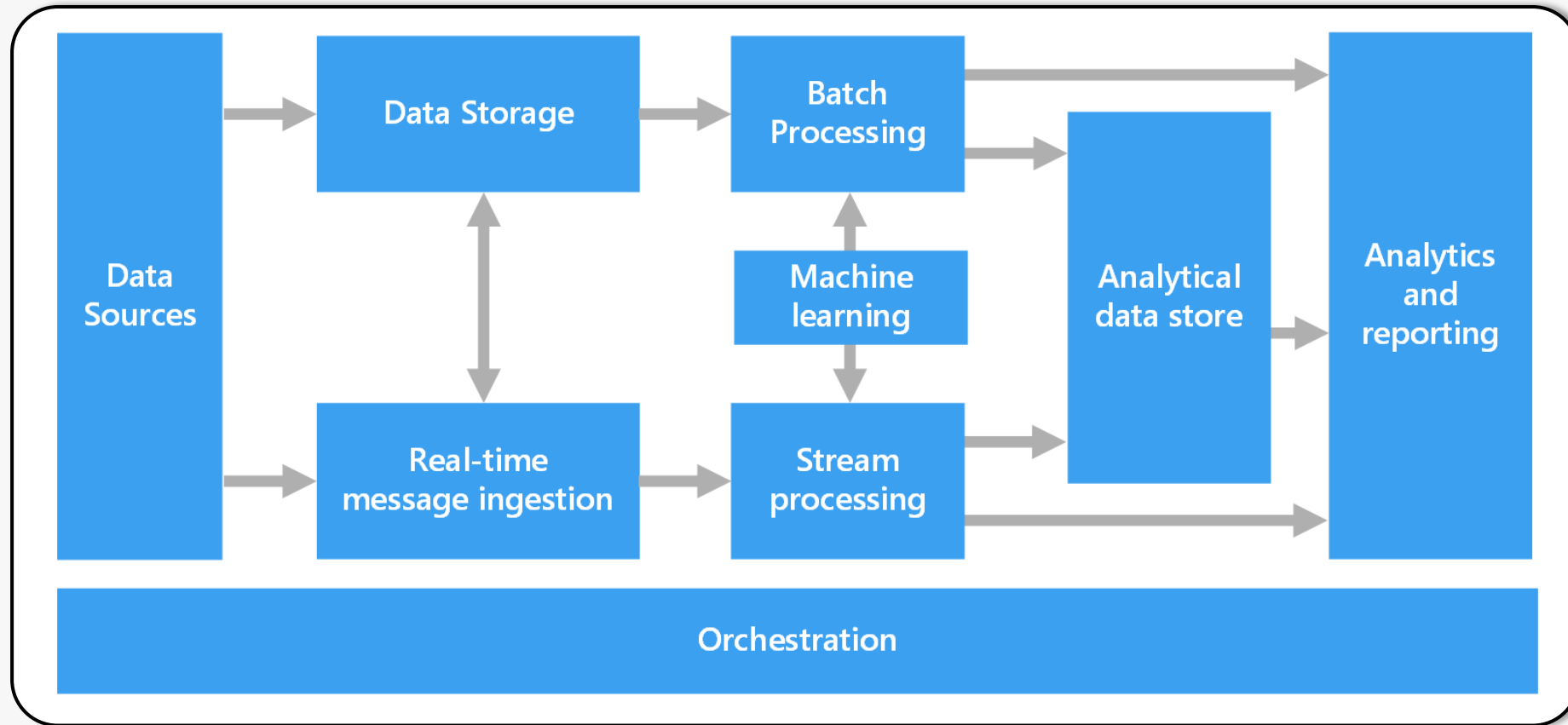
- ☐ Latence élevée
- ☐ Faible tolérance aux pannes
- ☐ Complexité de mise en œuvre
- ☐ Limitations sur la scalabilité

04 bis

**Quelques compléments
sur les architectures
Big Data avec des
données en streaming**

Application

Composants d'une architecture Big Data

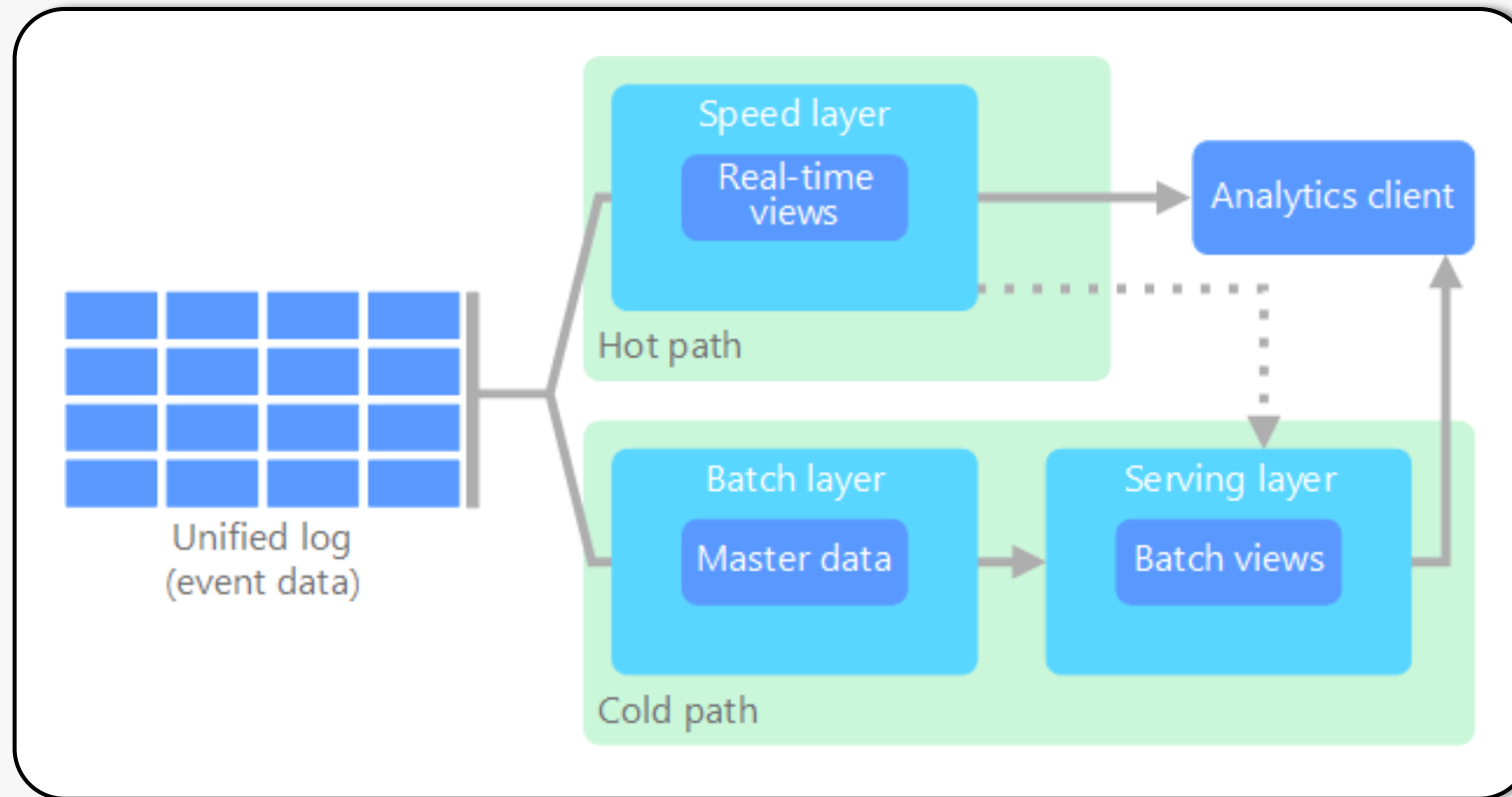


Source : <https://learn.microsoft.com/fr-fr/azure/architecture/databases/guide/big-data-architectures>

Application

Architecture lambda

Architecture lambda = deux chemins d'accès aux flux de données, 1 à chaud, 1 à froid



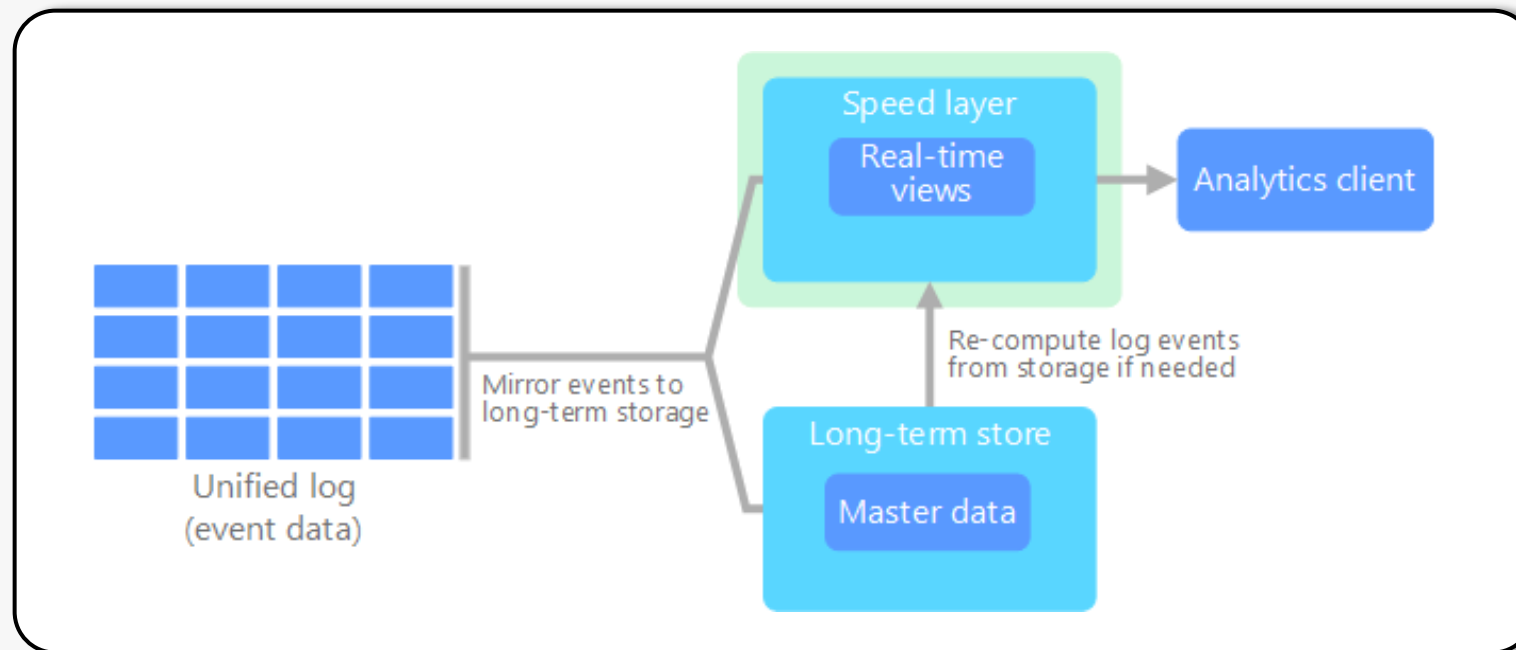
Source : <https://learn.microsoft.com/fr-fr/azure/architecture/databases/guide/big-data-architectures>

Application

Architecture kappa

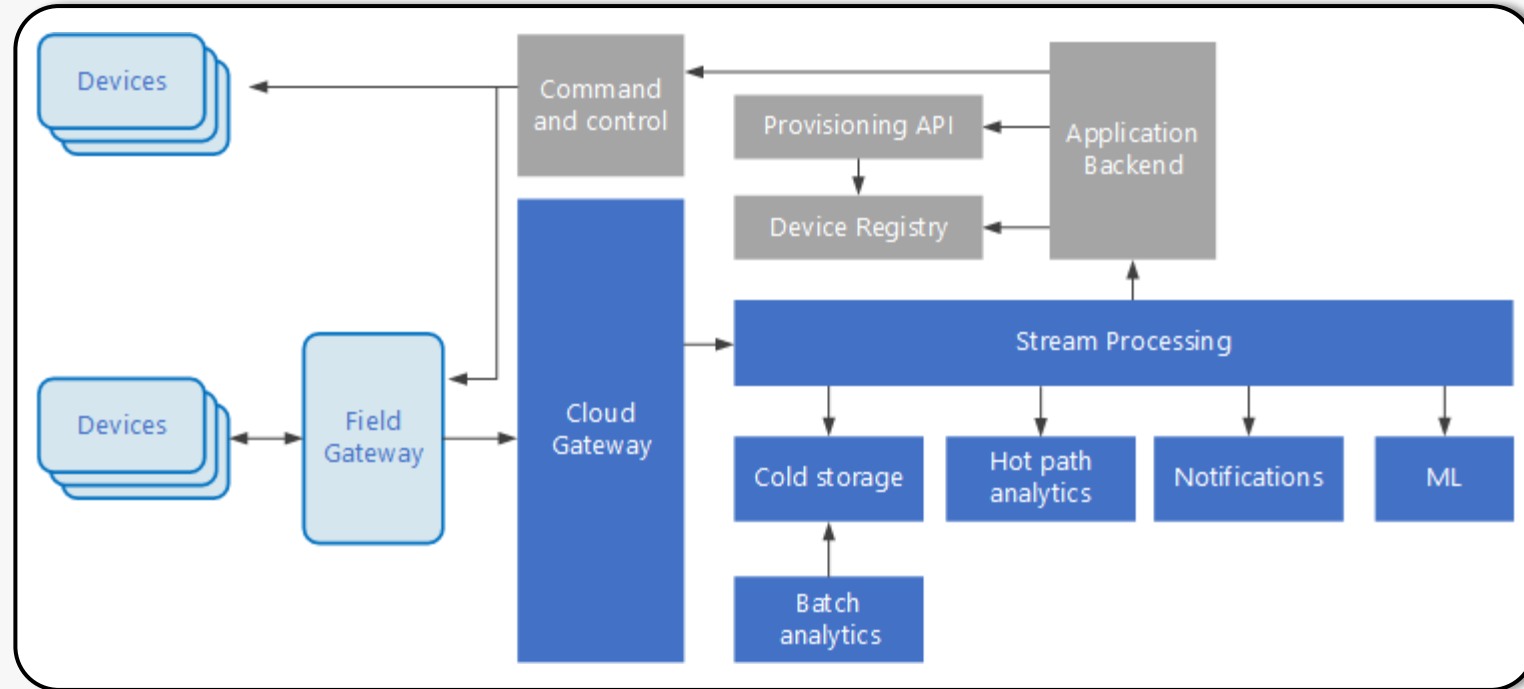
Architecture kappa = alternative à l'architecture lambda. Vise les mêmes objectifs de base que l'architecture lambda, mais :

Toutes les données transitent via un chemin unique en utilisant un système de traitement de flux.



Application

Architecture kappa – déclinaison IoT



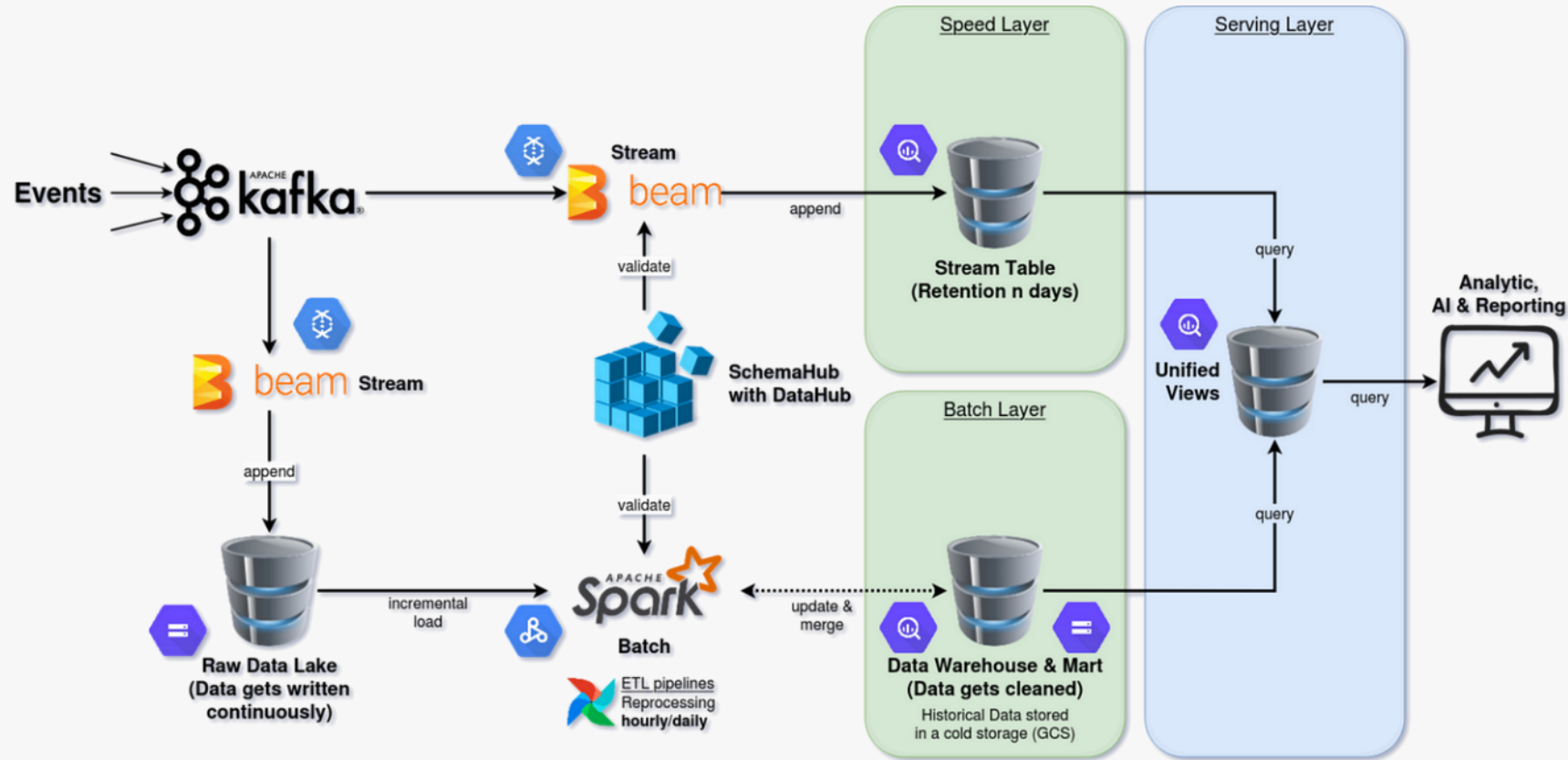
Quelques éléments supplémentaires sur les archi Big Data

Un premier lien

<https://www.xenonstack.com/blog/big-data-architecture>

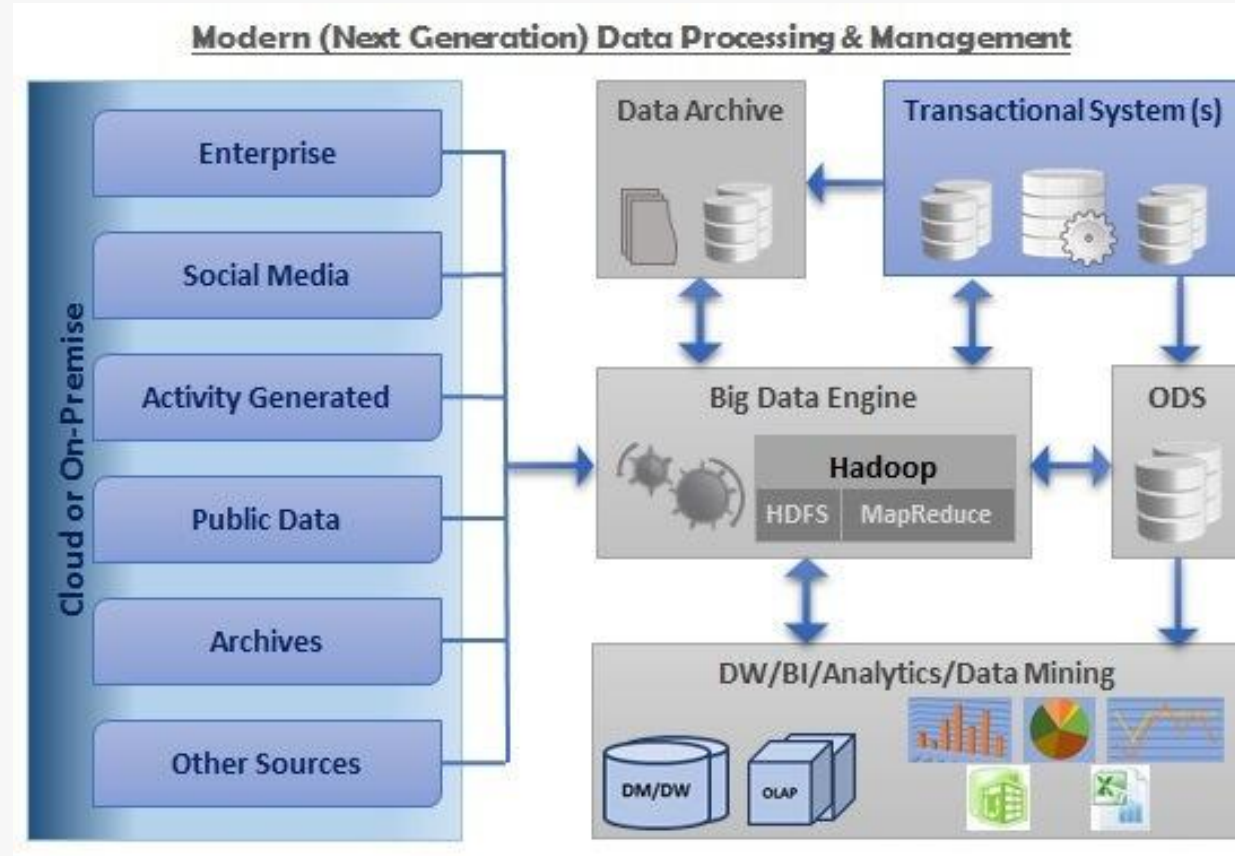
Quelques éléments supplémentaires sur les archi Big Data

Exemples de schéma d'architecture



Quelques éléments supplémentaires sur les archi Big Data

Exemples de schéma d'architecture



Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Stockage distribué (Data Lake & Data Warehouse)

Objectif : Stocker de grands volumes de données de manière scalable et accessible.

Outils classiques :

- HDFS (Hadoop Distributed File System)
- Amazon S3 / Google Cloud Storage / Azure Blob Storage (cloud)
- Ceph / GlusterFS / Amazon S3 (alternatives à HDFS on prem)
- Apache Iceberg / Delta Lake / Apache Hudi (formats avancés pour gérer les mutations et versions dans un Data Lake)
- Google BigQuery / Snowflake / Redshift (Data Warehouses modernes).

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Moteurs de traitement et de calcul distribué

Objectif : Exécuter des calculs massivement parallèles sur des clusters.

Outils classiques :

- Apache Spark (moteur de traitement en mémoire, standard de facto)
- Apache Flink (traitement en temps réel et batch avec latence très faible)
- Apache Hadoop (MapReduce) (ancien modèle batch, moins utilisé aujourd'hui)
- Dask / Ray (alternatives Python natives pour le traitement distribué)

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Orchestration et gestion des workflows

Objectif : Automatiser l'exécution de pipelines de données.

Outils classiques :

- Apache Airflow (standard open-source pour gérer des DAGs de jobs)
- Dagster (alternative moderne à Airflow, orientée DataOps)
- Luigi (ancien orchestrateur développé par Spotify)
- Azure Data Factory / Google Dataflow (cloud)

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Gestion des ressources et des clusters

Objectif : Allouer efficacement les ressources aux jobs Big Data.

Outils classiques :

- YARN (Hadoop Yet Another Resource Negotiator) (gestionnaire de ressources Hadoop)
- Apache Mesos (moins populaire aujourd'hui, alternative à YARN)
- Kubernetes (conteneurs, microservices, mais malgré tout de plus en plus utilisé pour exécuter Spark & Flink conteneurisés)
- Et les outils cloud dédiés comme EMR

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Traitement des flux de données en streaming

Objectif : Analyser les données en temps réel.

Outils classiques :

- **Systèmes de messagerie** (qui font parfois plus) :
 - Apache Kafka (système de messagerie ultra-scalable)
 - Apache Pulsar (concurrent de Kafka, avec un peu plus de flexibilité)
 - AWS Kinesis, Azure Events Hub... (cloud)
- **Stream Processing Engines** :
 - Apache Flink (traitement de flux avec latence ultra-basse)
 - Spark Streaming / Structured Streaming (traitement de flux avec Spark)

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Ingestion et ETL (Extract, Transform, Load)

Objectif : Récupérer, transformer et charger des données depuis différentes sources.

Outils classiques :

- Apache NiFi (solution d'ingestion graphique)
- Fivetran / Stitch (outils SaaS d'ingestion automatisée)
- dbt (Data Build Tool) (transformation SQL « moderne »)
- Talend / Informatica – ETL traditionnels

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Bases de données distribuées (OLAP & OLTP)

Objectif : Stocker et interroger des données massives.

Outils classiques :

- Apache Cassandra (base NoSQL pour écritures distribuées)
- Apache HBase (base NoSQL sur HDFS, utilisée avec Hadoop)
- Google Bigtable (alternative cloud).
- ClickHouse / Apache Druid (bases OLAP rapides pour analytics)

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Indexation et recherche full-text

Objectif : Rendre les données facilement interrogeables.

Outils classiques :

- Elasticsearch (standard pour la recherche full-text distribuée)
- Apache Solr (alternative open-source historique)
- OpenSearch (fork open-source d'Elasticsearch)

Quelques éléments supplémentaires sur les archi Big Data

Les grandes briques

Machine Learning et IA distribués

Objectif : Entraîner des modèles de ML/IA sur des clusters.

Outils classiques :

- MLlib (Spark ML) (ML intégré à Spark)
- TensorFlow on Spark / Horovod (entraînement de modèles IA sur cluster)
- Ray / Dask-ML (alternatives distribuées pour le ML en Python)
- Feast (Feature Store pour la gestion des features ML)

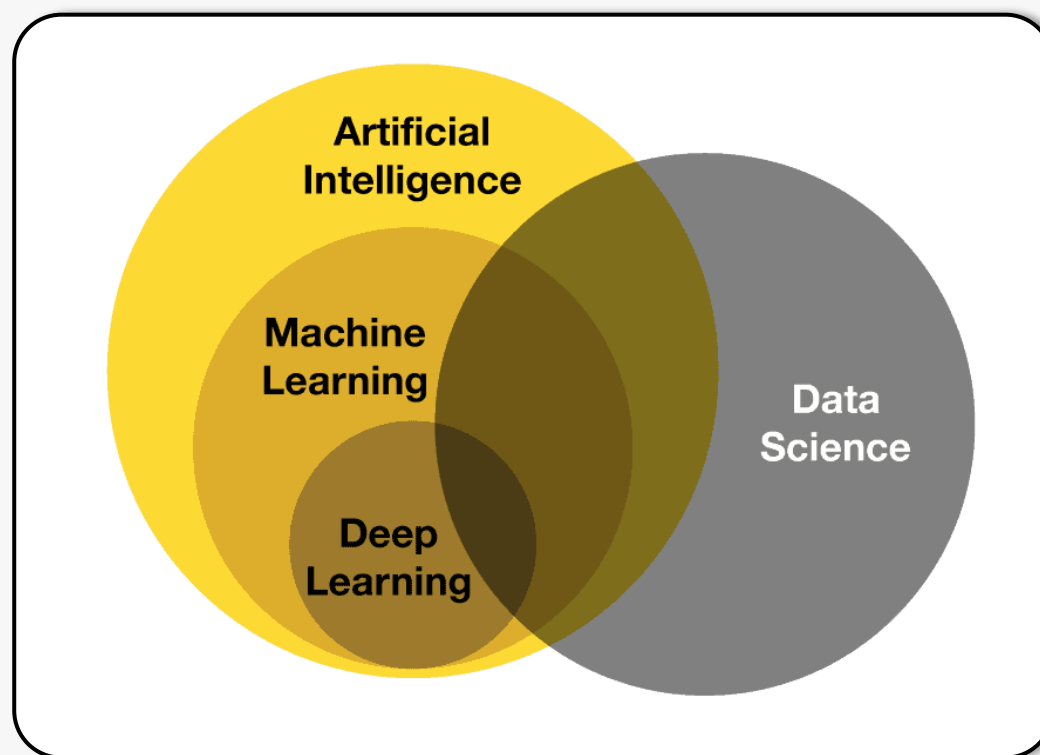
05

Machine Learning avec Spark

Petit aparté sur les concepts d'IA et de ML

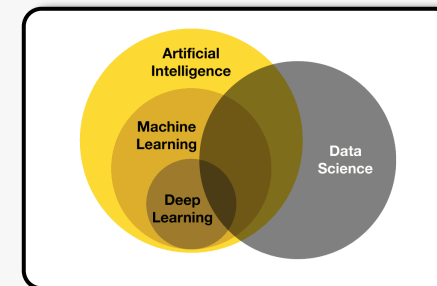
Notions élémentaires : IA, ML, DL, DS

L'intelligence artificielle (IA) et l'apprentissage automatique (machine learning) représentent deux domaines interconnectés qui ont évolué de manière significative au fil de l'histoire, transformant la façon dont nous abordons la résolution de problèmes complexes et la prise de décisions.



Petit aparté sur les concepts d'IA et de ML

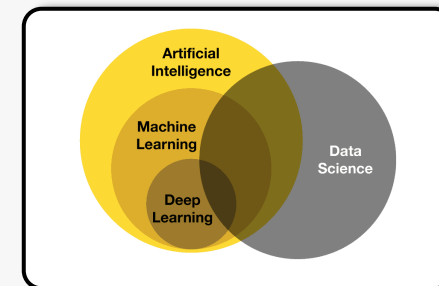
Notions élémentaires : IA, ML, DL, DS



- **Intelligence artificielle** : Ensemble des théories et des techniques développant des programmes informatiques complexes capables de **simuler certains traits de l'intelligence humaine** (raisonnement, apprentissage...). Elles permettent donc à une machine de réaliser des tâches et de résoudre des problèmes normalement réservés aux humains
- **Machine Learning** : Discipline regroupant connaissances et méthodes permettant de donner aux systèmes informatiques la capacité à apprendre une tâche **sans être explicitement programmé** à cette fin.
- **Deep Learning** : Ensemble de méthodes de machine learning tentant de modéliser avec un **haut niveau d'abstraction** des données grâce à des architectures articulées de différentes transformations non linéaires.
- **Data science** : Discipline regroupant connaissances et méthodes permettant de **traiter des représentations** de faits stockés **sous forme numérique**.

Petit aparté sur les concepts d'IA et de ML

Les débuts



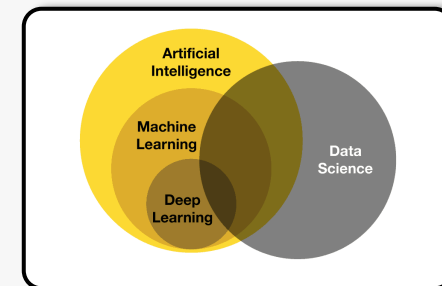
- Les premiers concepts d'IA remontent à l'Antiquité, avec des **mythes** et des histoires mettant en scène des **créatures artificielles douées d'intelligence**.
- Cependant, le véritable développement de l'IA a commencé au **XXe siècle** avec les travaux d'Alan Turing et sa machine universelle, qui a jeté les bases du calcul et de la pensée mécanique. Au cours des **années 1950 et 1960**, le terme *intelligence artificielle* a été popularisé, et des chercheurs ont entrepris de créer des programmes informatiques capables de simuler des processus de réflexion humaine.

« Toute activité intellectuelle peut être décrite avec suffisamment de précision pour être simulée par une machine » *John McCarthy (1927 – 2011)*

- Les premières ambitions de l'IA ont été confrontées à des défis techniques et conceptuels majeurs, ce qui a conduit à une période de réduction des attentes, connue sous le nom d'**hiver de l'IA**. Pendant ce temps, l'apprentissage automatique a commencé à émerger comme un sous-domaine de l'IA. Les **années 1980 et 1990** ont vu la résurgence de l'intérêt pour l'IA grâce à l'amélioration des capacités de calcul et à de nouvelles approches, notamment les réseaux neuronaux et les méthodes statistiques.

Petit aparté sur les concepts d'IA et de ML

L'histoire récente



- Le **début du XXIe siècle** a été marqué par une **accélération sans précédent dans les domaines de l'IA et de l'apprentissage automatique**. Les avancées dans les **algorithmes**, l'augmentation de la **puissance de calcul** et la disponibilité de **grandes quantités de données** ont ouvert de nouvelles possibilités. Les réseaux neuronaux profonds, en particulier les réseaux de neurones convolutionnels et les réseaux de neurones récurrents, ont donné naissance au domaine du deep learning, qui a révolutionné la vision par ordinateur, la compréhension du langage naturel et bien plus encore.
- De nos jours, l'IA et l'apprentissage automatique sont omniprésents, touchant presque tous les aspects de nos vies, des **moteurs de recherche** et des **recommandations** de contenu aux **voitures autonomes** et à la **médecine personnalisée**. Les systèmes d'IA sont devenus de plus en plus sophistiqués, capable de battre les champions mondiaux aux **échecs** et au **jeu de go**, ainsi que de traiter des quantités massives de données pour en extraire des informations précieuses.

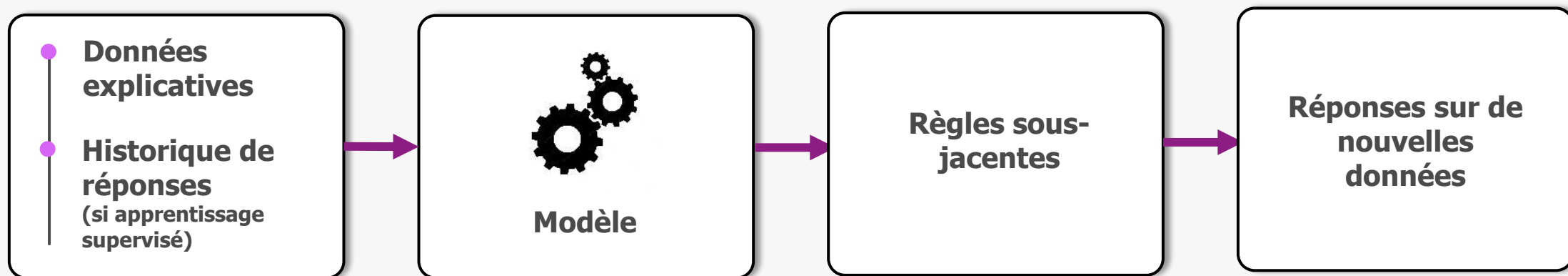
Petit aparté sur les concepts d'IA et de ML

Différence entre programmation classique et apprentissage automatique

Programmation classique :

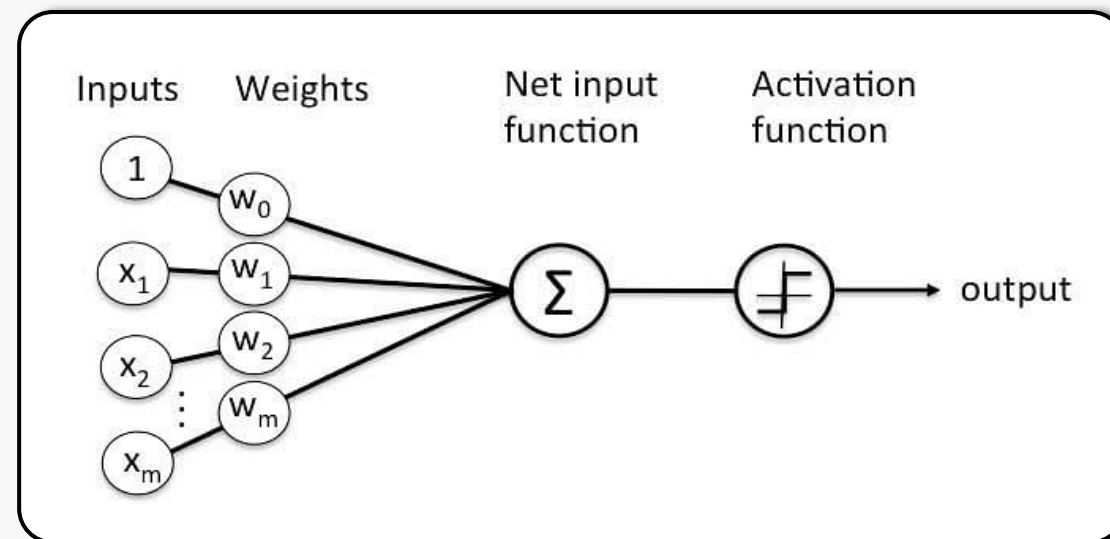
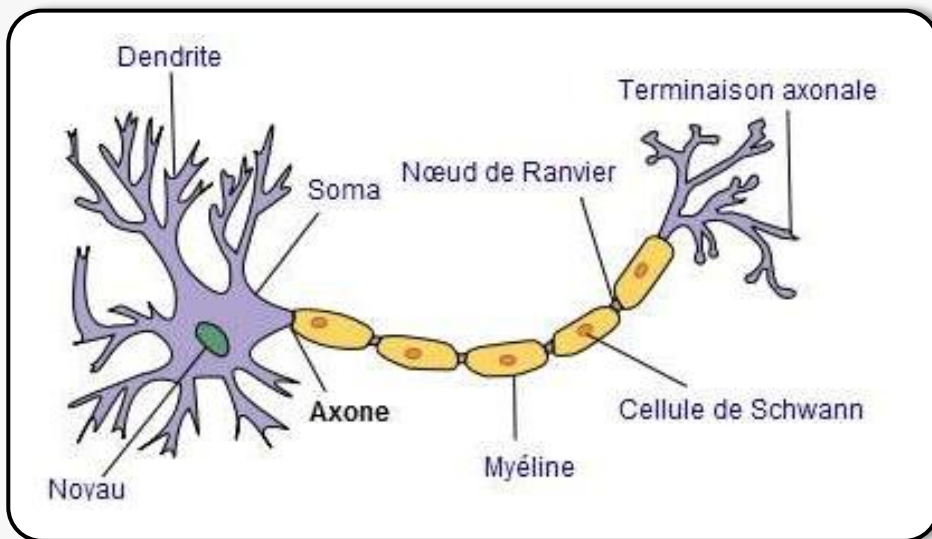


Machine Learning :



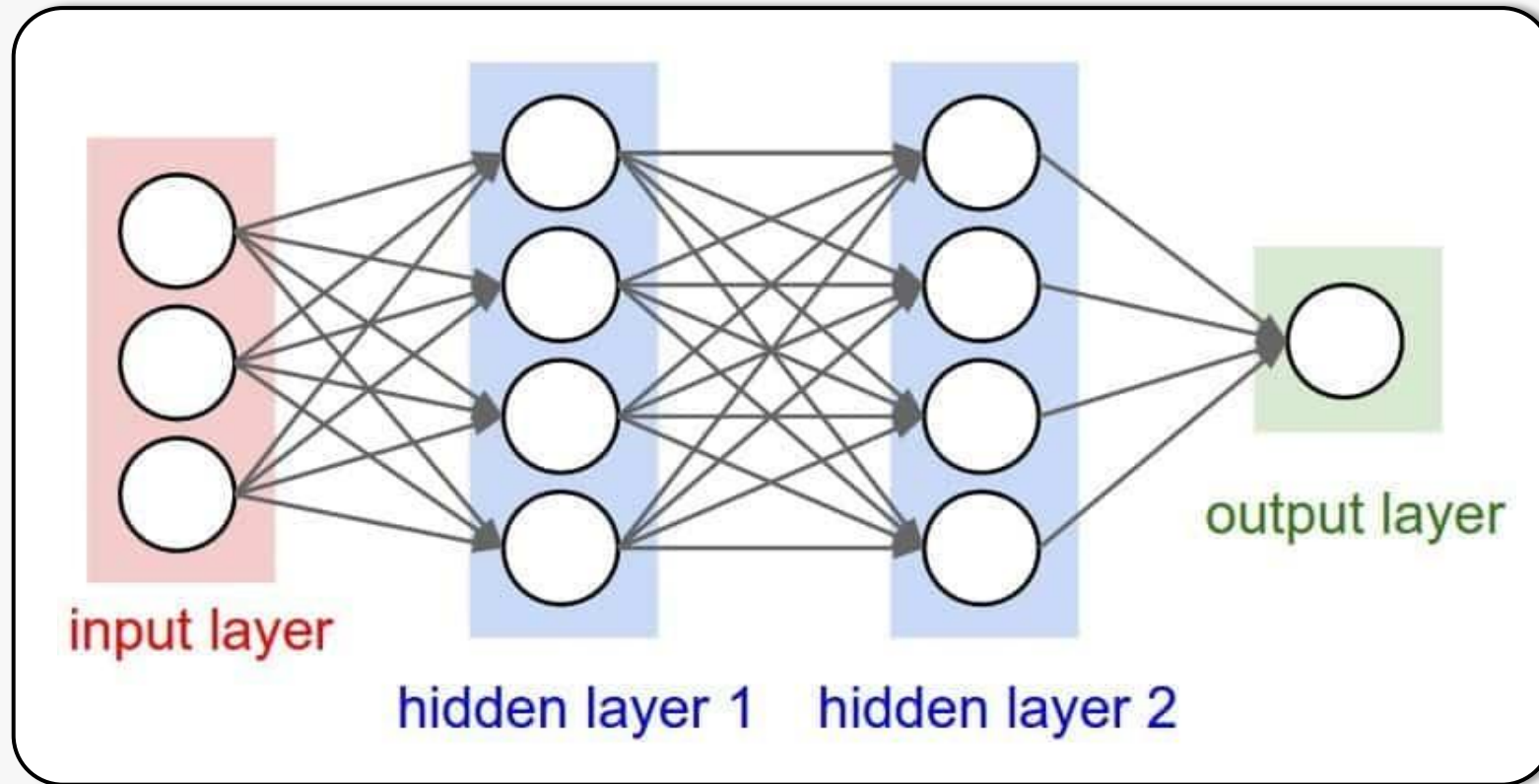
Petit aparté sur les concepts d'IA et de ML

Zoom sur le Deep Learning : du neurone au perceptron



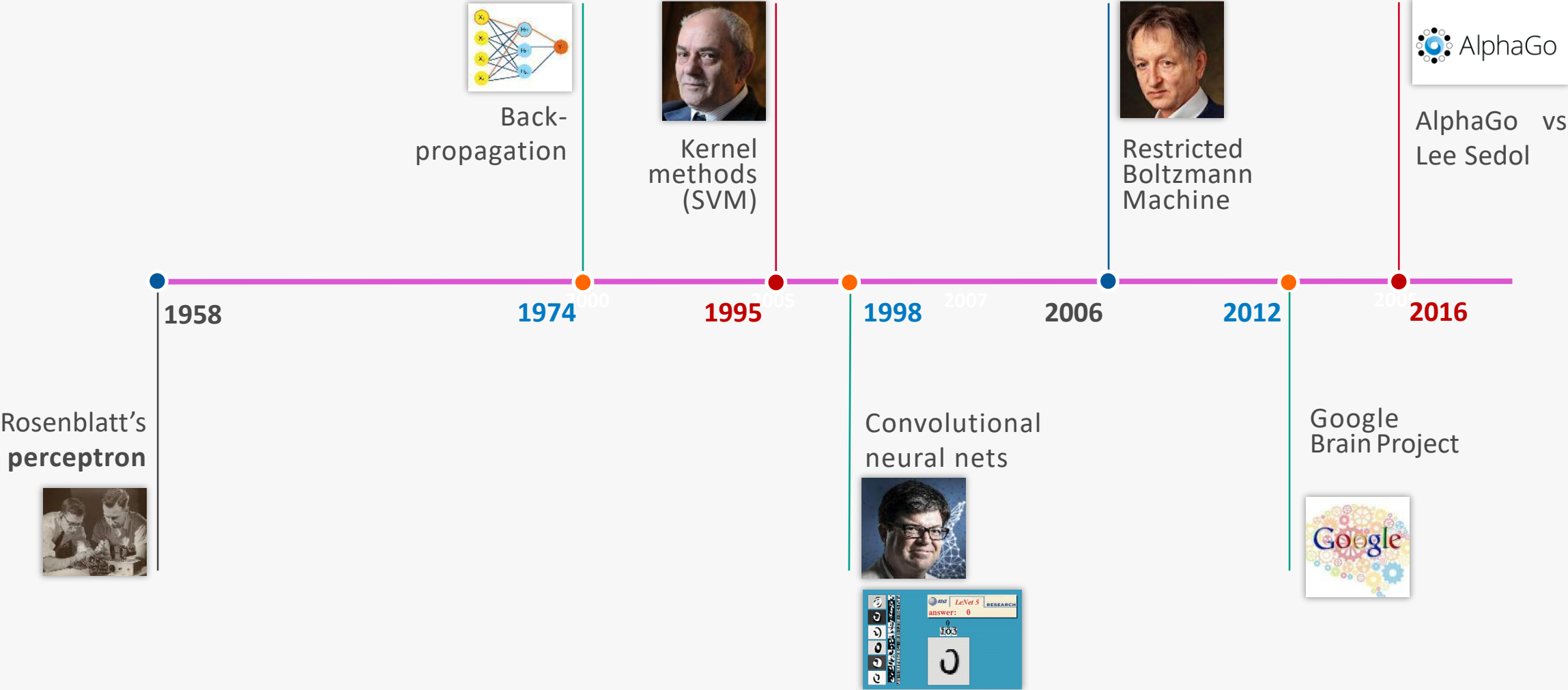
Petit aparté sur les concepts d'IA et de ML

Zoom sur le Deep Learning : réseaux de neurones



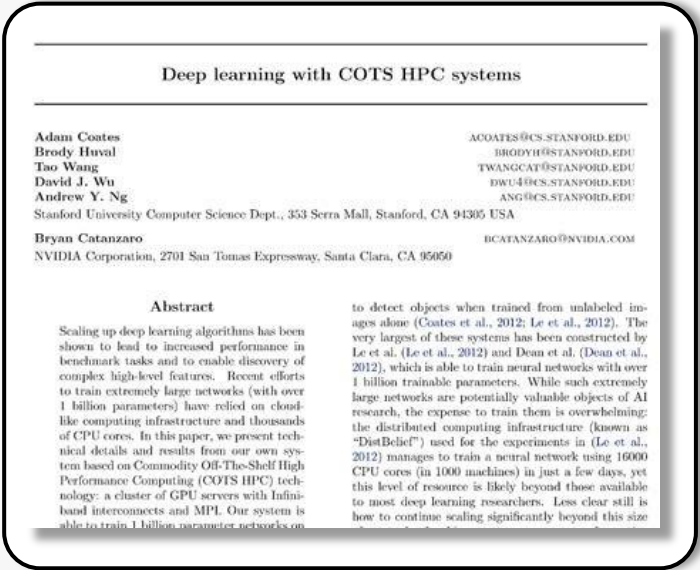
Petit aparté sur les concepts d'IA et de ML

Zoom sur le Deep Learning : chronologie

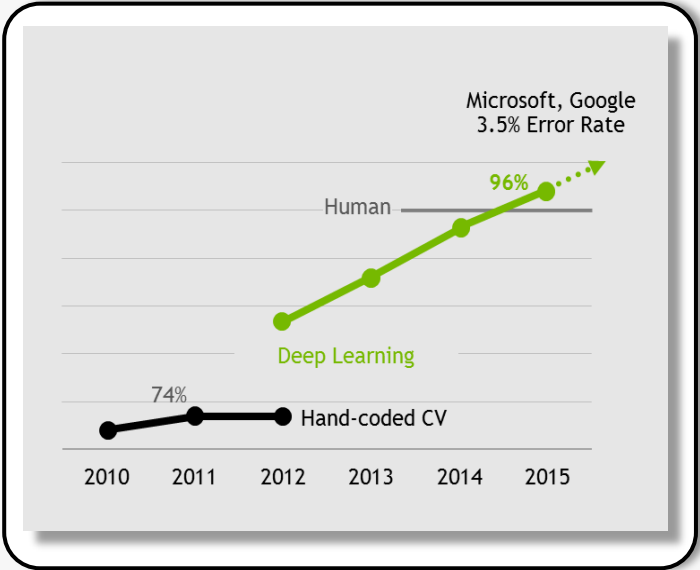


Petit aparté sur les concepts d'IA et de ML

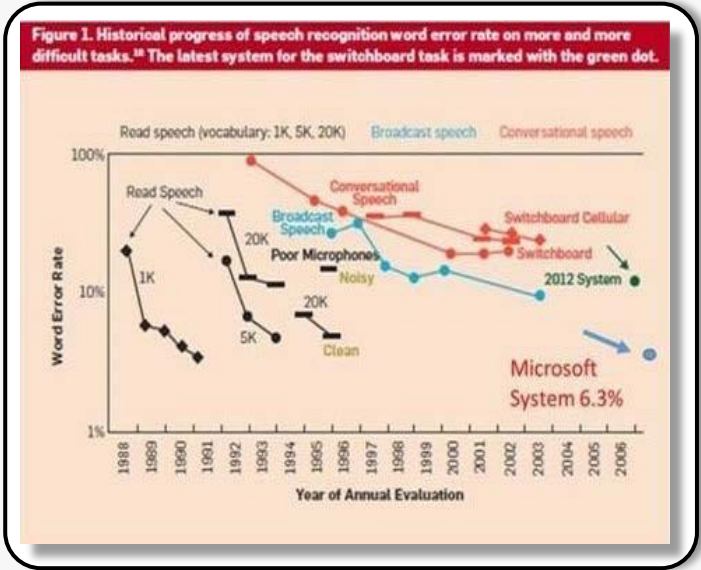
Zoom sur le Deep Learning : chronologie



2012 : Deep Learning et GPUs



2015 : ImageNet challenge - de meilleures résultats que l'homme dans la reconnaissance d'image grâce au Deep Learning

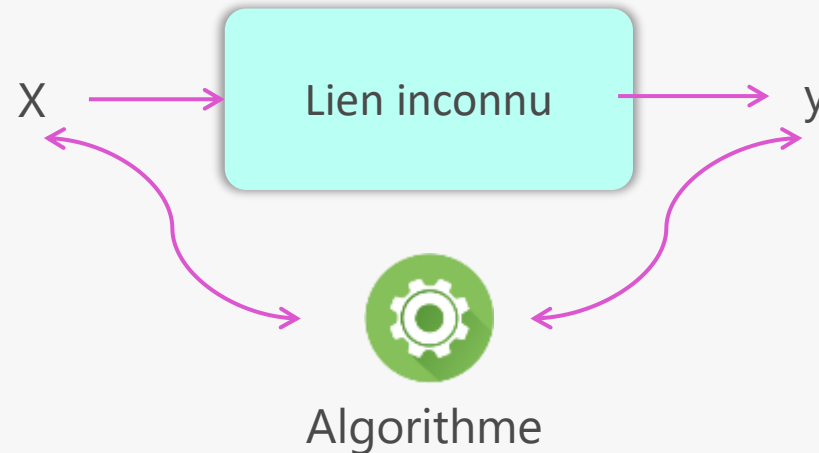


2016 : Microsoft annonce que son IA comprend la parole aussi bien qu'un humain

Petit aparté sur les concepts d'IA et de ML

Statistiques (inférentielles), analyse et algèbre

Nous cherchons à prédire la cible (quelle qu'elle soit) en utilisant les données disponibles.



Trouver une fonction f qui minimise la perte d'information ($\text{Loss}(f(X), y)$).

Petit aparté sur les concepts d'IA et de ML

Statistiques (inférentielles), analyse et algèbre

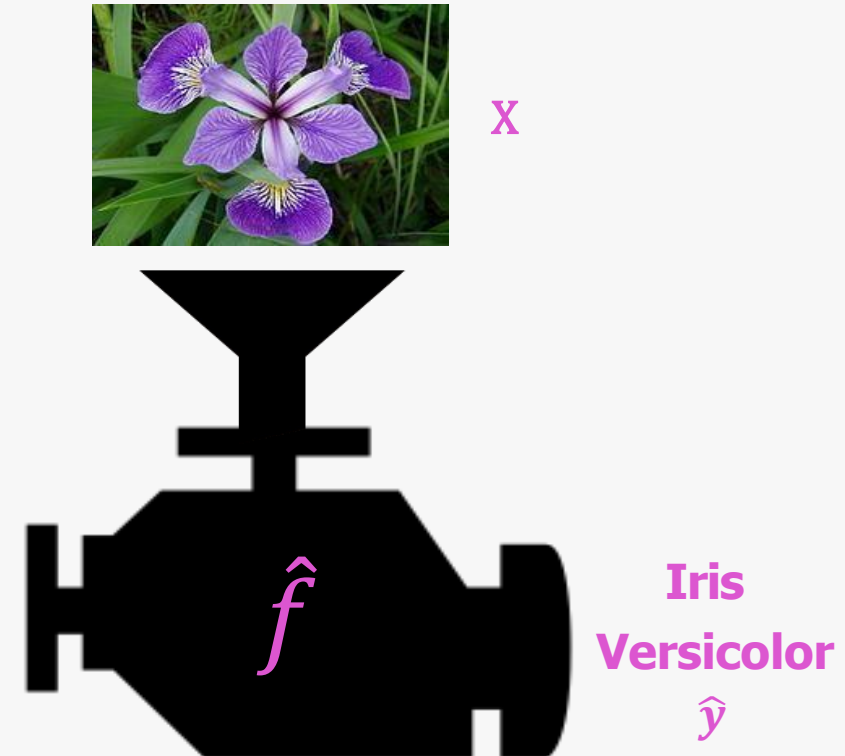
Le Machine Learning peut se résumer à la recherche de la meilleure **fonction $\hat{f}$** possible solution en f de l'équation :

$$y = f(X)$$

où :

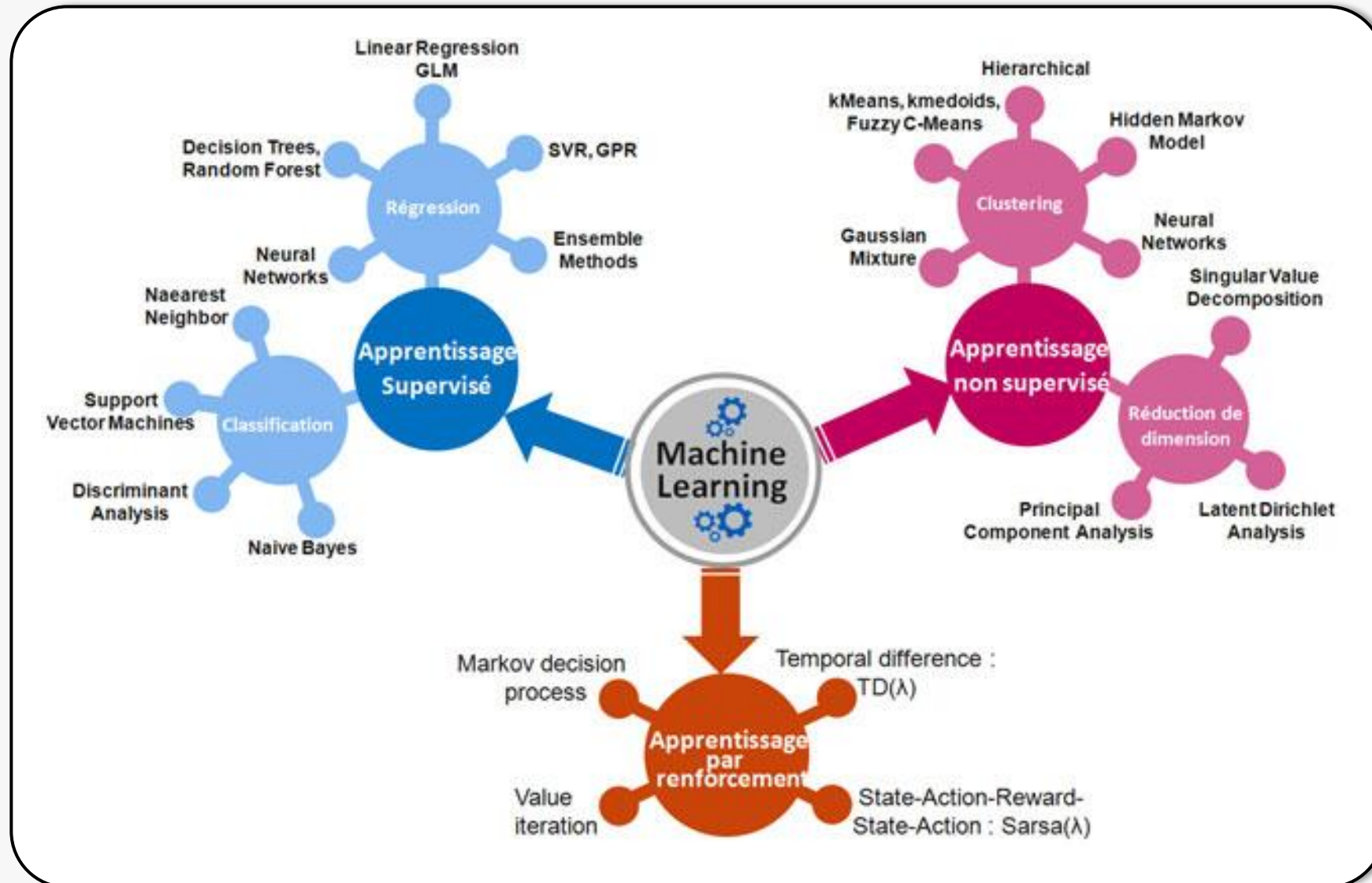
- ➡ X est constitué par l'ensemble des données d'entrée (csv, texte, image, vidéo, audio...)
- ➡ y est l'ensemble des valeurs de sortie (qualitatives, ex. : hot dog, cat..., ou quantitatives, ex. : 38.3, 42 ans...)

$\hat{f}$ correspond au modèle : l'ensemble des algorithmes mis en œuvre pour estimer la valeur de sortie



Petit aparté sur les concepts d'IA et de ML

Les différents types d'apprentissage



Apprentissage supervisé : répéter un exemple

- Un algorithme supervisé apprend à faire des prédictions ou des classifications en se basant sur des **données d'entraînement étiquetées**. Dans cette approche, l'ensemble de données utilisé pour l'entraînement est constitué d'**exemples d'entrée associés à leurs sorties souhaitées** ou étiquettes correspondantes.
- L'objectif de l'apprentissage supervisé est de permettre à l'algorithme de **généraliser les motifs à partir des données fournies**, puis d'**appliquer cette connaissance à de nouvelles données non vues pour réaliser des prédictions** ou des classifications précises.
- Le processus implique que l'algorithme ajuste de manière itérative ses paramètres internes afin de minimiser la différence entre ses prédictions et les étiquettes réelles dans les données d'entraînement. L'**algorithme apprend des écarts** et améliore progressivement sa capacité à réaliser des prédictions précises sur des données non vues.

Apprentissage supervisé : répéter un exemple

Apprentissage supervisé : Le modèle apprend grâce aux labels associés aux données.
Exemple : Identifier si un champignon est dangereux ou comestible.



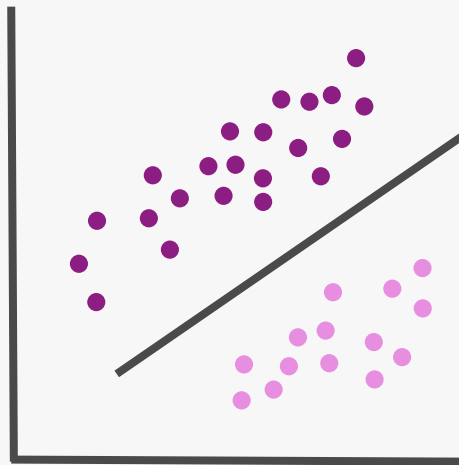
X : Couleur du pied, odeur, hauteur, présence d'un bulbe etc.
 Y : Label « dangereux » ou « comestible »

Algorithmes d'apprentissage supervisé : k-plus proches voisins (KNN), régression logistique, arbre de décision, SVM ...

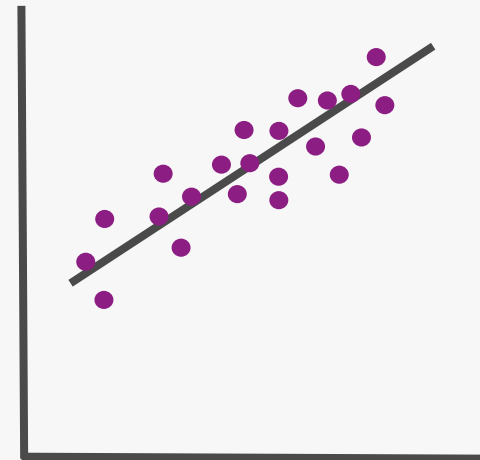
Apprentissage supervisé : répéter un exemple

L'apprentissage supervisé peut être catégorisé en deux types principaux : la classification et la régression.

- ➡ En **classification**, l'objectif est de prédire une étiquette catégorielle ou une **classe**.
- ➡ En **régression**, l'objectif est de prédire une **valeur numérique**.



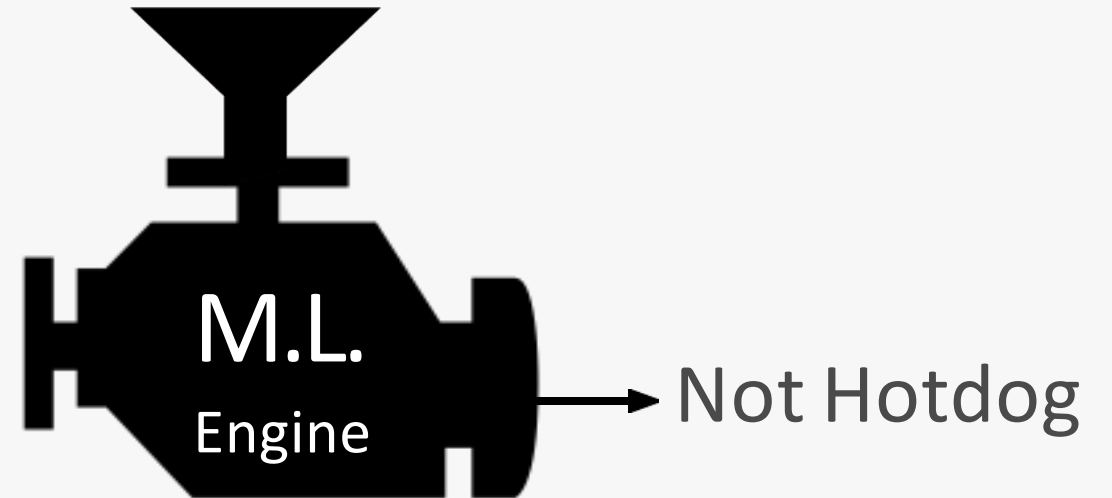
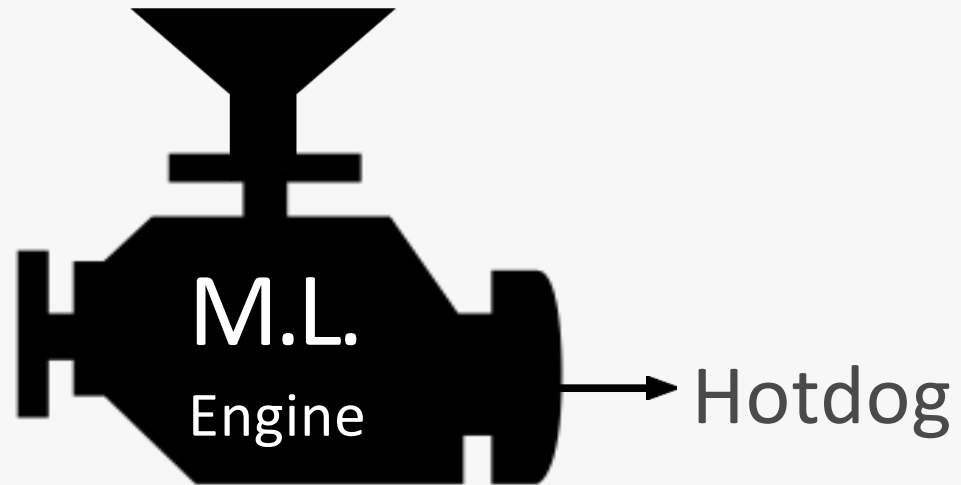
Classification



Régression

Apprentissage supervisé : répéter un exemple

A votre avis, classification ou régression ?



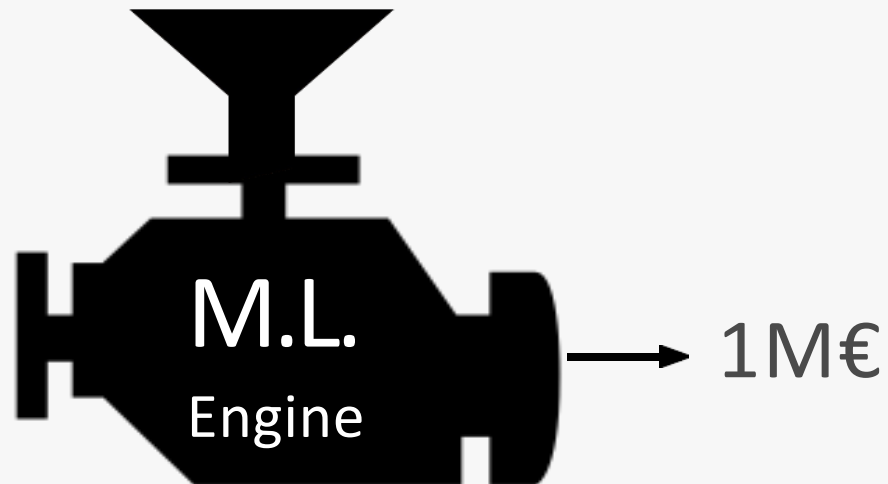
Apprentissage supervisé : répéter un exemple

A votre avis, classification ou régression ?



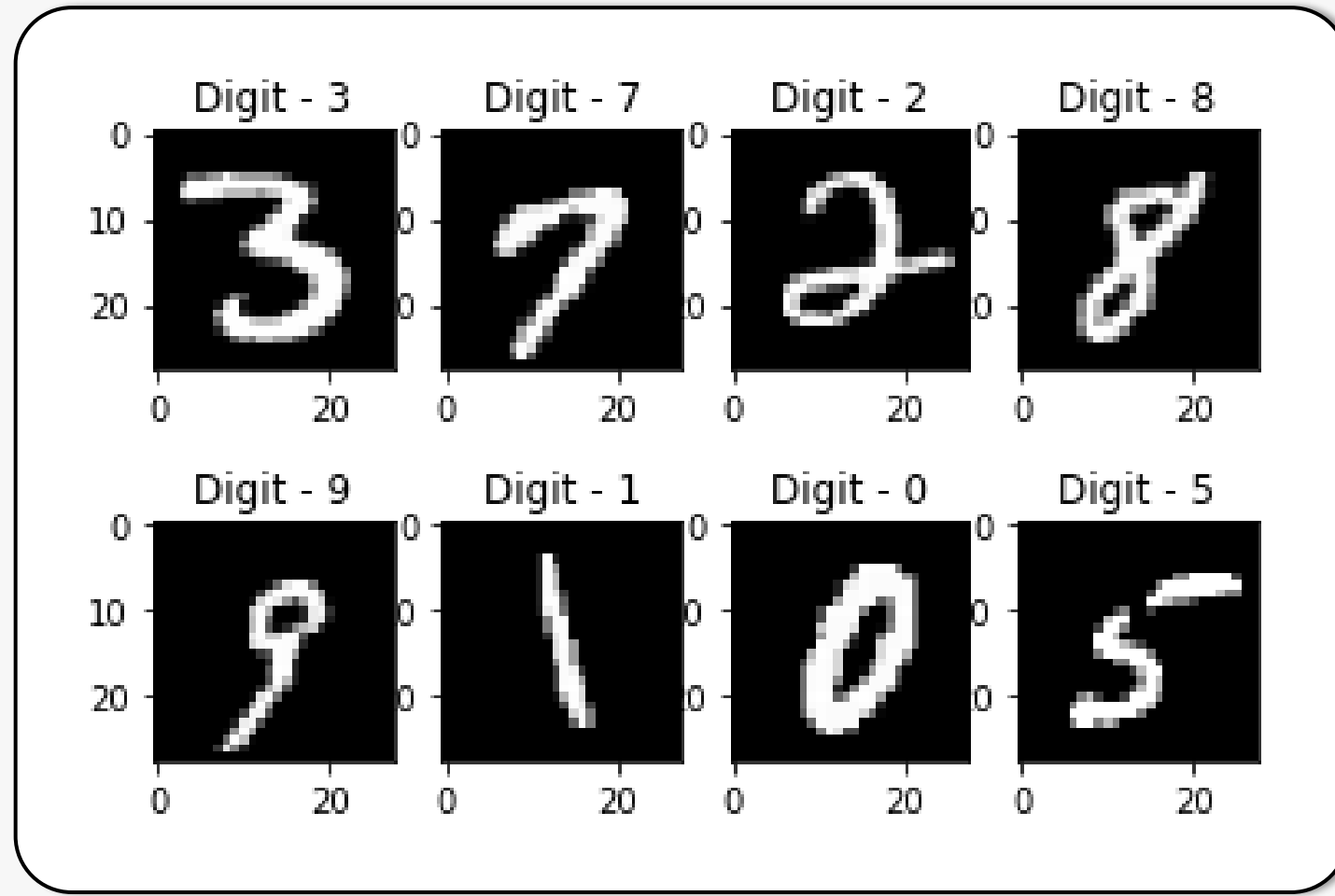
Surface: 100m²

Localisation: Paris 14^{ème}



Apprentissage supervisé : répéter un exemple

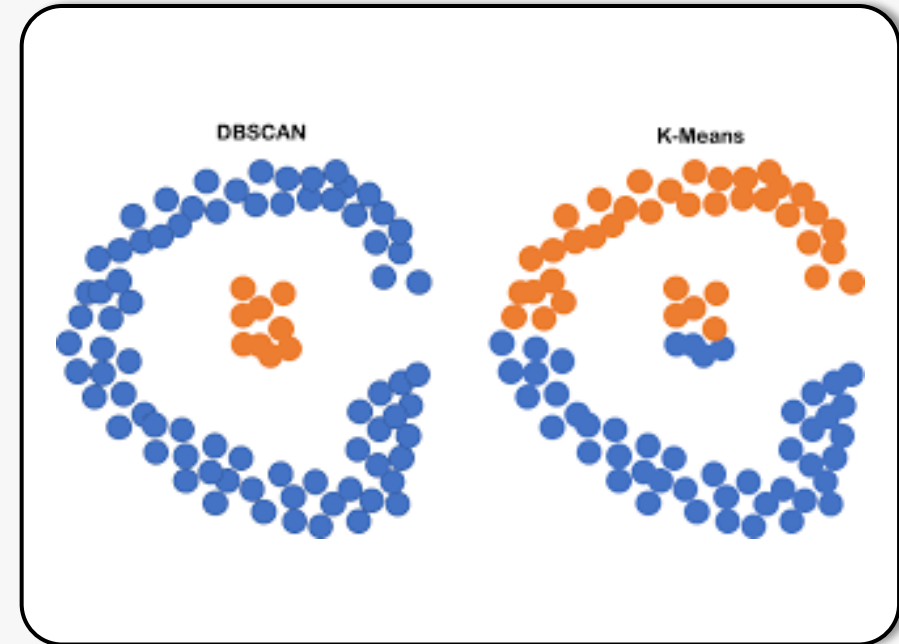
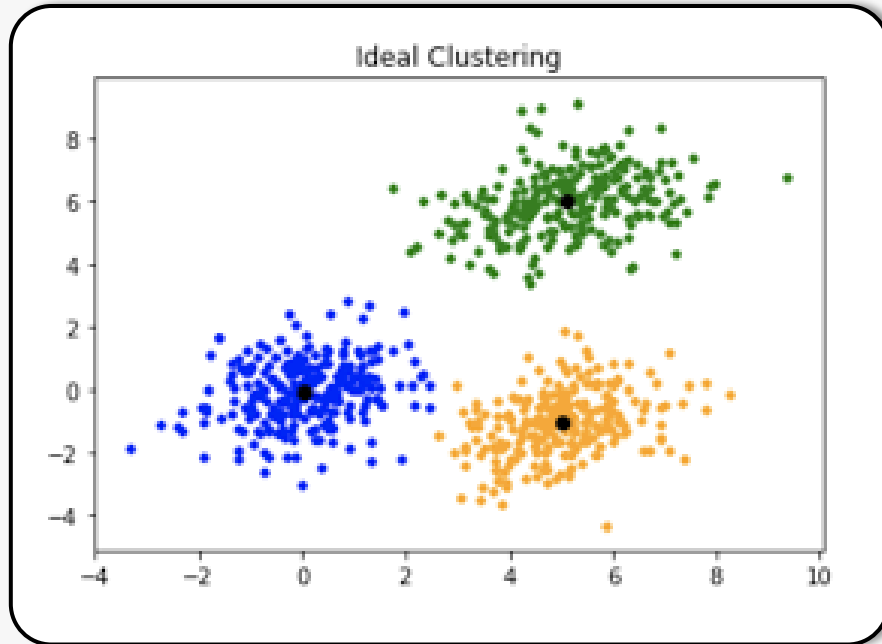
A votre avis, classification ou régression ?



Apprentissage non supervisé : découvrir les données

- L'apprentissage non supervisé est une approche clé de l'apprentissage automatique où les algorithmes explorent des données **sans étiquettes ni réponses préexistantes**. Contrairement à l'apprentissage supervisé, où les données sont étiquetées, l'apprentissage non supervisé vise à **découvrir des structures** et des relations cachées au sein des données. Les algorithmes non supervisés sont utilisés pour des tâches telles que la **regroupement** (clustering) et la **réduction de dimensionnalité**.
- Dans le regroupement, les algorithmes identifient des motifs ou des groupes naturels dans les données en fonction de similarités entre les exemples. Par exemple, un algorithme de clustering pourrait **segmenter des clients** en groupes basés sur leurs comportements d'achat similaires, sans avoir besoin de connaître à l'avance les catégories de ces groupes.
- La réduction de dimensionnalité, quant à elle, permet de **simplifier les données** en réduisant le nombre de caractéristiques tout en préservant l'information essentielle. Cela facilite la visualisation et l'analyse des données complexes.
- L'apprentissage non supervisé est par exemple très utilisé en commerce et marketing via la segmentation de marché, et de clients, donc. Il permet de trouver des tendances, des structures et des regroupements significatifs dans les données, ce qui peut conduire à des découvertes utiles et à une meilleure compréhension des données sans nécessiter de directives préalables.

Apprentissage non supervisé : découvrir les données



Machine learning

Packages ML de Spark

Deux packages principaux :

MLlib



Première bibliothèque de Machine Learning intégrée à Spark (développement démarré en 2012 et open-sourcé en 2013, 1^{er} package avec Spark 0.8).

Offre une gamme d'algorithmes de Machine Learning distribués, adaptés au traitement de grands volumes de données.

Comprend des algorithmes pour la classification, la régression, le clustering, la réduction de dimension, la classification de texte...

Basé sur les RDD.

Obsolète, mais reste relativement utilisée.

Spark ML



Nouvelle API de Machine Learning introduite dans les versions plus récentes de Spark.

Construit sur les DataFrames (ce qui rend son utilisation plus conviviale).

API plus riche et plus cohérente pour la construction de pipelines de Machine Learning, l'optimisation des modèles et l'évaluation des performances.

Inclut des fonctionnalités avancées telles que le support des modèles d'apprentissage profond, la prise en charge des transformations de données structurées et non structurées, ainsi que des outils pour l'optimisation des hyperparamètres.

Machine learning

Les abstractions de Spark ML

- Pipelines ML** Permettent de chaîner ensemble plusieurs étapes de traitement des données et d'apprentissage dans un flux unique.
- Transformers** Composants qui transforment un DataFrame en un autre (par exemple en ajoutant de nouvelles colonnes ou en modifiant les données existantes).
- Estimateurs** Algorithmes d'apprentissage qui peuvent être ajustés sur des données pour produire un modèle.
Utilisés comme étape finale dans un pipeline.
- Modèles** Résultats de l'entraînement des estimateurs sur des données.
Peuvent être utilisés pour faire des prédictions ou pour évaluer la qualité du modèle.



L'API Spark ML propose également des outils pour la sélection et l'évaluation de modèles, comme la validation croisée (cross validation) et la recherche de grille (grid search).

Machine learning

Spark ML

La documentation est extrêmement détaillée, c'est la golden source pour faire du ML avec Spark.

Il faut avoir le réflexe de s'y référer !

Spark ML Programming Guide

`spark.ml` is a new package introduced in Spark 1.2, which aims to provide a uniform set of high-level APIs that help users create and tune practical machine learning pipelines. It is currently an alpha component, and we would like to hear back from the community about how it fits real-world use cases and how it could be improved.

Note that we will keep supporting and adding features to `spark.mllib` along with the development of `spark.ml`. Users should be comfortable using `spark.mllib` features and expect more features coming. Developers should contribute new algorithms to `spark.mllib` and can optionally contribute to `spark.ml`.

Table of Contents

- [Main Concepts](#)
 - [ML Dataset](#)
 - [ML Algorithms](#)
 - [Transformers](#)
 - [Estimators](#)
 - [Properties of ML Algorithms](#)
 - [Pipeline](#)
 - [How It Works](#)
 - [Details](#)
 - [Parameters](#)
- [Code Examples](#)
 - [Example: Estimator, Transformer, and Param](#)
 - [Example: Pipeline](#)

Machine learning

Démo



**Parcours de la documentation
de Spark ML**

Le cycle analytique ML avec Spark

Voyons maintenant le cycle analytique des données dans un cadre d'apprentissage automatique avec Spark.



Ingestion de données

- **Identification des sources de données** : fichiers locaux ou données sur le cloud, bases de données, flux de données en temps réel...
- **Choix de la méthode d'ingestion** : Spark SQL, Spark Streaming, Spark Structured Streaming...
- **Chargement des données** : `spark.read.csv()`, `spark.read.json()`, `spark.read.jdbc()`...
- **Traitement initial des données** : (suppression des données inutiles, filtrage des valeurs aberrantes, conversion de types de données...)
- **Traitement spécifique** (parfois) : par exemple, flux de données en continu
- **Persistance des données** : tables Spark, fichiers Parquet, bases de données...
- Vérification de la **qualité des données**

Préparation et feature engineering

Le chargement des données

```
df_csv = spark.read.csv(  
    "chemin/fichier.csv",  
    header=True,  
    inferSchema=True,  
)  
  
df_json = spark.read.json("chemin/fichier.json")
```

Préparation et feature engineering

Créer des DataFrames (rappel)

```
# spark is an existing SparkSession
df = spark.read.json("examples/src/main/resources/people.json")
# Displays the content of the DataFrame to stdout
df.show()
# +-----+-----+
# | age| name|
# +-----+-----+
# |null|Michael|
# | 30| Andy|
# | 19| Justin|
# +-----+-----+
```

Préparation et feature engineering

Exploration

Comprendre la structure et les caractéristiques des données, identifier les schémas, les tendances et les relations entre les variables...

Chargement des données (rappel)

```
# Chargement des données à partir d'un fichier CSV
df = spark.read.csv(
    "chemin/fichier.csv",
    header=True,
    inferSchema=True,
)
```

Préparation et feature engineering

Exploration

Analyse de la structure des données (rappel)

```
# Afficher le schéma du DataFrame
df.printSchema()

# Afficher les premières lignes du DataFrame
df.show()
```

Préparation et feature engineering

Le chargement des données

Analyse de la structure des données (rappel)

```
# Afficher le schéma du DataFrame
df.printSchema()

# Afficher les premières lignes du DataFrame
df.show()
```

Préparation et feature engineering

Données quantitatives et qualitatives

- Les variables **qualitatives** représentent des caractéristiques non numériques qui décrivent des **catégories** ou des **groupes distincts**.
- Elles sont souvent utilisées pour classer ou regrouper des données en fonction de caractéristiques spécifiques, telles que le genre, la couleur, la région géographique...
- Les variables qualitatives ne peuvent pas être soumises à des opérations mathématiques standard, mais elles jouent un rôle crucial dans l'analyse des tendances et des modèles au sein de différents groupes.

➔ **Au sein des variables qualitatives, une caractéristique essentielle à bien prendre en compte est la présence ou non d'une relation d'ordre sur les valeurs.** La possibilité de hiérarchiser les valeurs a un impact considérable sur la manière de les traiter et de les modéliser.

Couleur
Rouge
Jaune
Bleu
Marron
Vert

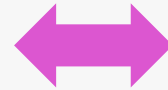
Avis client
Très satisfait
Satisfait
Moyennement satisfait
Peu satisfait
Totalement insatisfait

Préparation et feature engineering

Données quantitatives et qualitatives

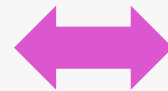
➡ A noter qu'il existe un ensemble particulier de valeurs qualitatives, très répandu : celui des variables binaires.

Statut marital
Marié
Non Marié
Marié
Non Marié



Marié
1
0
1
0

A décroché
Oui
Non
Non
Oui



A décroché
1
0
0
1

Préparation et feature engineering

Caractère itératif de l'exploration et de la préparation des données

Le nettoyage des données implique généralement plusieurs étapes telles que la suppression des valeurs manquantes, le traitement des valeurs aberrantes, la normalisation des données.

Ce nettoyage est généralement réalisé dans la foulée de ce qui aura été identifié dans l'analyse exploratoire et tourné vers les traitements qu'on souhaite effectuer. Les frontières entre ces différentes étapes théoriques sont en pratique généralement assez diffuses : les itérations sont classiquement nombreuses.

Exploration

Statistiques descriptives (rappel)

```
# Calculer les statistiques descriptives pour les colonnes numériques
df.describe().show()
```

	fat_100g	saturated-fat_100g	sugars_100g	fiber_100g	proteins_100g
count	320772.000000	320772.000000	320772.000000	320772.000000	320772.000000
mean	9.679226	3.671132	12.221732	1.79242	5.733644
std	16.262918	7.163720	20.662178	10.27665	8.061912
min	0.000000	0.000000	-17.860000	-6.70000	-800.000000
25%	0.000000	0.000000	0.000000	0.00000	0.000000
50%	1.150000	0.020000	2.860000	0.00000	3.000000
75%	14.290000	4.500000	14.840000	2.30000	8.240000
max	714.290000	550.000000	3520.000000	5380.00000	430.000000

Exploration

Analyse des distributions

```
import matplotlib.pyplot as plt

plt.hist(column_data, bins=20)
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.title("Distribution of column_name")
plt.show()
```

matplotlib est une bibliothèque « **locale** » : elle tourne sur le **driver**. Les DataFrame Spark résident dans le cluster. Pour qu'un graphique standard voie les données, il faut les ramener sur le driver (avec `collect()` ou `toPandas()`).

Transférez seulement ce qui est nécessaire au graphique (échantillons, agrégats, statistiques).

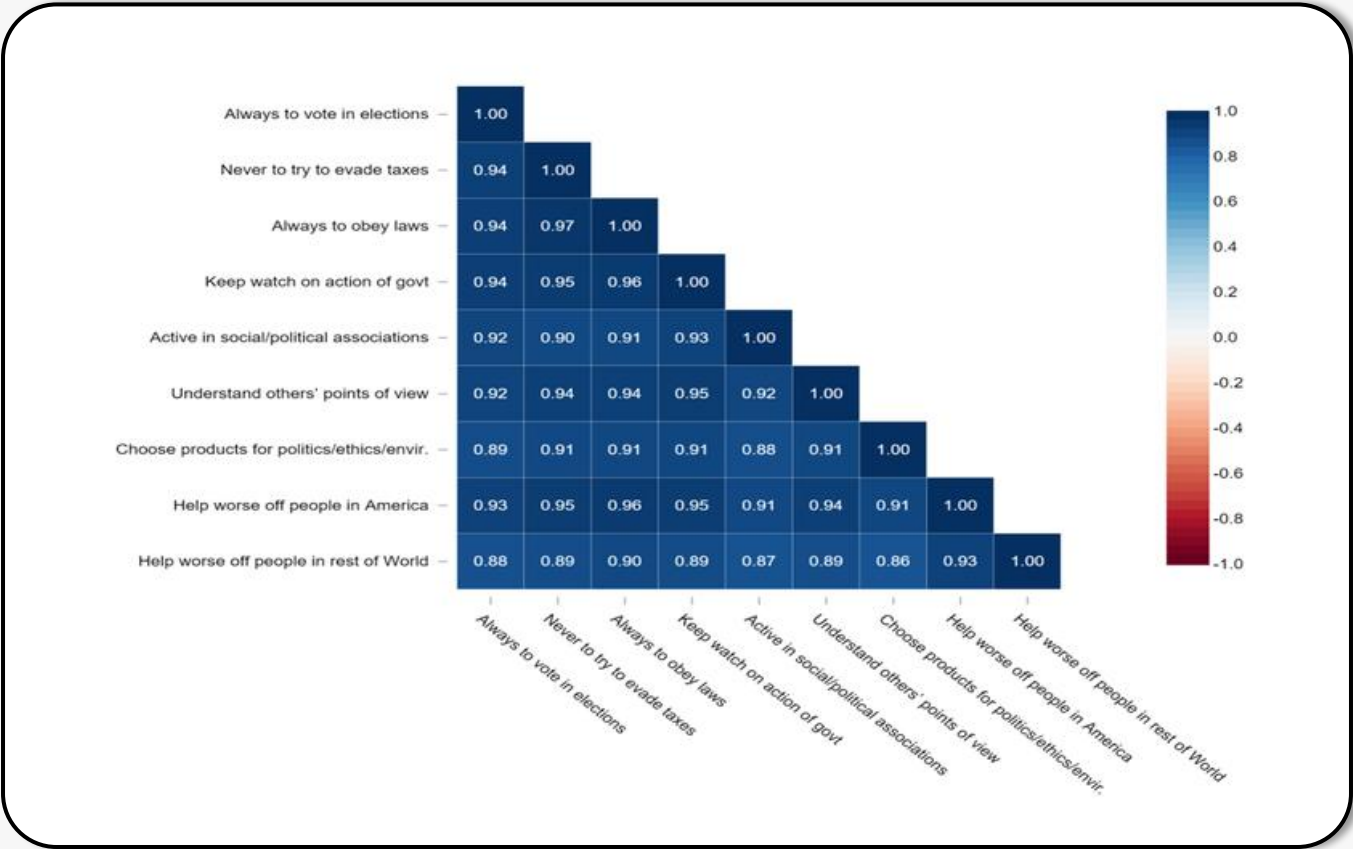
Dès lors, même avec un DataFrame Spark de plusieurs gigaoctets, on peut sans problème tracer :

- un boxplot issu de 1 000 points-témoins tirés au sort,
- une courbe d'évolution quotidienne déjà réduite à 365 points,
- une heatmap de corrélations calculée sur les seules 20 colonnes numériques...

Il existe aussi des librairies pour visualiser des gros jeux de données : `datashader`, `vaex`...

Exploration

Analyse des corrélations

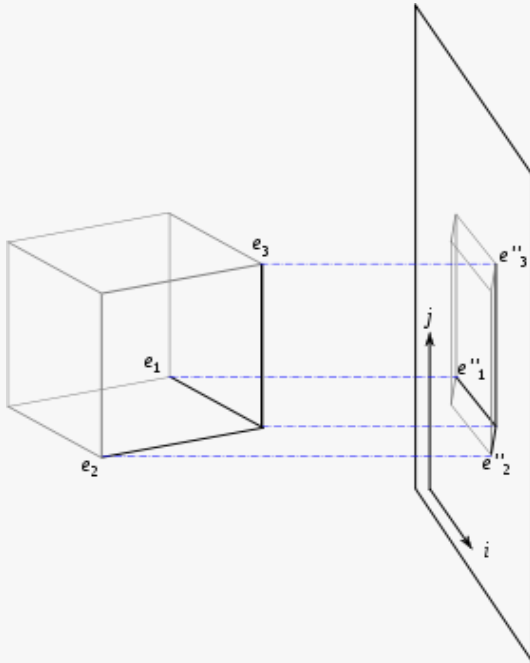


Préparation et feature engineering

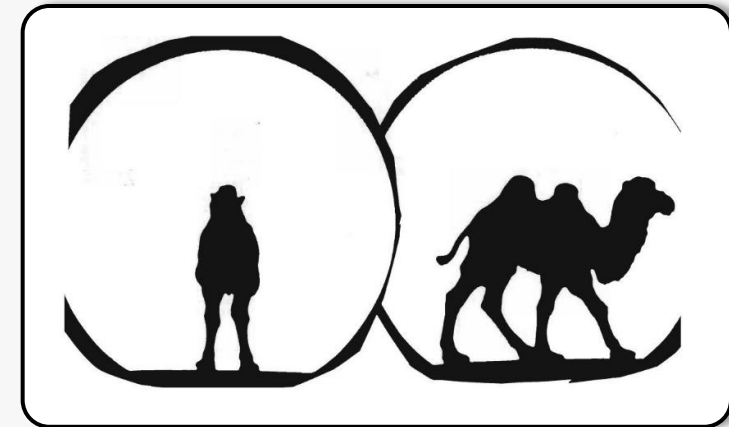
ACP : objectifs et projections

L'ACP a deux objectifs principaux. Elle permet d'étudier :

- ➡ la **variabilité entre les individus** : quelles sont les différences et les ressemblances entre individus ?
- ➡ les **liaisons entre les variables** : y a-t-il des groupes de variables très corrélées entre elles, qui peuvent être regroupées en de nouvelles variables synthétiques ?



Il y a des projections pour lesquelles on voit moins bien que d'autres, avec lesquelles on perd plus d'information que d'autres (exemple du cinéma).

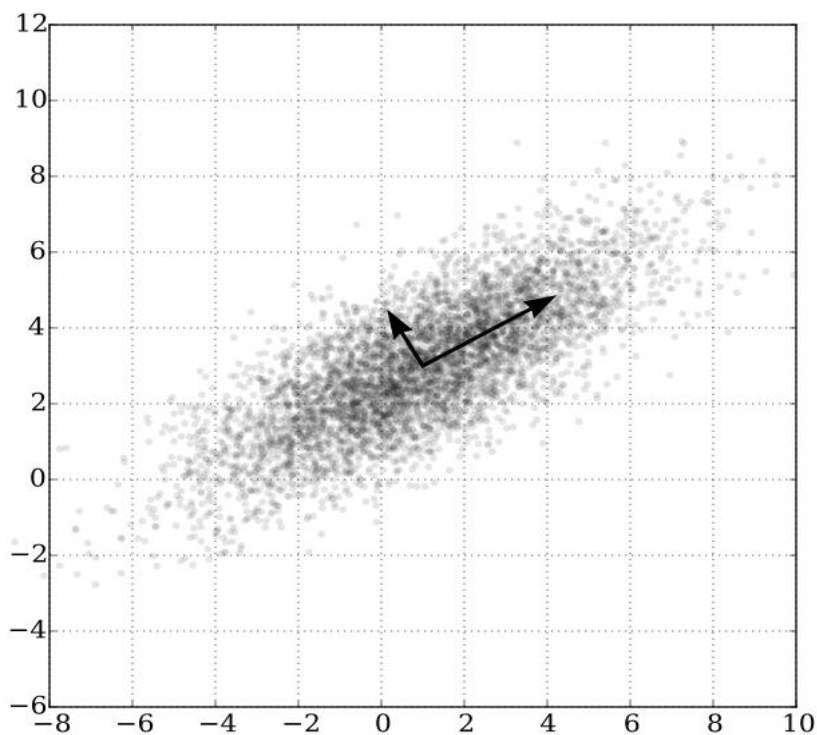


Préparation et feature engineering

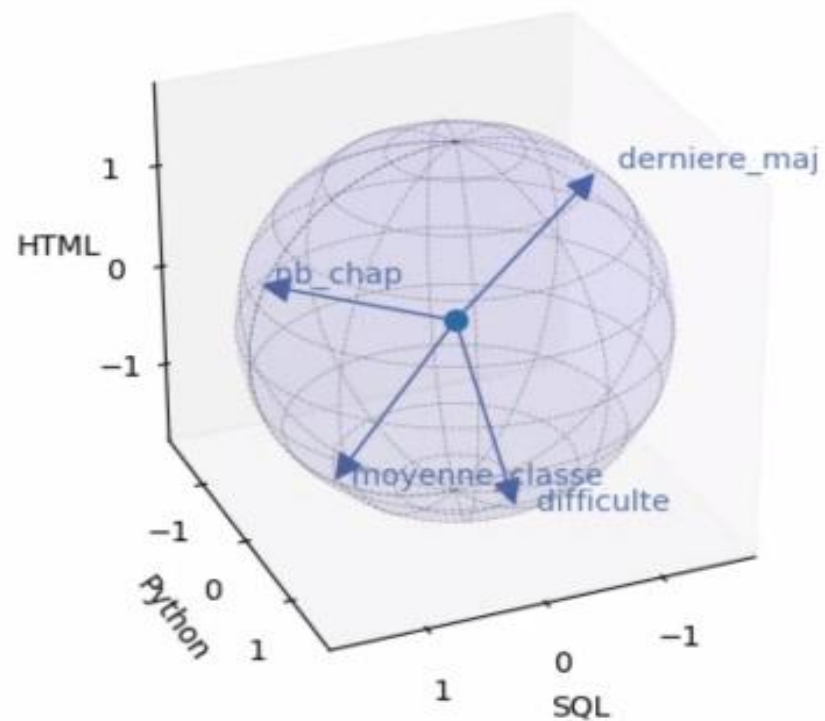
ACP : nuages des individus et des variables

Mots-clefs

Espaces vectoriels normés, réduction d'endomorphismes, sous-espaces propres, matrice de covariance, axes d'inertie.



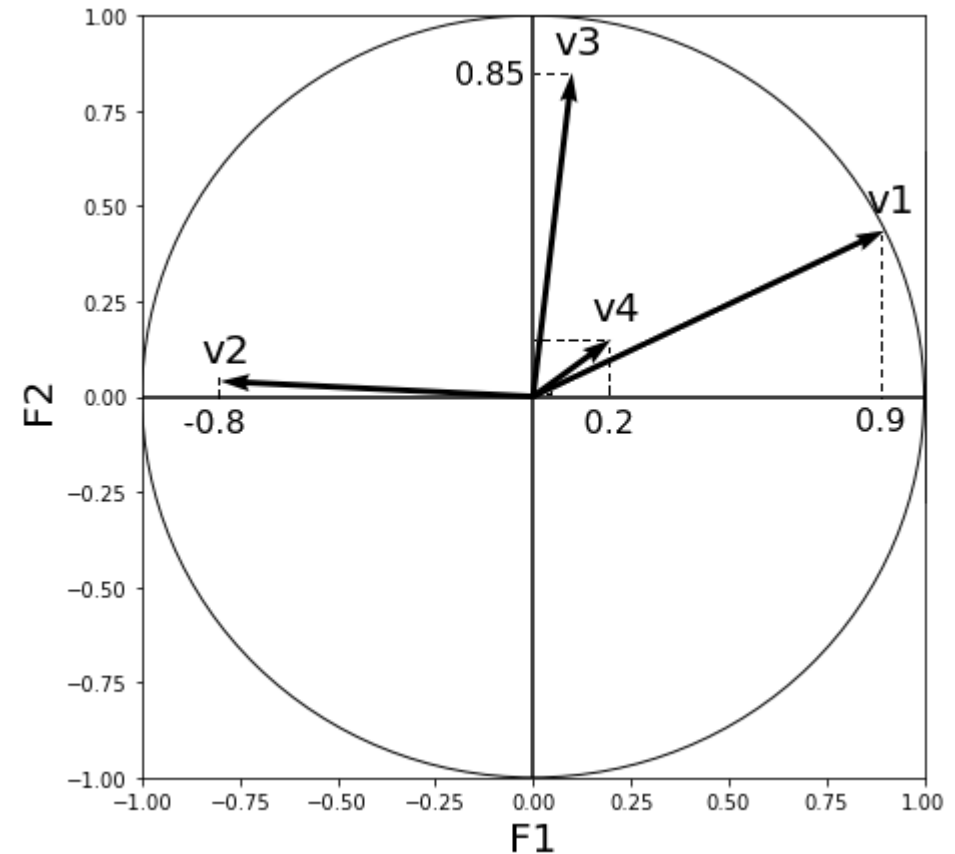
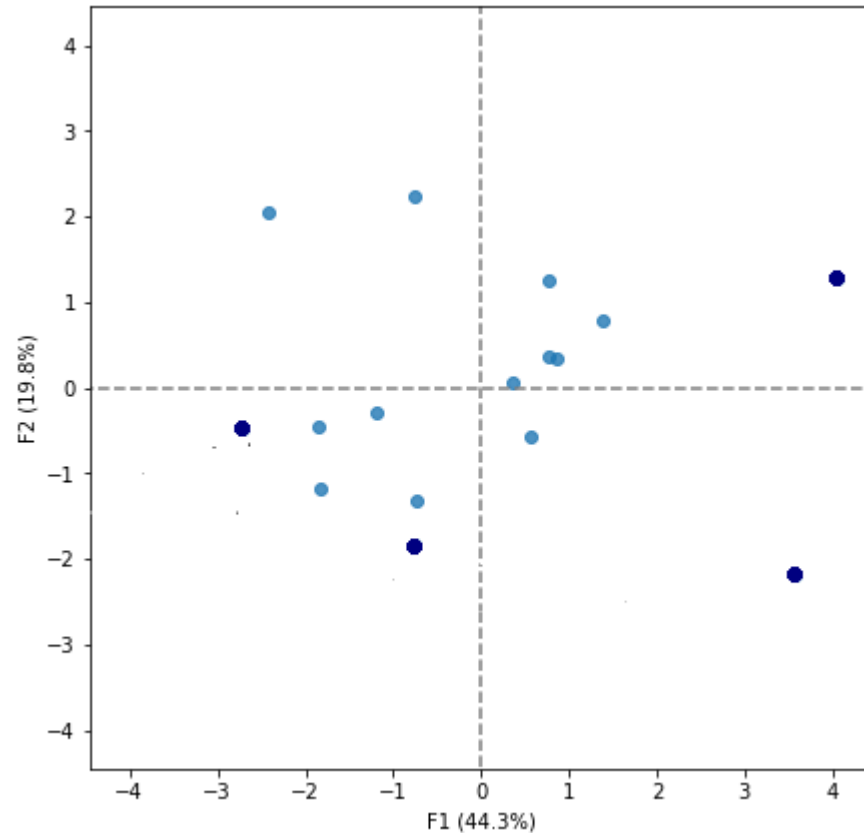
Nuage des individus



Nuage des variables

Préparation et feature engineering

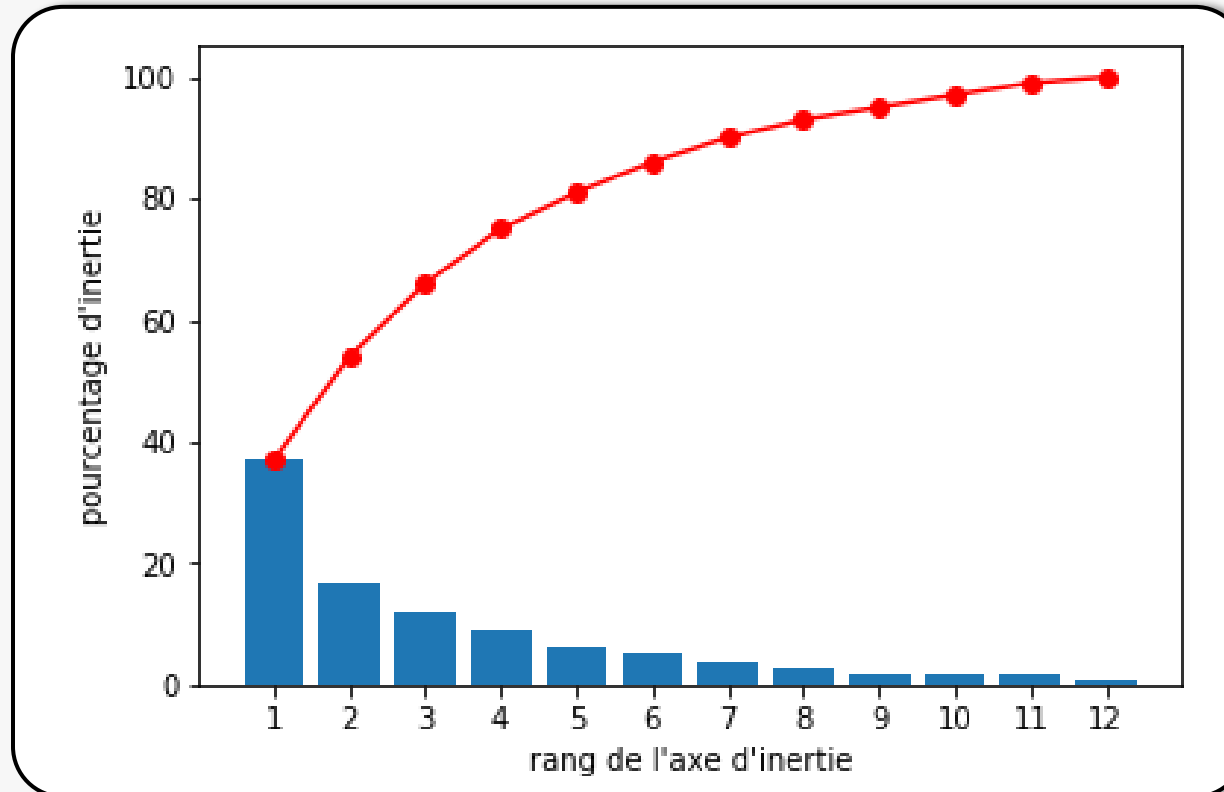
ACP : plan factoriel et cercle des corrélations



Plan factoriel et cercle des corrélations

Préparation et feature engineering

ACP pour réduire la dimension : éboulis des valeurs propres



Eboulis des valeurs propres

Préparation et feature engineering

Spécificités de l'ACP

- Déjà, certes l'ACP donne des informations sur la corrélation des variables, mais si les variables d'un groupe vous semblent corrélées, alors calculez tout simplement les coefficients de corrélation entre elles (ou la matrice de corrélation), et vous en aurez le cœur net ! De même, si certains individus semblent similaires, car ils ont des abscisses ou des ordonnées à peu près égales sur un plan factoriel, alors vérifiez-le sur vos données initiales.
- Comme l'ACP utilise le coefficient r^2 , elle ne peut donc mesurer que les liaisons **linéaires** entre les variables. Pour passer outre ce problème, on peut utiliser l'ACP avec noyau, ou kernel PCA en anglais. Le coefficient r^2 est très sensible aux outliers : c'est donc le cas aussi pour l'ACP. Avantage ou inconvénient ? Tout dépend de la situation.
- Autre limite potentielle : l'**interprétabilité** des axes (et donc des composantes sélectionnées si l'ACP est utilisée comme technique de réduction des dimensions). Mais il est possible de s'en sortir.

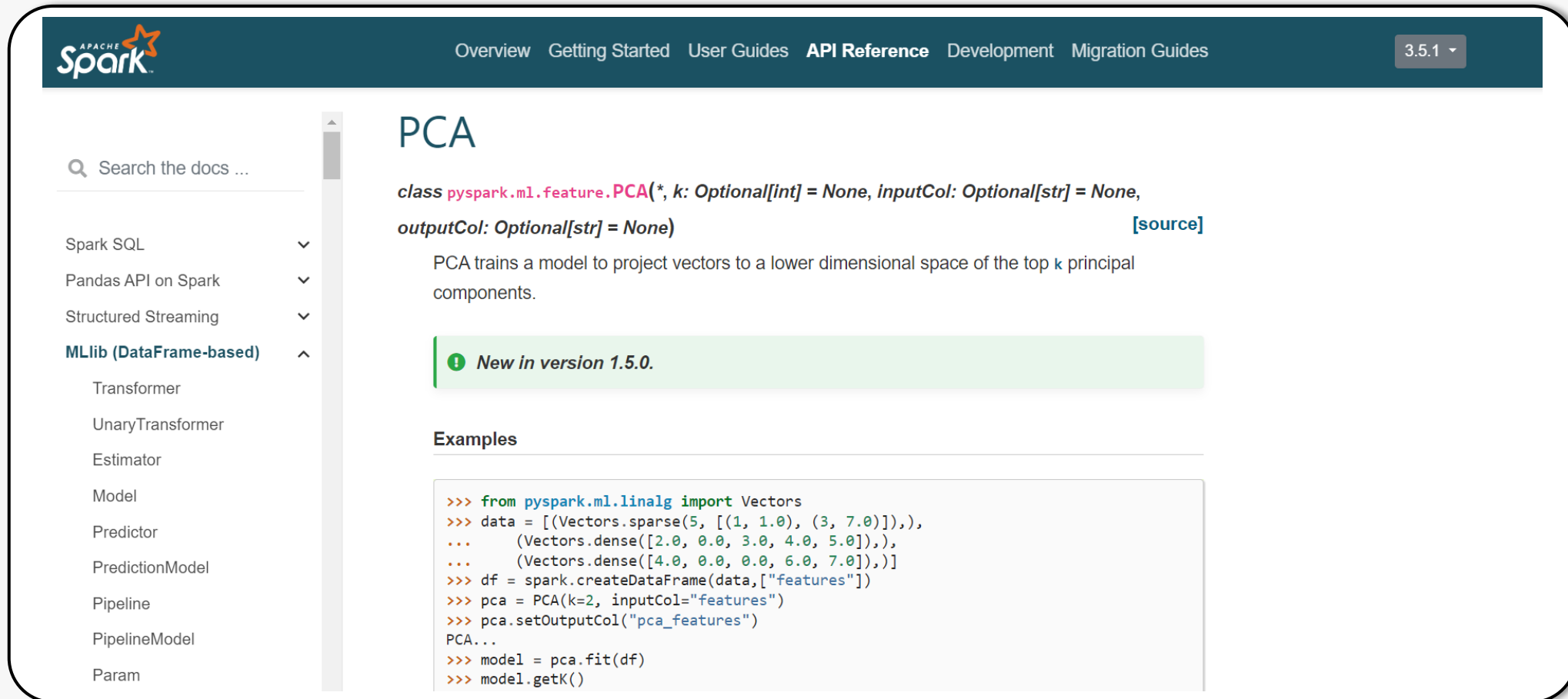
Parfois, un axe d'inertie n'était dû qu'à un petit groupe d'individus (ou même à un seul) : ce sont les outliers. Un individu situé très loin de tous les autres a tendance à « attirer » dans sa direction l'un des axes d'inertie (bien souvent le premier). La **non-robustesse de l'ACP aux outliers** n'est donc pas vraiment problématique. Au contraire, elle permet de détecter facilement les outliers, qu'ils soient intéressants ou non, et de rapidement « passer à autre chose » sans avoir à relancer le calcul de l'ACP (il suffit d'ignorer la composante concernée).
- Autre inconvénient de l'ACP : elle ne se limite qu'aux **variables quantitatives** ! Mais il existe d'autres méthodes factorielles permettant de traiter d'autres variables (l'analyse des correspondances multiples pour des variables qualitatives, et l'analyse factorielle des données mixtes).

Pour aller plus loin

https://www.youtube.com/watch?time_continue=2&v=FgakZw6K1QQ&embeds_referring_euri=https%3A%2F%2Fopenclassrooms.com%2F&embeds_referring_origin=https%3A%2F%2Fopenclassrooms.com&source_ve_path=Mjg2NjY&feature=emb_logo&ab_channel=StatQuestwithJoshStarmar

Préparation et feature engineering

ACP avec PySpark



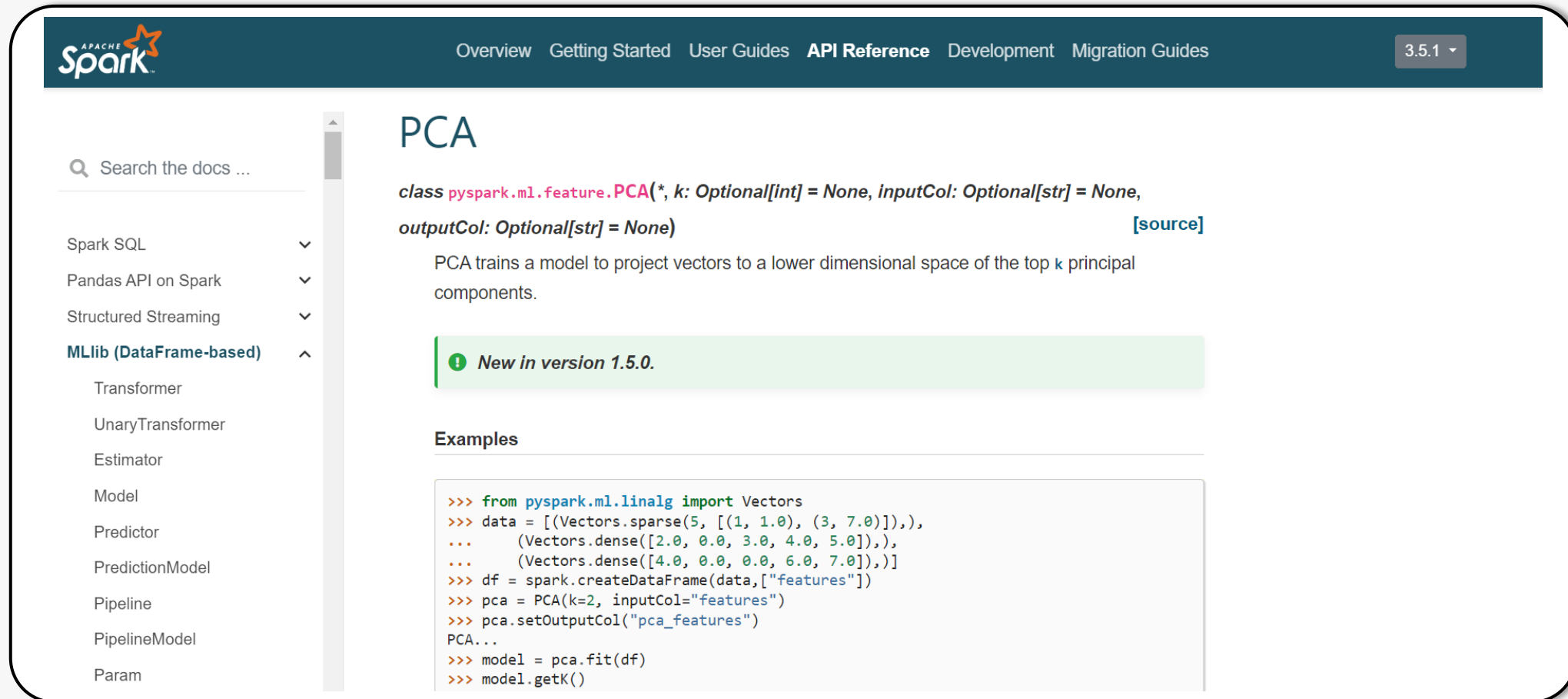
The screenshot shows the Apache Spark documentation page for the PCA class. The page has a dark blue header with the Spark logo and navigation links: Overview, Getting Started, User Guides, API Reference (selected), Development, and Migration Guides. A version dropdown shows 3.5.1. On the left, a sidebar contains a search bar and a list of categories: Spark SQL, Pandas API on Spark, Structured Streaming, MLlib (DataFrame-based) (expanded), Transformer, UnaryTransformer, Estimator, Model, Predictor, PredictionModel, Pipeline, PipelineModel, and Param. The main content area is titled 'PCA' and shows the class signature: `class pyspark.ml.feature.PCA(*, k: Optional[int] = None, inputCol: Optional[str] = None, outputCol: Optional[str] = None)` with a [source] link. Below the signature, it states: 'PCA trains a model to project vectors to a lower dimensional space of the top `k` principal components.' A green callout box indicates 'New in version 1.5.0.' The 'Examples' section contains a code block with the following Python code:

```
>>> from pyspark.ml.linalg import Vectors
>>> data = [(Vectors.sparse(5, [(1, 1.0), (3, 7.0)]),),
...         (Vectors.dense([2.0, 0.0, 3.0, 4.0, 5.0]),),
...         (Vectors.dense([4.0, 0.0, 0.0, 6.0, 7.0]),)]
>>> df = spark.createDataFrame(data, ["features"])
>>> pca = PCA(k=2, inputCol="features")
>>> pca.setOutputCol("pca_features")
PCA...
>>> model = pca.fit(df)
>>> model.getK()
```

<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.PCA.html>

Préparation et feature engineering

ACP avec PySpark



The screenshot shows the Apache Spark documentation page for the PCA class. The page has a dark blue header with the Spark logo and navigation links: Overview, Getting Started, User Guides, API Reference (selected), Development, and Migration Guides. A version dropdown shows 3.5.1. On the left, a sidebar contains a search bar and a list of categories: Spark SQL, Pandas API on Spark, Structured Streaming, MLlib (DataFrame-based) (expanded), Transformer, UnaryTransformer, Estimator, Model, Predictor, PredictionModel, Pipeline, PipelineModel, and Param. The main content area is titled 'PCA' and shows the class signature: `class pyspark.ml.feature.PCA(*, k: Optional[int] = None, inputCol: Optional[str] = None, outputCol: Optional[str] = None)` with a [source] link. Below the signature, it states: 'PCA trains a model to project vectors to a lower dimensional space of the top `k` principal components.' A green callout box indicates 'New in version 1.5.0.' The 'Examples' section contains a code block with the following Python code:

```
>>> from pyspark.ml.linalg import Vectors
>>> data = [(Vectors.sparse(5, [(1, 1.0), (3, 7.0)]),),
...         (Vectors.dense([2.0, 0.0, 3.0, 4.0, 5.0]),),
...         (Vectors.dense([4.0, 0.0, 0.0, 6.0, 7.0]),)]
>>> df = spark.createDataFrame(data, ["features"])
>>> pca = PCA(k=2, inputCol="features")
>>> pca.setOutputCol("pca_features")
PCA...
>>> model = pca.fit(df)
>>> model.getK()
```

<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.PCA.html>

Préparation et feature engineering

Analyse et préparation des données : traitement des valeurs manquantes

- Le traitement des valeurs manquantes occupe une place cruciale dans le domaine du Machine Learning, car des données incomplètes peuvent avoir un impact significatif sur la qualité des modèles, voire les rendre inutilisables.
- Différentes techniques existent pour gérer ce problème.
- La **quantité** de valeurs manquantes et la compréhension de la **raison** au manque entrent en compte dans le choix de la technique appliquée.

Les familles de techniques employées sont :

- ➡ dans le cas de *fausses* valeurs manquantes, le **remplacement par la valeur attendue** (0, chaîne de caractère vide),
- ➡ la **suppression** de données (lignes ou colonnes),
- ➡ l'**imputation simple**, par exemple par la moyenne ou la médiane (pour les données numériques) ou le mode (pour les données numériques ou catégorielles),
- ➡ l'**imputation avancée**, basée sur des modèles d'imputation.

A noter que certains modèles prennent en charge la présence de valeurs manquantes (k-NN, naive Bayes...) de façon native, mais il est classique de rendre systématique le traitement de ces valeurs manquantes.

Préparation et feature engineering

Nettoyage avec Spark

Valeurs manquantes `dropna()` permet d'éliminer les lignes contenant des valeurs manquantes
 `fillna()` permet de remplacer les valeurs manquantes par une valeur choisie

Valeurs aberrantes

```
lower_bound = 0 # Valeur minimale
upper_bound = 100 # Valeur maximale
df_cleaned = df.filter(
    (col("column_name") >= lower_bound)
    &
    (col("column_name") <= upper_bound)
)
```

Préparation et feature engineering

Détection et correction des valeurs aberrantes

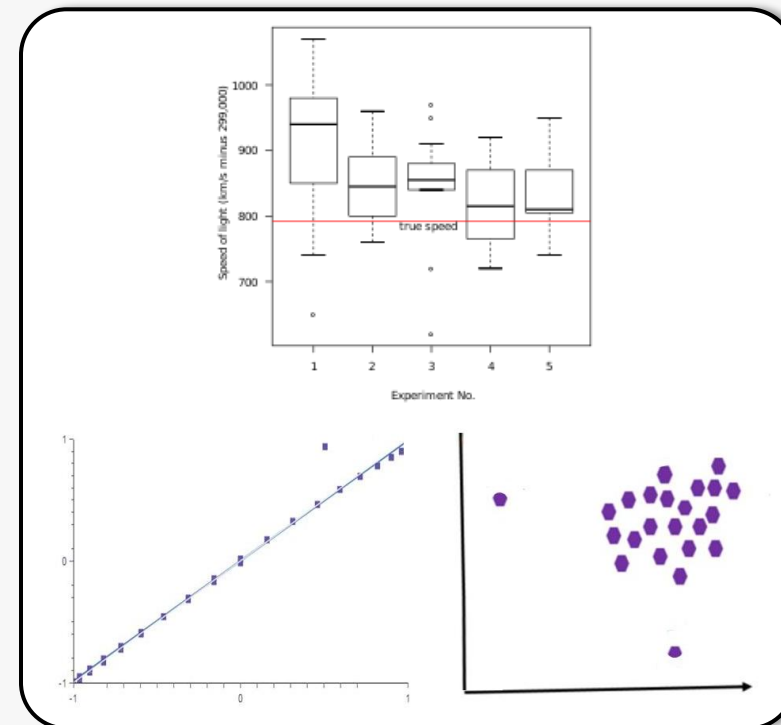
Certaines valeurs sont très « particulières » vis-à-vis de la distribution de données : elles sont atypiques. Il s'agit d'un argument **statistique**. La majorité des méthodes de détection sont de type **quartiles** ou **écart-type**.

Certaines valeurs posent problème vis-à-vis de notre connaissance de la variable représentée : elles sont aberrantes. Il s'agit d'un argument **métier**.

La plupart des valeurs aberrantes sont atypiques (mais pas toujours en cas de problème systématique).

Sucre pour 100g	Gras pour 100 g
0	20
33	113
70	60
-18	3
21	68

Âge (ans)
31
147
23
41
37



Préparation et feature engineering

Détection et correction des valeurs aberrantes

Pour identifier les valeurs atypiques et aberrantes (anomalies) dans un DataFrame Spark, on utilise différentes techniques statistiques et méthodes de visualisation assez classiques, qui ont leur syntaxe Spark.

La première chose à faire est de se demander s'il existe des **critères « métiers »** permettant de dire qu'une valeur est aberrante (exemple : une quantité de produits négative, un âge d'une personne supérieur à 130 ans, un bâtiment à 400 étages...). Parfois, l'incohérence se situe entre plusieurs valeurs d'une même ligne (exemple : temps quotidien consacré à dormir 8h, temps quotidien consacré à travailler 8h, temps quotidien consacré aux repas 1h, temps quotidien consacré aux loisirs 8h).

Préparation et feature engineering

Détection et correction des valeurs aberrantes

On peut utiliser la méthode des **boîtes à moustaches** pour identifier les valeurs atypiques dans chaque colonne numérique du DataFrame. Généralement, on définit des seuils liés à l'écart interquartile afin d'identifier les valeurs qui s'en éloignent de manière significativement. Classiquement, les points de données atypiques se situent à l'extérieur de 1,5 fois l'écart interquartile (IQR) au-dessus du troisième quartile (Q3) ou en dessous du premier quartile (Q1). La fonction `approxQuantile()` de Spark permet de calculer les quartiles. D'autres visualisations, telles que les graphiques de dispersion, les histogrammes et les matrices de corrélation... permettent d'identifier visuellement les valeurs aberrantes dans les données.

Pour identifier les valeurs aberrantes qui surviennent dans plusieurs dimensions (colonnes), vous pouvez utiliser des techniques telles que l'analyse des composantes principales (PCA) pour **réduire la dimensionnalité**, puis identifier les points extrêmes dans l'espace de projection.

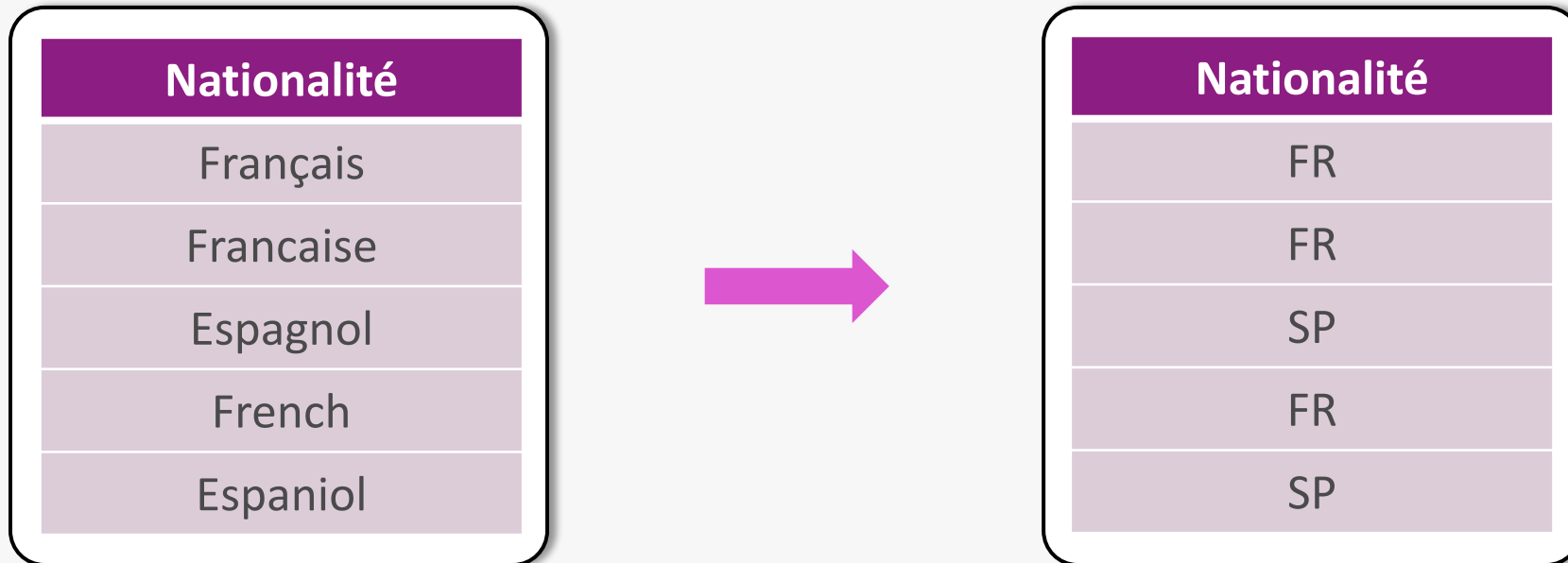
Enfin, on peut utiliser des algorithmes de clustering tels que le **DBSCAN** pour regrouper les points de données en fonction de leur densité et identifier les points isolés comme des valeurs aberrantes.

Préparation et feature engineering

Préparation des données : autres nettoyages

Certaines formes de données peuvent demander une préparation plus poussée et plus spécifique. C'est par exemple le cas des données textuelles.

Dans ce cas, les problèmes de qualité peuvent être nombreux (orthographe, casse, espaces et caractères spéciaux...) et laborieux à régler du fait de la complexité intrinsèque de la langue.



Préparation et feature engineering

Normalisation avec Spark ML

Dans PySpark, différents scalers classiques sont implémentés.

Standard Scaler

$$X'_i = \frac{X_i - \mu}{\sigma} = \frac{X_i - X_{\text{mean}}}{X_{\text{std}}}$$

StandardScaler

```
from pyspark.ml.feature import StandardScaler

scaler = StandardScaler(
    inputCol="features",
    outputCol="scaled_features",
)
scaler_model = scaler.fit(df)
df_scaled = scaler_model.transform(df)
```

Préparation et feature engineering

Normalisation avec Spark ML

Dans PySpark, différents scalers classiques sont implémentés.

Min-max Scaler

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
from pyspark.ml.feature import MinMaxScaler

scaler = MinMaxScaler(
    inputCol="features",
    outputCol="scaled_features",
)
scaler_model = scaler.fit(df)
df_scaled = scaler_model.transform(df)
```

Préparation et feature engineering

Normalisation avec Spark ML

Dans PySpark, différents scalers classiques sont implémentés.

Max-abs Scaler

$$x_{scaled} = \frac{x}{\max(|x|)}$$

```
from pyspark.ml.feature import MaxAbsScaler

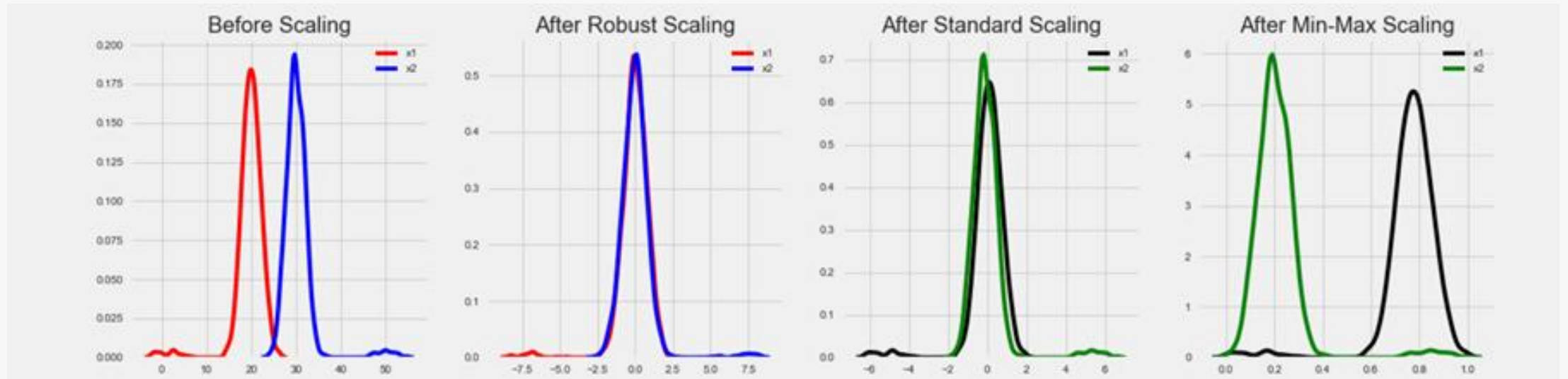
scaler = MaxAbsScaler(
    inputCol="features",
    outputCol="scaled_features",
)
scaler_model = scaler.fit(df)
df_scaled = scaler_model.transform(df)
```

Préparation et feature engineering

Normalisation avec Spark ML

Dans PySpark, différents scalers classiques sont implémentés.

Robust Scaler



```
from pyspark.ml.feature import RobustScaler

scaler = RobustScaler(
    inputCol="features",
    outputCol="scaled_features",
)
scaler_model = scaler.fit(df)
df_scaled = scaler_model.transform(df)
```

Préparation et feature engineering

Encoding avec Spark : String Indexer

```
from pyspark.ml.feature import StringIndexer

indexer = StringIndexer(
    inputCol="nom_colonne_catégorielle",
    outputCol="nom_colonne_indexée",
)

indexed_df = indexer.fit(df).transform(df)
```

Préparation et feature engineering

Encoding avec Spark : Binary Encoder

```
from feature_engine.encoding import BinaryEncoder

encoder = BinaryEncoder(cols=["nom_colonne_catégorielle"])

encoded_df = encoder.fit_transform(df)
```

Préparation et feature engineering

Encoding avec Spark : One Hot Encoder

```
from pyspark.ml.feature import OneHotEncoder

encoder = OneHotEncoder(
    inputCol="nom_colonne_indexée",
    outputCol="nom_colonne_encodée",
)

encoded_df = encoder.transform(indexed_df)
```


Feature creation avec Spark

Création de nouvelles colonnes à partir de colonnes existantes

```
from pyspark.sql.functions import col

# Créer une nouvelle colonne en appliquant une transformation
mathématique
df = df.withColumn("nouvelle_colonne", col("colonne_existante") * 2)

# Créer une nouvelle colonne en extrayant des informations à partir
d'une colonne existante
df = df.withColumn("longueur_texte", col("texte").cast("int"))
```

Feature creation avec Spark

Création de fonctionnalités à partir de valeurs agrégées

```
from pyspark.sql.functions import avg, max, min

avg_value = df.groupBy(
    "colonne_agrégation"
).agg(
    avg("colonne_valeur")
).alias("moyenne_valeur")

range_value = df.groupBy(
    "colonne_agrégation"
).agg(
    (max("colonne_valeur") - min("colonne_valeur"))
).alias(
    "étendue_valeur"
)
```

Feature creation avec Spark

Création de fonctionnalités à partir de fenêtres temporelles

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag

# Définir une fenêtre temporelle pour calculer les valeurs retardées
window = Window.partitionBy("id").orderBy("timestamp")

# Créer une nouvelle colonne avec des valeurs retardées
lag_value = lag("valeur", 1).over(window)

# Remplir les valeurs manquantes avec une valeur par défaut
df = df.withColumn("valeur_retardée", lag_value.fillna(0))
```

Feature creation avec Spark

Création de nouvelles colonnes à partir de colonnes existantes

```
from pyspark.sql.functions import col

# Créer une nouvelle colonne en appliquant une transformation mathématique
df = df.withColumn("nouvelle_colonne", col("colonne_existante") * 2)

# Créer une nouvelle colonne en extrayant des informations à partir d'une
colonne existante
df = df.withColumn("longueur_texte", col("texte").cast("int"))
```

Apprentissage

Vector assembler

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["feature1", "feature2", ...],
    outputCol="features",
)

df_preprocessed = assembler.transform(df)
```

Apprentissage

Train test split

```
train_data, test_data =  
df_preprocessed.randomSplit([0.7, 0.3], seed=42)
```

Apprentissage

Modèles

```
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(
    featuresCol="features",
    labelCol="label",
)

lr_model = lr.fit(train_data)
```

Apprentissage

Evaluation

```
from pyspark.ml.evaluation import RegressionEvaluator

predictions = lr_model.transform(test_data)

evaluator = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="rmse",
)
rmse = evaluator.evaluate(predictions)
print("RMSE:", rmse)
```


Apprentissage

Réglage des hyperparamètres

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

param_grid = ParamGridBuilder() \
    .addGrid(lr.regParam, [0.1, 0.01]) \
    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0]) \
    .build()

crossval = CrossValidator(estimator=lr,
                          estimatorParamMaps=param_grid,
                          evaluator=evaluator,
                          numFolds=5)

cv_model = crossval.fit(train_data)

best_lr_model = cv_model.bestModel
```

Apprentissage

Interprétation des résultats

```
coefficients = best_lr_model.coefficients  
intercept = best_lr_model.intercept  
print("Coefficients:", coefficients)  
print("Intercept:", intercept)
```

Industrialisation

- **Exportation du modèle**
- **Développement de l'infrastructure**
- **Déploiement du modèle**
- **Exposition d'une API**
- **Surveillance et maintenance**
- **Intégration continue et déploiement continu (CI/CD)**

Bilan : préparation et feature engineering

Fonctions PySpark classiques pour la préparation des données (rappel, 1/2)

Catégorie	Fonction/méthode	Description
Chargement	spark.read.csv()	Charge les données à partir d'un fichier CSV.
	spark.read.format()	Charge les données à partir de différents formats de fichiers (Parquet, JSON...).
	spark.read.jdbc()	Charge les données à partir d'une base de données JDBC.
	spark.readStream()	Charge les données en streaming à partir de différentes sources.
Exploration	DataFrame.printSchema()	Affiche le schéma des données.
	DataFrame.show()	Affiche les premières lignes du DataFrame.
	DataFrame.describe()	Affiche des statistiques descriptives pour les colonnes numériques.
Nettoyage	DataFrame.dropna()	Supprime les lignes contenant des valeurs manquantes.
	DataFrame.fillna()	Remplace les valeurs manquantes par une valeur spécifiée.
	DataFrame.dropDuplicates()	Supprime les lignes en double.
Transformation	DataFrame.select()	Sélectionne un sous-ensemble de colonnes du DataFrame.
	DataFrame.filter()	Filtre les lignes du DataFrame en fonction d'une condition.
	DataFrame.groupBy()	Regroupe les données en fonction des valeurs d'une ou plusieurs colonne(s).
	DataFrame.join()	Effectue une jointure entre deux DataFrames.
	DataFrame.withColumn()	Ajoute une nouvelle colonne à partir d'une transformation sur les valeurs existantes d'une colonne (similaire à .map() sur les RDD).

Bilan : préparation et feature engineering

Fonctions PySpark classiques pour la préparation des données (rappel, 2/2)

Catégorie	Fonction/méthode	Description
Feature engineering		Ajout de colonnes avec des expressions SQL ou des fonctions UDF Utilisation de fonctions intégrées comme concat(), substring(), split()...
Partitionnement	DataFrame.repartition() DataFrame.coalesce()	Réorganise les partitions du DataFrame. Réduit le nombre de partitions du DataFrame.
Enregistrement	DataFrame.write.format() DataFrame.write.saveAsTable() DataFrame.writeStream()	Ecrit les données dans différents formats de fichiers. Enregistre le DataFrame en tant que table dans le catalogue Spark SQL. Ecrit les données en streaming vers une destination.

Pipelines

Utilité

- Manière efficace d'organiser et de chaîner ensemble plusieurs étapes de traitement des données, y compris la préparation, la transformation et la modélisation, de manière séquentielle.
- Garantit que toutes les étapes sont exécutées dans un ordre spécifié
- Permet de réutiliser facilement les workflows de traitement des données.

Pipelines

Création d'un pipeline

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import LogisticRegression

indexer = StringIndexer(
    inputCol="colonne_catégorielle",
    outputCol="colonne_indexée",
)
encoder = OneHotEncoder(
    inputCol="colonne_indexée",
    outputCol="colonne_encodée",
)
assembler = VectorAssembler(
    inputCols=["colonne_numérique", "colonne_encodée"],
    outputCol="features",
)
classifier = LogisticRegression(featuresCol="features", labelCol="label")

pipeline = Pipeline(stages=[indexer, encoder, assembler, classifier])
```

Pipelines

Entraînement d'un pipeline

```
# Diviser les données en ensembles d'apprentissage et de test
train_data, test_data = df.randomSplit([0.7, 0.3], seed=42)

# Entraîner le pipeline sur les données d'apprentissage
model = pipeline.fit(train_data)
```


Pipelines

Utilisation du pipeline pour faire des prédictions

```
# Faire des prédictions sur les données de test
predictions = model.transform(test_data)

# Afficher les prédictions
predictions.select("features", "label", "prediction").show()
```

Pipelines

Evaluation des performances

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Évaluer les performances du modèle
evaluator = BinaryClassificationEvaluator(
    rawPredictionCol="rawPrediction",
    labelCol="label",
)
accuracy = evaluator.evaluate(predictions)
print("Accuracy:", accuracy)
```

Machine Learning avec Spark

TP Spark ML pour l'entraînement et l'inférence



Régression linéaire
ACP

Machine learning

Etude de code



Spark et udf pour l'inference :
Extraction de features
d'images à partir d'un modèle
de Deep Learning déjà
entraîné (inférence seule)

Machine learning

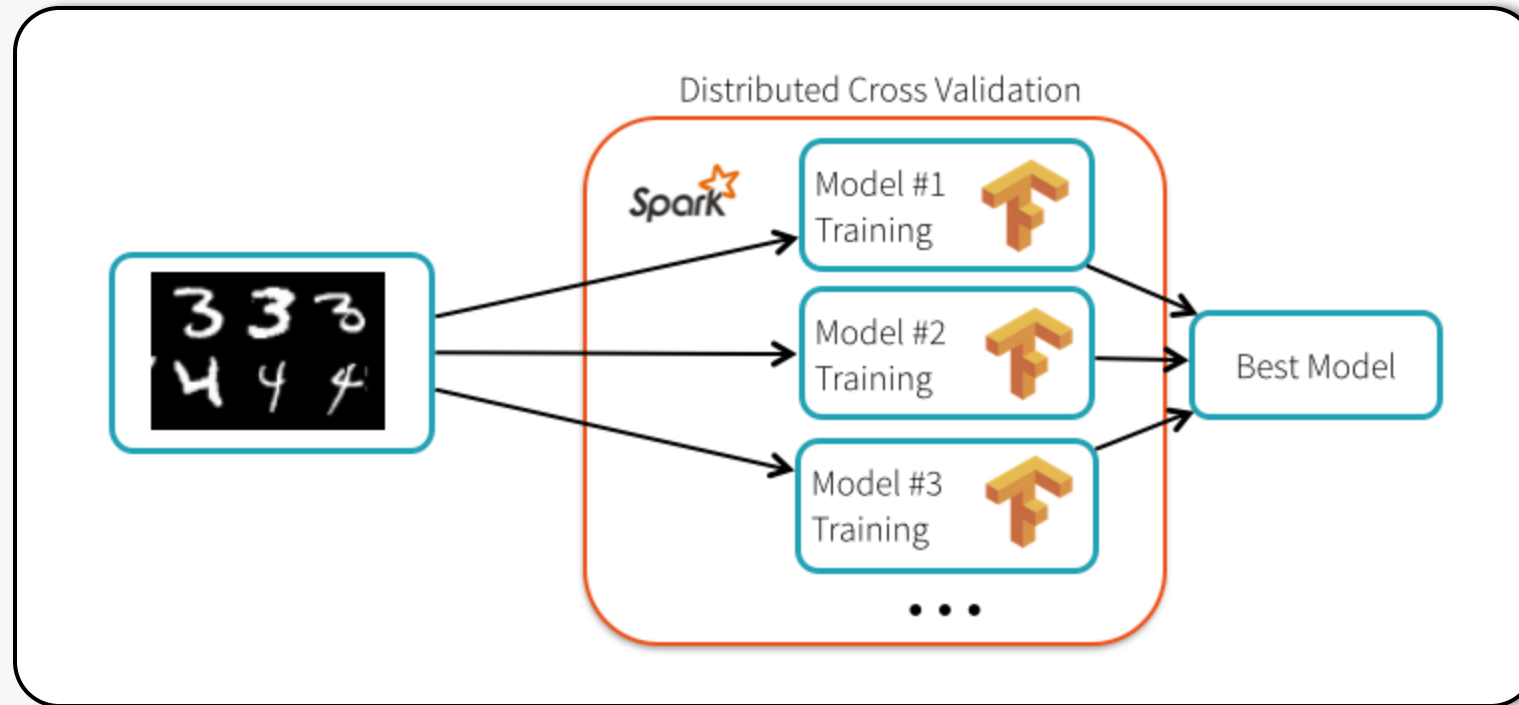
Etude de code



MLFlow

Machine learning

Deep Learning



Question

Composants de Spark ML

Quels composants de Spark ML permettent de chaîner ensemble plusieurs étapes de traitement des données et d'apprentissage dans un flux unique ?

- ☐ Les Estimators
- ☐ Les Models
- ☐ Les Transformers
- ☐ Les Pipelines

Question

Composants de Spark ML

Quelle abstraction de Spark ML est utilisée pour représenter les algorithmes d'apprentissage ajustables sur des données pour produire un modèle ?

- ☐ Estimator
- ☐ Model
- ☐ Transformer
- ☐ Pipeline

Question

Composants de Spark ML

De quel type est la composée de deux transformers dans Spark ML ?

- ☐ Transformer
- ☐ Function
- ☐ Model
- ☐ Estimator

06

Graphes avec Spark

Graphes avec Spark

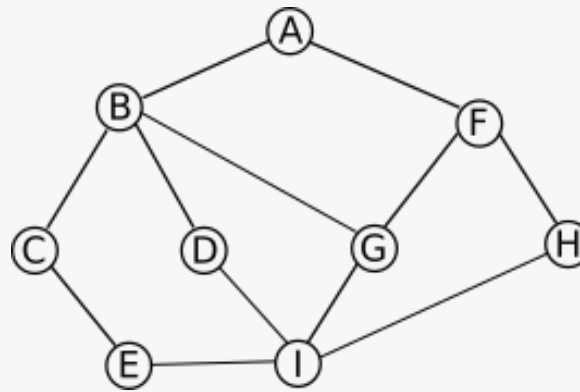
Principes

- Les graphes représentent des structures de données composées de sommets et d'arêtes, où les sommets représentent des entités et les arêtes représentent les relations entre ces entités. Ils constituent une abstraction flexible pour décrire les relations entre des objets discrets.

- Formellement, un graphe $G = (V, E)$ est défini par deux ensembles :

- ➡ V ensemble fini d'éléments distincts représentant les sommets/nœuds/points du graphe.

- ➡ E ensemble de paires de $V \times V$ non ordonnées ou ordonnées de sommets, représentant les arêtes/connexions entre les sommets du graphe.

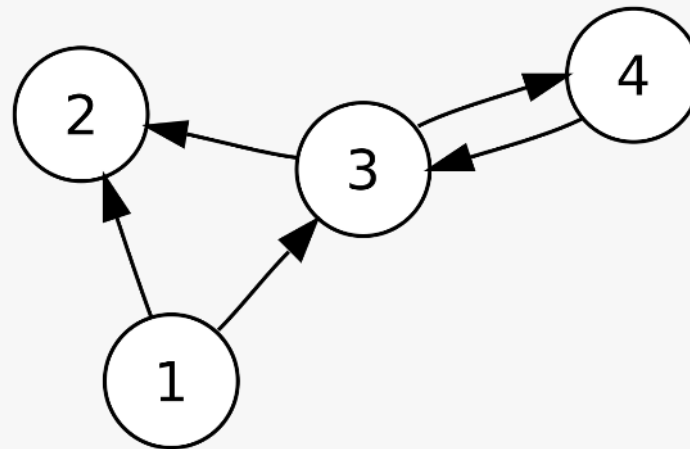


Graphes avec Spark

Principes

Les arêtes peuvent avoir différentes caractéristiques selon le type de graphe :

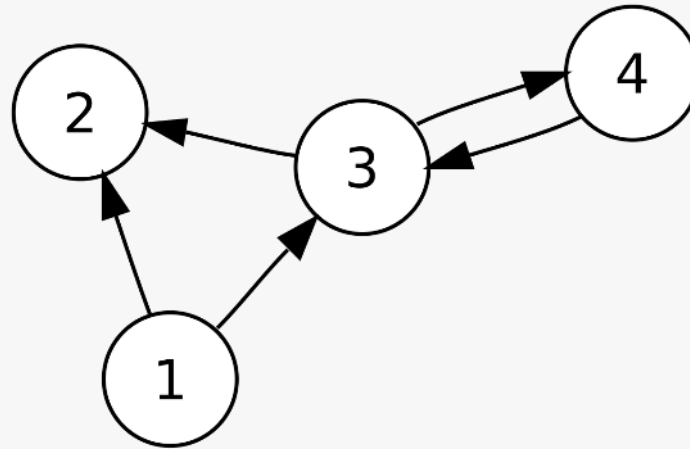
- ➡ Dans un graphe **non orienté**, les arêtes ne sont pas directionnelles.
- ➡ Dans un graphe **orienté**, chaque arête a une direction spécifique.
- ➡ Dans un graphe **pondéré**, chaque arête est associée à un poids ou une valeur.
- ➡ Dans un graphe **étiqueté**, les arêtes sont associées à des étiquettes ou des valeurs.



Question

Types de graphes

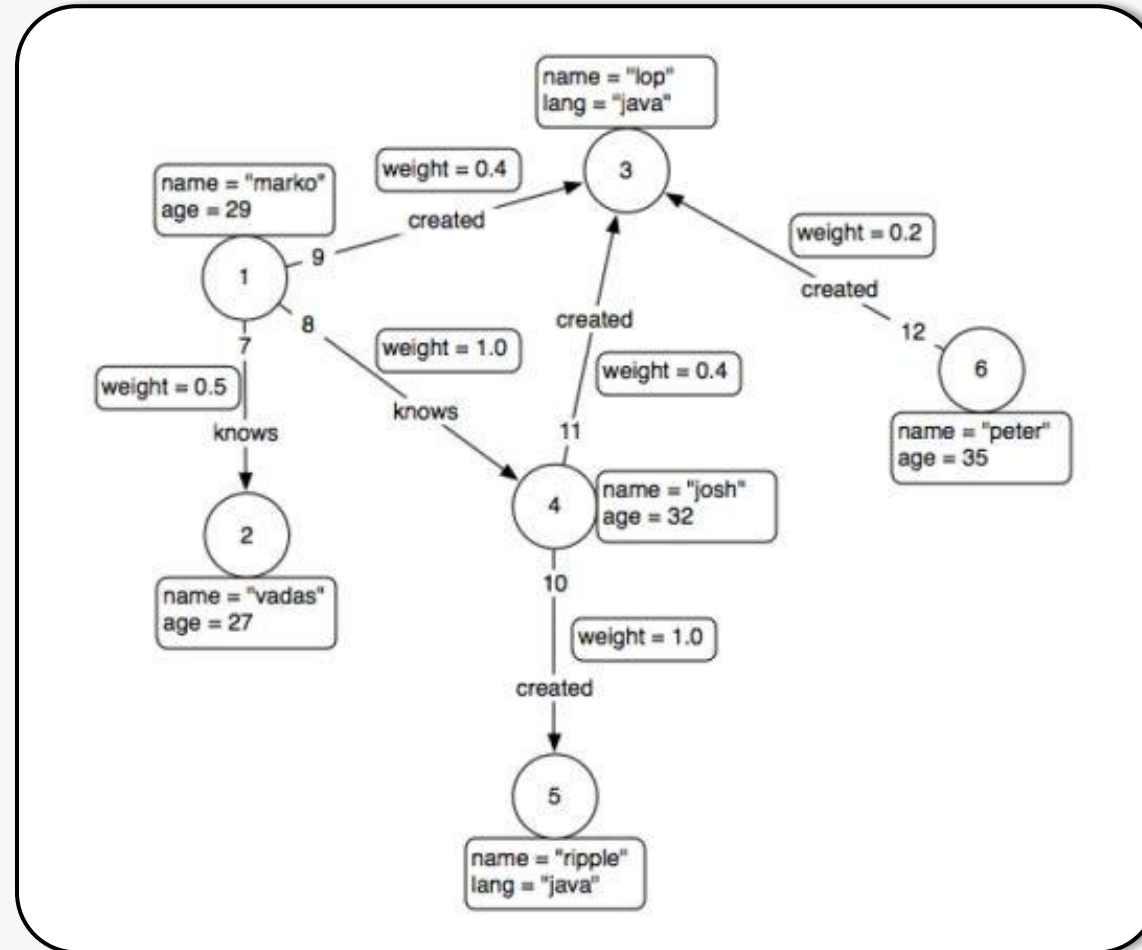
Proposer une légende pour le schéma précédent :



- ☐ Graphe orienté
- ☐ Graphe pondéré
- ☐ Graphe étiqueté
- ☐ Graphe acyclique

Graphes avec Spark

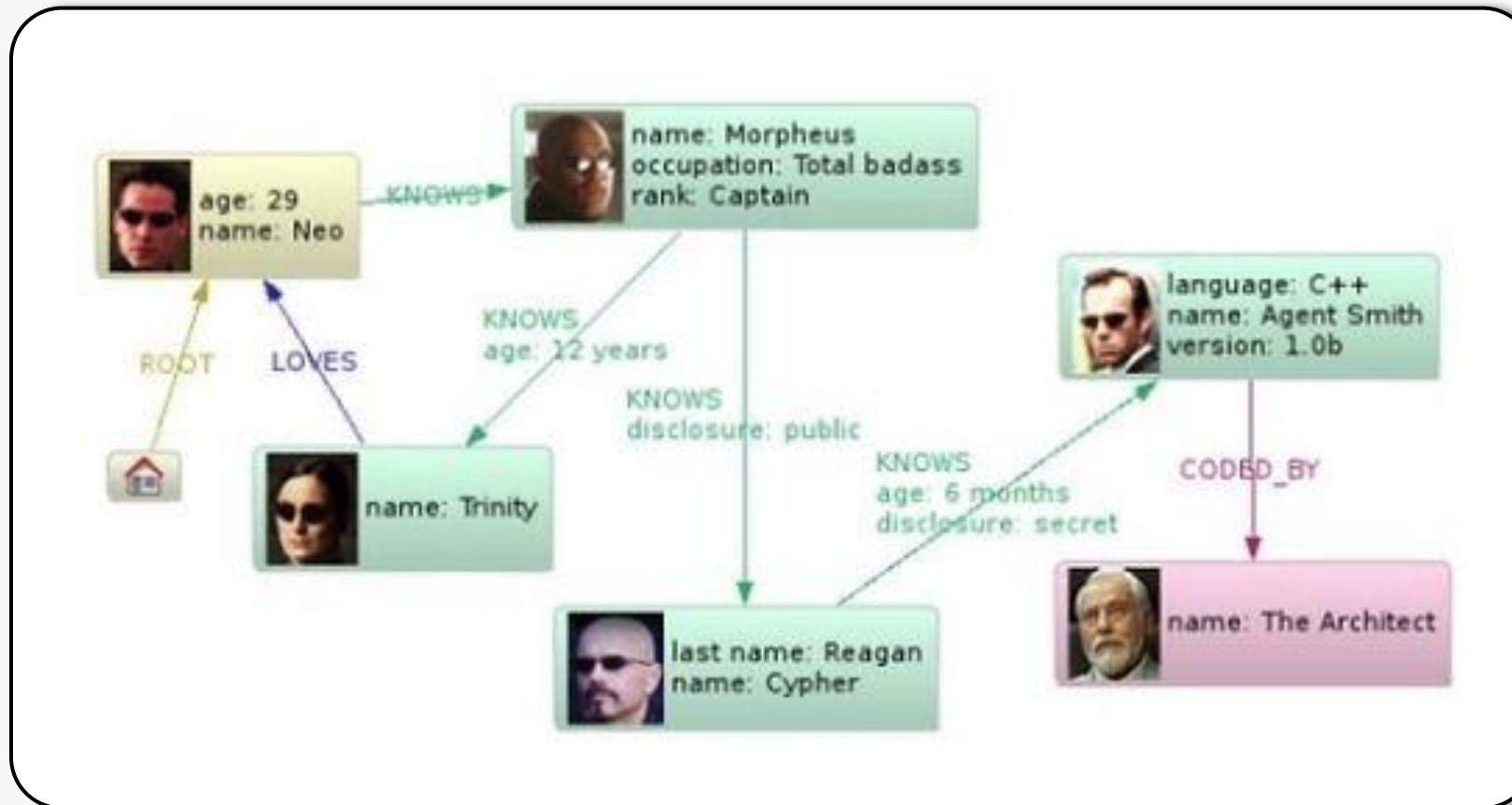
Exemples illustrés



<https://www.infoq.com/fr/articles/graph-nosql-neo4j/>

Graphes avec Spark

Exemples illustrés



<https://www.infoq.com/fr/articles/graph-nosql-neo4j/>

Graphes avec Spark

Application : moteurs de recherche

PageRank

- Évalue la pertinence d'une page web en analysant les liens entrants de manière à attribuer un score de popularité, favorisant ainsi les pages reliées à un plus grand nombre de sources de qualité.
- Internet est modélisé comme un graphe probabiliste.
- Pas d'apprentissage automatique.
- Mais, quid du point d'entrée dans cette représentation de l'autorité et de la popularité des pages via leurs liens ?

Pour aller plus loin sur PageRank, par exemple :

Marche aléatoire, état stationnaire, matrices irréductibles et graphes connexes, théorème de Perron-Frobenius

<https://www.concours-centrale-supelec.fr/CentraleSupelec/2021/PSI/M051.pdf>

Graphes avec Spark

Application : moteurs de recherche

RankBrain

- Il vise à comprendre le sens et la signification des requêtes de recherche.
- Il s'efforce de comprendre le **langage naturel** et les **intentions** derrière les requêtes des utilisateurs. Cela est particulièrement important car de nombreuses recherches sont formulées de manière complexe et ne reposent pas uniquement sur des mots-clés spécifiques. RankBrain analyse le contexte et les relations sémantiques entre les mots pour fournir des résultats de recherche plus pertinents. Il peut traiter des requêtes qu'il n'a jamais vues auparavant.
- Il parvient à faire cela en se basant sur des modèles d'apprentissage automatique (plus précisément de NLP).

RankBrain complète PageRank en améliorant la compréhension des requêtes complexes et en offrant des résultats plus précis et pertinents pour les utilisateurs.

Graphes avec Spark

Graphes et MapReduce

Application des moteurs de calcul parallèles tels que MapReduce ou Spark sur les grands graphes : pas immédiate !

- Les graphes ne peuvent pas être traités efficacement sur une seule machine en raison de leur taille ou du coût associé MAIS les moteurs MapReduce/Spark ne sont pas adaptés à ces problèmes (en raison du calcul du graphe piloté par les données et du manque de structure des graphes).
- MapReduce et Spark fonctionnent selon une approche **record-centric**, traitant les données indépendamment sur différentes machines. Les difficultés de parallélisation résident dans le fait que la structure des calculs est motivée par la topologie des graphes (noeuds et arêtes), qui n'est pas connue à l'avance, rendant difficile la fourniture d'un équilibrage de charge efficace.
- Les algorithmes de graphes se concentrent généralement sur l'exploration de structures de graphes et la notion de voisinage est cruciale pour les calculs. Cependant, la notion de localité sur les calculs et l'accès aux données est floue, ce qui complique la résolution efficace des problèmes.
- De nombreux problèmes peuvent être résolus avec une approche centrée sur les sommets, exprimant le calcul du point de vue d'un sommet dans le graphe.

Graphes avec Spark

Moteurs de traitement de graphes

À partir de 2009-10, plusieurs moteurs de traitement de graphes ont émergé : Pregel, GraphLab, PowerGraph, GraphX, Gelly...

- ➡ Ils imposent des restrictions sur les types de calculs possibles.
- ➡ Ils introduisent de nouvelles techniques de partitionnement de graphes pour améliorer les performances.
- ➡ Ils démontrent souvent de meilleures performances que les systèmes de données parallèles traditionnels.
- ➡ Ils adoptent une perspective des graphes centrée sur les sommets, ce qui permet de garantir le parallélisme en partitionnant le graphe et en réalisant des opérations sur les sommets.

Graphes avec Spark

Pregel

- Développé chez Google pour traiter des tâches telles que le calcul du PageRank, la recherche des chemins les plus courts, et les algorithmes de correspondance bipartite, qui consistent à fournir un sous-ensemble d'arêtes d'un graphe bipartite sans points de terminaison communs.
- Présenté à Sigmod 2010 par G. Malewicz et al.
- Repose sur le modèle Bulk Synchronous Parallel (BSP) avec un passage de message.
- Adopte une approche Maître-Esclave et utilise le partitionnement de graphe par hachage.
- Intègre une tolérance aux pannes par point de contrôle.
- Implémentation en C++.

Graphes avec Spark

Modèle Bulk Synchronous Parallel (BSP)

- Modèle de calcul parallèle qui divise le calcul en une série d'itérations synchronisées, appelées super-étapes.
- Dans chaque super-étape, les nœuds du calcul parallèle exécutent des calculs locaux sur leurs données, puis échangent des messages avec d'autres nœuds. Une fois que tous les nœuds ont terminé leurs calculs locaux et leurs échanges de messages, la super-étape suivante commence.
- Ce modèle offre une structure claire pour le parallélisme de données et la communication entre les processus. Il permet également de contrôler facilement la progression de l'algorithme à travers les super-étapes.
- Le modèle BSP est largement utilisé dans le domaine du calcul distribué et parallèle, et il a été popularisé par des frameworks comme Pregel et d'autres systèmes de traitement de graphes distribués.

Graphes avec Spark

BSP & Pregel

- Dans Pregel (ainsi que dans son équivalent open source, Apache Giraph), le modèle Bulk Synchronous Parallel (BSP) est au cœur de son fonctionnement.
- Ce modèle BSP offre une abstraction puissante pour la programmation de graphes distribués, permettant aux développeurs de concevoir des algorithmes efficaces tout en se souciant moins des détails de la communication et de la synchronisation entre les nœuds.

Graphes avec Spark

BSP & Pregel

Division en super-étapes

Chaque super-étape consiste en un ensemble d'opérations de calcul et d'échange de messages entre les nœuds du graphe.

Calcul local

Pendant chaque super-étape, chaque nœud effectue des calculs locaux sur les données qu'il détient. Ces calculs peuvent inclure des mises à jour de l'état du nœud ou des messages à envoyer à d'autres nœuds.

Échange synchronisé de messages

À la fin de chaque super-étape, les nœuds échangent des messages avec leurs voisins dans le graphe. Ces messages peuvent contenir des mises à jour de l'état ou des informations nécessaires pour les calculs futurs.

Synchronisation

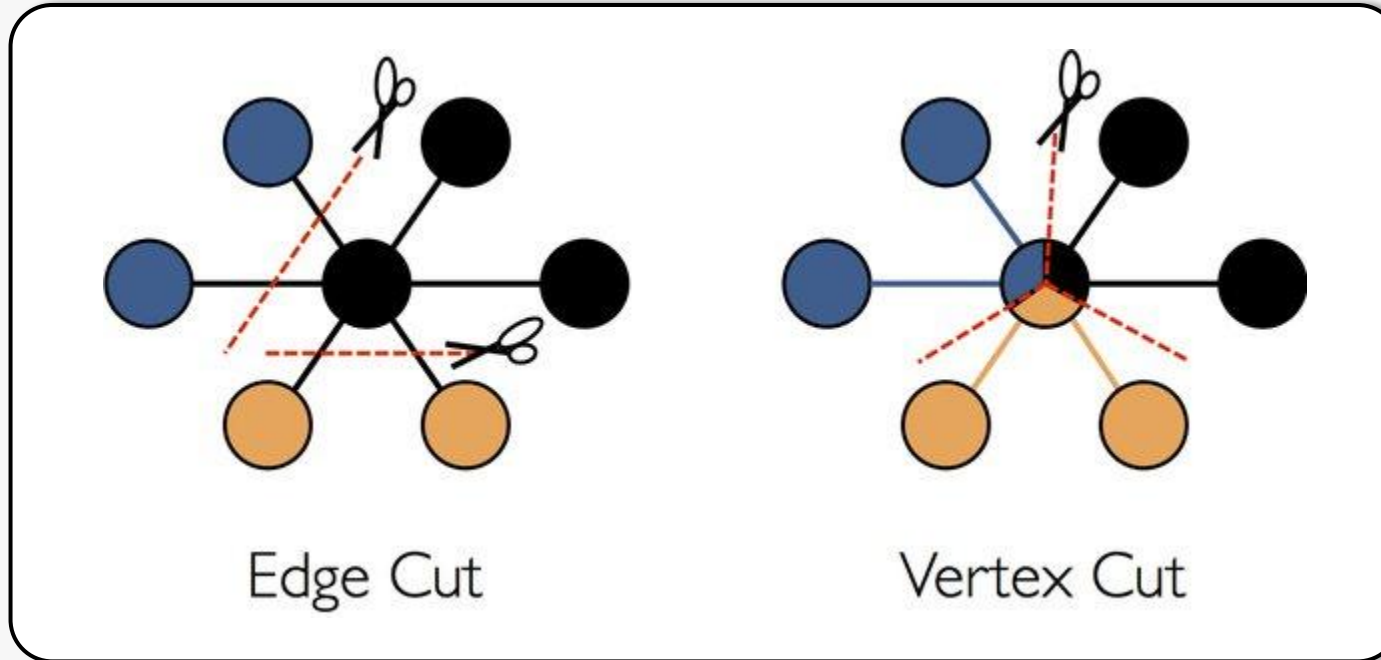
Une fois tous les messages échangés, la super-étape se termine et une nouvelle super-étape commence. Cette synchronisation permet de garantir que tous les nœuds effectuent les mêmes opérations dans le même ordre, assurant ainsi la cohérence du calcul.

Itération

Le processus se répète jusqu'à ce qu'un critère d'arrêt prédéfini soit atteint, tel qu'un nombre maximum d'itérations ou une convergence des résultats.

Graphes avec Spark

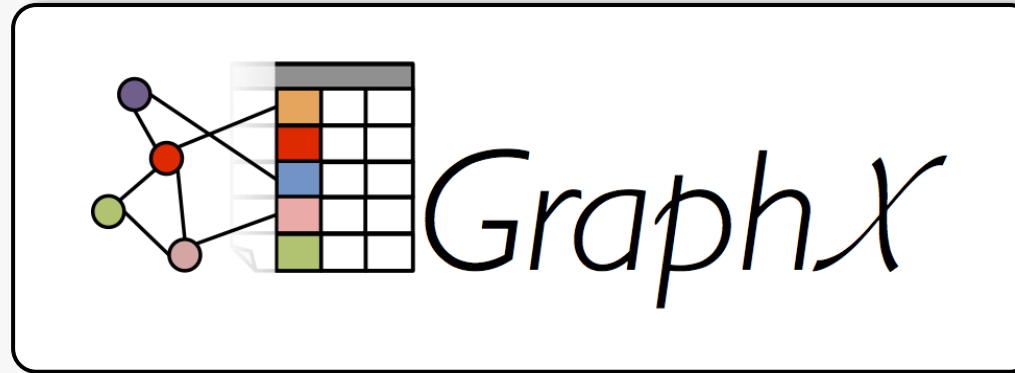
Partitionnement de graphes



Source : <https://www.quora.com/What-is-a-cut-edge-and-a-cut-vertex-in-a-complete-graph>

Graphes avec Spark

GraphX : introduction



- GraphX est un composant d'Apache Spark.
- Il étend le RDD Spark en introduisant une nouvelle abstraction de graphe.
- Cette abstraction permet de représenter un multigraphe dirigé avec des propriétés attachées à chaque sommet et arête.
- GraphX unifie l'ETL, l'analyse exploratoire et le calcul de graphe itératif au sein d'un seul système.

Graphes avec Spark

GraphX : caractéristiques et fonctionnalités

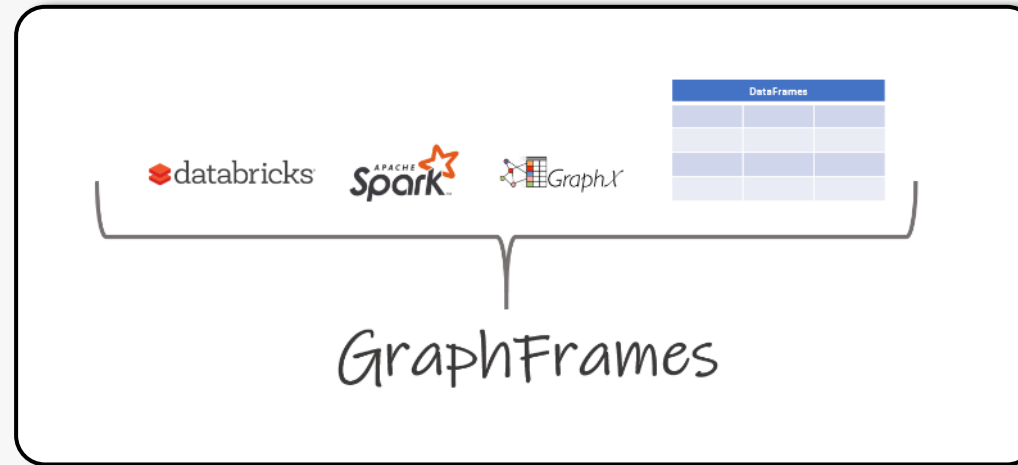
- GraphX permet la visualisation des mêmes données sous forme de graphes et de collections.
- Il permet également de transformer et de joindre efficacement des graphes avec des RDD.
- Les utilisateurs peuvent écrire des algorithmes de graphes itératifs personnalisés en utilisant l'API Pregel.

Optimisations et techniques de traitement :

- ➡ GraphX est basé sur le modèle BSP (Bulk Synchronous Parallel).
- ➡ Il s'appuie sur plusieurs techniques d'optimisation pour traiter les données retardataires.
- ➡ GraphX adopte la notion de coupe du sommet.
- ➡ Il permet de gérer de nombreuses opérations sans duplication de données, ce qui facilite la courbe d'apprentissage et garantit une performance rapide.

Graphes avec Spark

GraphFrames : introduction



- GraphFrames est une bibliothèque de traitement de graphes basée sur les DataFrames pour Apache Spark.
- Elle tire parti de l'évolutivité et des performances des DataFrames et offre une API uniforme pour Scala, Java et Python.

Graphes avec Spark

GraphFrames : caractéristiques et fonctionnalités

- GraphFrames fournit une API déclarative.
- Implémentée au-dessus de Spark SQL, elle optimise l'ensemble du calcul en utilisant l'optimisation des jointures et l'algorithme de sélection des vues.
- Un GraphFrame se compose d'une trame de données de sommets, d'arêtes et de triplets, tout comme GraphX, ainsi que d'un motif pour faire correspondre les formes des sous-graphes.
- Les motifs sont exprimés dans un langage similaire à Cypher, et les UDF (User Defined Functions) en ligne sont acceptées.

Défis et améliorations :

- ➡ Le principal défi de GraphFrames réside dans la planification des requêtes.
- ➡ Des améliorations ont été apportées à l'optimiseur Catalyst de Spark SQL.
- ➡ Le planificateur de requêtes est implémenté comme une couche au-dessus de Catalyst, utilisant des modèles en entrée, collectant des statistiques à l'aide des API Catalyst et émettant un plan logique Catalyst.
- ➡ Les vues matérialisées sont utilisées pour optimiser le traitement des requêtes et peuvent être créées manuellement par l'utilisateur.
- ➡ Le système peut également proposer des vues classées en fonction de critères tels que le coût et la taille.

Les formats de données

TP sur les graphes



**Manipulation de graphes,
algorithmes classiques (page
rank, shortest paths)**

07

The end...

Pour terminer

TP final (si vous avez du temps)



**Variations dans des
portefeuilles boursiers**

Compléments

Ressources utiles pour les bases de Spark

Tuto :

<https://spark.apache.org/docs/latest/quick-start.html#basics>

<https://sparkbyexamples.com/>

Et bien sûr la documentation officielle de Spark, notamment la documentation des API :

<https://spark.apache.org/docs/latest/>

<https://spark.apache.org/docs/latest/api/python/index.html>

https://spark.apache.org/docs/latest/api/python/getting_started/quickstart_df.html

Il y a aussi beaucoup de vidéos. A titre personnel je ne suis pas fan du format vidéo car c'est très compliqué d'adapter la vitesse, mais parmi les ressources vidéo que j'ai consultées, les meilleures furent celles de la plateforme Pluralsight (licence payante).

Complément 1

Retour un peu plus détaillé sur Spark UI

driver node port

Spark 3.4.0

Jobs Stages Storage Environment Executors SQL / DataFrame

Spark UI Tutorial application UI

Spark Jobs (?)

User:
Total Uptime: 26 s
Scheduling Mode: FIFO
Completed Jobs: 4

Event Timeline

Completed Jobs (4)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
3	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2023/06/04 13:47:34	0.2 s	1/1 (1 skipped)	1/1 (4 skipped)
2	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2023/06/04 13:47:33	0.5 s	1/1	4/4
1	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2023/06/04 13:47:33	0.2 s	1/1	3/3
0	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2023/06/04 13:47:30	2 s	1/1	1/1

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Complément 1

Retour un peu plus détaillé sur Spark UI

Pourquoi utiliser la Spark UI ?

- Diagnostic des erreurs
- Contrôle de l'utilisation des ressources et optimisation des performances

Plus précisément, les principales fonctionnalités de la Spark UI :

Analyse de la performance : l'UI permet d'identifier les tâches lentes ou les stages qui prennent plus de temps que prévu.

Suivi de l'utilisation des ressources : l'UI permet de vérifier l'utilisation de la mémoire et du processeur par les différents exécuteurs et les ajuster pour éviter une surcharge de certaines ressources.

Visualisation des bottlenecks : essentiellement à partir des informations sur les shuffles, les tâches échouées, et l'utilisation des ressources.

Suivi de l'exécution en temps réel

Complément 1

Retour un peu plus détaillé sur Spark UI

« Ajuster », qu'est-ce que ça veut dire en fait ? -> Ajuster les ressources allouées à Spark.

- Spécifier la quantité de mémoire à allouer pour chaque executor (avec `spark.executor.memory`).

Exemple : `--conf spark.executor.memory=4g`

- Déterminer le nombre de cœurs CPU à allouer à chaque executor (avec `spark.executor.cores`).

Exemple : `--conf spark.executor.cores=4`

- Idem pour le driver.

- Ici bien entendu, on ne modifie pas directement la mémoire physique du serveur, mais on spécifie la quantité de mémoire virtuelle que chaque exécuteur peut utiliser, qui peut être utilisée pour stocker des données en mémoire, effectuer des calculs, et exécuter des tâches.
- Note : `spark.executor.memory` définit la quantité de mémoire que chaque exécuteur peut utiliser pour stocker les données. Pour définir la mémoire supplémentaire pour gérer les tâches autres que les données (objets natifs de notre langage, fichiers temporaires...), on utilise `spark.executor.memoryOverhead`.
- Enfin, on peut aussi jouer sur `spark.sql.shuffle.partition`.

Complément 1

Retour un peu plus détaillé sur Spark UI

« Ajuster », qu'est-ce que ça veut dire en fait ? -> Parfois, revenir sur le code.

- Par exemple, avec des `broadcast()`, `persist()`...

Complément 2

Apports de Spark 3

En résumé – performances

- ➡ Un moteur d'exécution vectorisé, qui peut améliorer considérablement les performances de certains types de requêtes.
- ➡ Framework Adaptive Query Execution (AQE), qui optimise dynamiquement l'exécution des requêtes à l'exécution en utilisant :
 - Un coalescing dynamique des partitions de shuffle, regroupant les tâches petites et contiguës.
 - Un changement dynamique des stratégies de jointure, en modifiant la stratégie de jointure en fonction des statistiques d'exécution.
 - Une optimisation dynamique des jointures de type "skew", en optimisant les jointures avec des données déséquilibrées.

Complément 2

Apports de Spark 3

AQE

Depuis Spark 3.0, AQE permet d'adapter dynamiquement le partitionnement en fonction des statistiques réelles des données.

Repartitionnement dynamique des jointures (si Spark détecte un déséquilibre des partitions, il ajuste leur taille à la volée). Par exemple, si une table est beaucoup plus petite que l'autre, Spark peut utiliser Broadcast Join au lieu d'un Shuffle Join.

Fusion des petites partitions et éclatement des partitions trop grandes.

Complément 2

Apports de Spark 3

En résumé – nouvelles fonctionnalités

- Amélioration du support pour les sources de données modernes telles que Delta Lake et amélioration du support pour les anciennes sources telles que Parquet/CSV, ainsi que de nombreuses améliorations des connecteurs.
- Extension du support SQL, notamment une meilleure gestion des valeurs nulles et un meilleur support pour les types de données complexes et la conformité ANSI SQL.
- Des améliorations significatives des API Pandas ont été introduites dans Spark 3, notamment avec l'ajout d'indications de type Python pour mieux typer les entrées/sorties des UDF. De plus, Spark 3 a enrichi les Pandas UDF avec de nouvelles fonctionnalités, en particulier SCALAR_ITER, qui permet d'itérer efficacement sur des batches de données plutôt que de traiter ligne par ligne. Meilleure gestion des erreurs Python, simplifiant les exceptions PySpark.
- Une nouvelle interface utilisateur pour le streaming structuré.
- Des accélérations allant jusqu'à 40 fois pour l'appel des UDF en R.
- Prise en charge des committers S3A à haute performance.

Complément 2

Apports de Spark 3

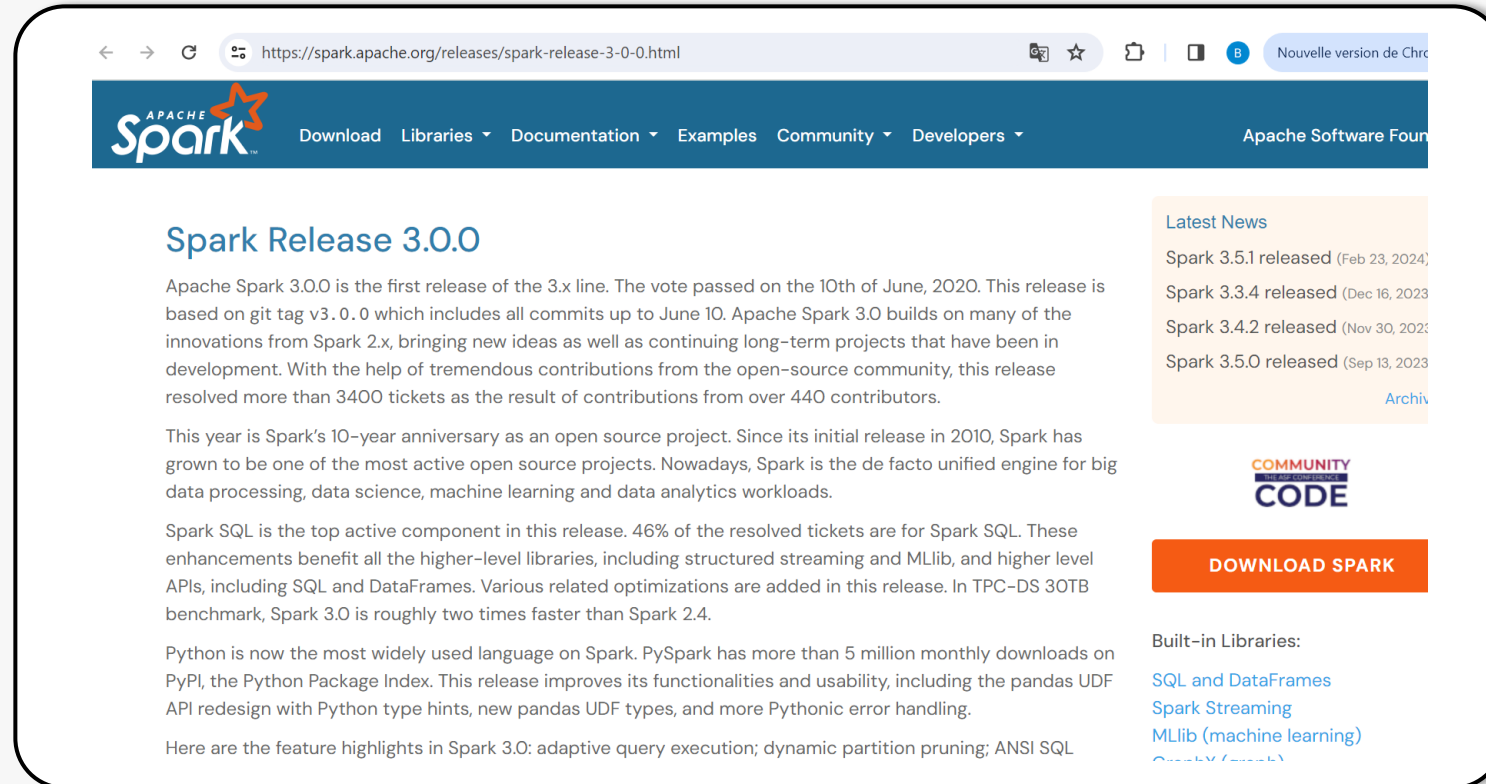
En résumé – changements de comportement et dépréciations

- Dépréciation de UserDefinedAggregateFunction (UDAF).
- Changement de comportement de `spark.emptyDataFrame`, où le DataFrame vide est maintenant optimisé pour utiliser LocalRelation au lieu d'un ensemble de données distribué résilient (RDD) vide.
- Dépréciation de UserDefinedFunction non typée, également connue sous le nom de `udf(AnyRef, DataType)`.
- Changement de schéma lors du regroupement sur une colonne imbriquée en utilisant `groupBy`.
- Changement de schéma lors de l'utilisation de `groupByKey`.
- Prise en charge dépréciée de Python 2.
- Prise en charge dépréciée de R < 3.4.

Complément 2

Apports de Spark 3

Pour une vue exhaustive :



<https://spark.apache.org/releases/spark-release-3-0-0.html>

Compléments [2:5]

Quelques compléments possiblement utiles disponibles

- Complément sur Kafka
- Complément Docker pour la conteneurisation
- Complément sur la JVM

Bilan

**Ensemble des
compétences
mobilisées**

Bilan

Point sur les compétences importantes

Concepts fondamentaux

- Historique de Spark
- Architecture logique, distribution, composants
- Actions et transformations
- RDD Lineage
- Notions de résilience, et de distribution
- Différence entre les API RDD et Dataframes (et Datasets)
- Anatomie d'une application (Spark Driver, Executor, Jobs, stages, tasks)
- Notion de closure

Bilan

Point sur les compétences importantes

Prise en main de Spark

- Exécution de commandes Spark
- Création et affichage de RDD
- Opérations sur les RDD : prise d'éléments, comptage...
- Immutabilité et transformation des RDD
- Tri des RDD
- Opérations ensemblistes sur les RDD
- Existence de Spark Java et Scala
- Opérations sur les DataFrames : création d'un DataFrame à partir d'un fichier CSV, comptage des entrées dans un DataFrame, filtrage et regroupement des données dans un DataFrame, tri et requêtes conditionnelles dans un DataFrame
- Ressources utiles (dont référence de l'API Spark DataFrame)

Bilan

Point sur les compétences importantes

Lecture et écriture de données en Spark et production, Spark SQL

- Chargement d'un fichier CSV dans un DataFrame Spark
- Affichage de la structure du DataFrame avec `printSchema()`
- Utilisation de l'en-tête du fichier, activation de l'option `header` pour obtenir les noms de colonnes
- Activation de `InferSchema()` pour déduire automatiquement les types de données
- Déclaration explicite d'un schéma avec un `StructType`
- Création d'une vue temporaire et affichage des données en utilisant SQL
- Utilisation de SQL DataFrame pour représenter les données de manière tabulaire
- Écriture dans un fichier Parquet, utilisation de `write.mode("overwrite").parquet()` pour enregistrer le DataFrame dans un fichier Parquet.

Bilan

Point sur les compétences importantes

Transformation de données et actions, Spark SQL

- Lazy vs eager design, lazy evaluation
- Présentation du Catalyst Optimizer et de ses optimisations
- Description des actions et des transformations, avec des exemples
- Transformations et syntaxe SQL : sélection (select), regroupement (groupBy), et tri (orderBy)
- Requêtes SQL : agrégations avec agg, jointures avec join
- Compréhension des opérations de shuffle et de leur fonctionnement
- Ordre des transformations
- Optimisation des performances (utilisation de `DataFrame.repartition()`, `DataFrame.coalesce()`, réglage de `spark.memory.fraction`, caching/persistance)

Bilan

Point sur les compétences importantes

Spark Streaming

- Streaming en temps réel avec Spark Structured Streaming
- Réception des données en streaming avec `awaitTermination`
- Traitement de flux de données avec Structured Streaming Dataframe
- Lecture de sources de données avec `readStream`
- Transformation de flux avec des opérations comme `select`, `filter`, `groupBy...`
- Fenêtres temporelles avec `window`
- Agrégations en temps réel avec `agg` ou `groupBy`
- Écriture de résultats en streaming avec `writeStream`
- Nom de la requête avec `queryName`
- Fréquence de traitement avec `trigger`
- Mention à l'oral d'intégration possible avec d'autres outils et services tels que Kafka

Fin de la formation

Merci !

Retour sur vos questions

Que fait `spark.read_csv` ? En particulier, charge-t-il toutes les données en mémoire ?

- Spark lit les métadonnées du fichier (emplacement, structure, éventuellement le schéma), crée un DataFrame logique qui décrit comment accéder aux données. Il ne lit pas encore toutes les lignes du CSV (lazy).
- Au-delà, Spark travaille de façon distribuée :
 - Les fichiers restent sur disque (local, HDFS, S3, ADLS...).
 - Les partitions sont lues progressivement par les executors.
 - Seules les données nécessaires à l'opération en cours sont chargées.
- Plus précisément, Spark va :
 - Obtenir la taille du fichier.
 - Le découper en "splits" logiques (par exemple des blocs de 128 Mo).
 - Affecter chaque split à un executor.
 - Chaque executor lit directement sa portion du fichier.